

bok25

基
礎

課、計算機網絡





TCP概述

TCP特点:

TCP是面向连接的传输层协议

在使用TCP之前必须建立连接，传送完后释放连接
每一条TCP连接必须是点对点的

TCP提供可靠交付的服务

通过TCP连接传送的数据，无差错、不丢失、不重复，且按序到达

TCP提供全双工通信

允许通信双方的应用进程在任何时候都能发送数据

TCP面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流
TCP并不知道所传送的字节流的含义



TCP概述

TCP特点:

TCP是面向连接的传输层协议

在使用TCP之前必须建立连接，传送完后释放连接
每一条TCP连接必须是点对点的

TCP提供可靠交付的服务

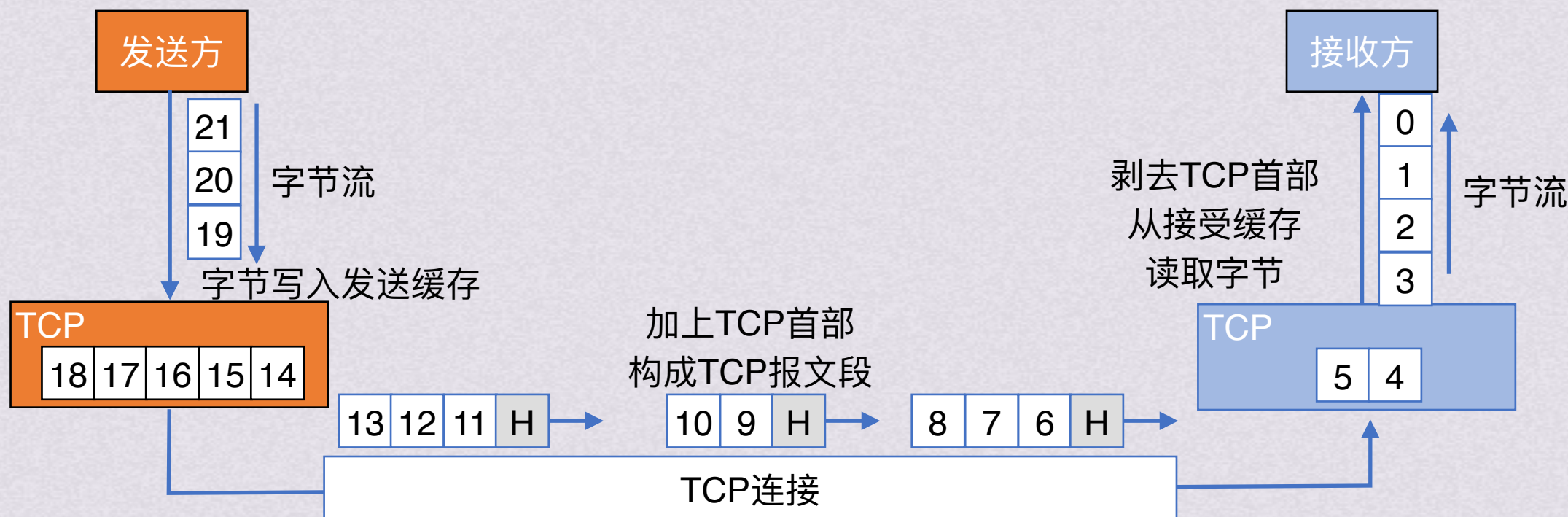
通过TCP连接传送的数据，无差错、不丢失、不重复，且按序到达

TCP面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流
TCP并不知道所传送的字节流的含义

TCP提供全双工通信

允许通信双方的应用进程在任何时候都能发送数据





TCP概述

TCP特点：

TCP是面向连接的传输层协议

在使用TCP之前必须建立连接，传送完后释放连接
每一条TCP连接必须是点对点的

TCP提供可靠交付的服务

通过TCP连接传送的数据，无差错、不丢失、不重复，且按序到达

TCP面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流
TCP并不知道所传送的字节流的含义

TCP提供全双工通信

允许通信双方的应用进程在任何时候都能发送数据





TCP概述

TCP特点：

TCP是面向连接的传输层协议

在使用TCP之前必须建立连接，传送完后释放连接
每一条TCP连接必须是点对点的

TCP提供可靠交付的服务

通过TCP连接传送的数据，无差错、不丢失、不重复，且按序到达

TCP面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流
TCP并不知道所传送的字节流的含义

TCP提供全双工通信

允许通信双方的应用进程在任何时候都能发送数据





TCP概述

TCP特点：

TCP是面向连接的传输层协议

在使用TCP之前必须建立连接，传送完后释放连接
每一条TCP连接必须是点对点的

TCP提供可靠交付的服务

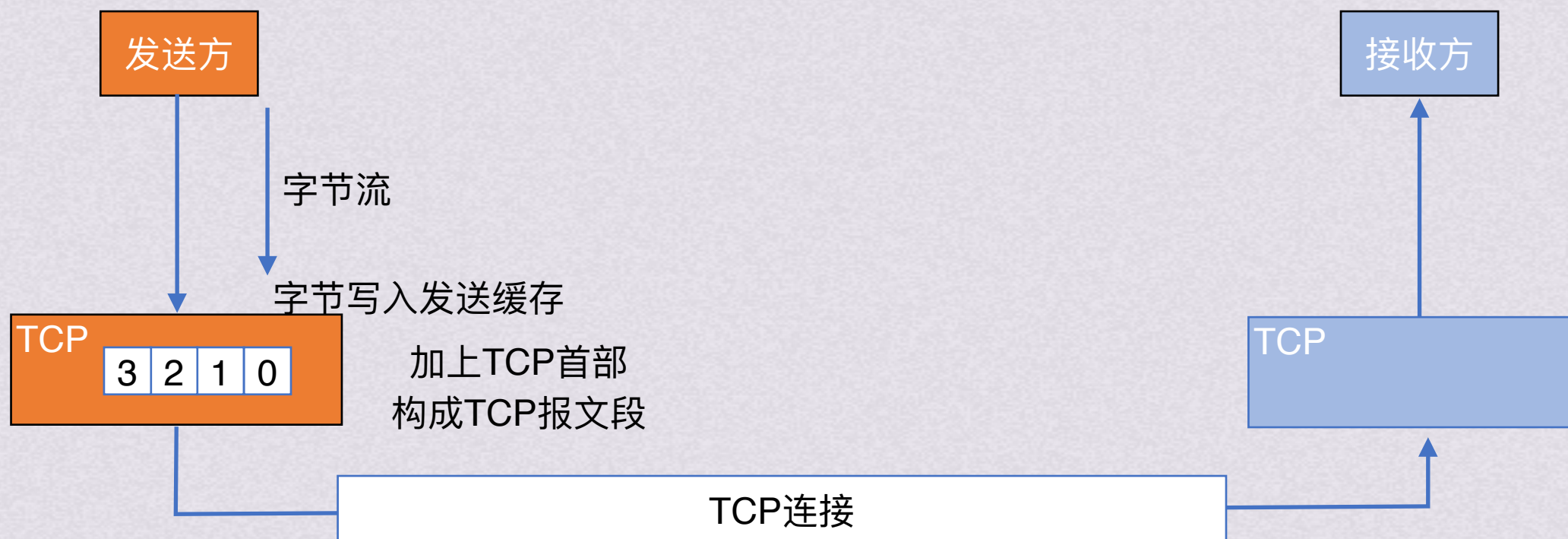
通过TCP连接传送的数据，无差错、不丢失、不重复，且按序到达

TCP面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流
TCP并不知道所传送的字节流的含义

TCP提供全双工通信

允许通信双方的应用进程在任何时候都能发送数据



TCP概述

TCP特点:

TCP是面向连接的传输层协议

在使用TCP之前必须建立连接，传送完后释放连接
每一条TCP连接必须是点对点的

TCP提供可靠交付的服务

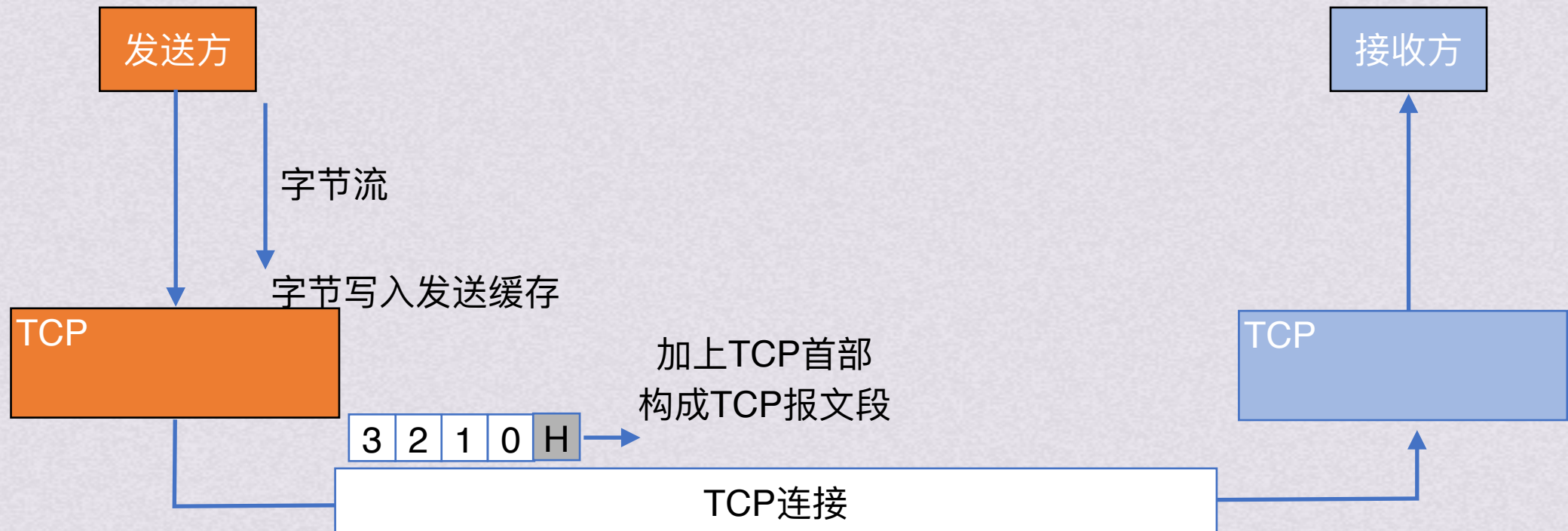
通过TCP连接传送的数据，无差错、不丢失、不重复，且按序到达

TCP面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流
TCP并不知道所传送的字节流的含义

TCP提供全双工通信

允许通信双方的应用进程在任何时候都能发送数据





TCP概述

TCP特点：

TCP是面向连接的传输层协议

在使用TCP之前必须建立连接，传送完后释放连接
每一条TCP连接必须是点对点的

TCP提供可靠交付的服务

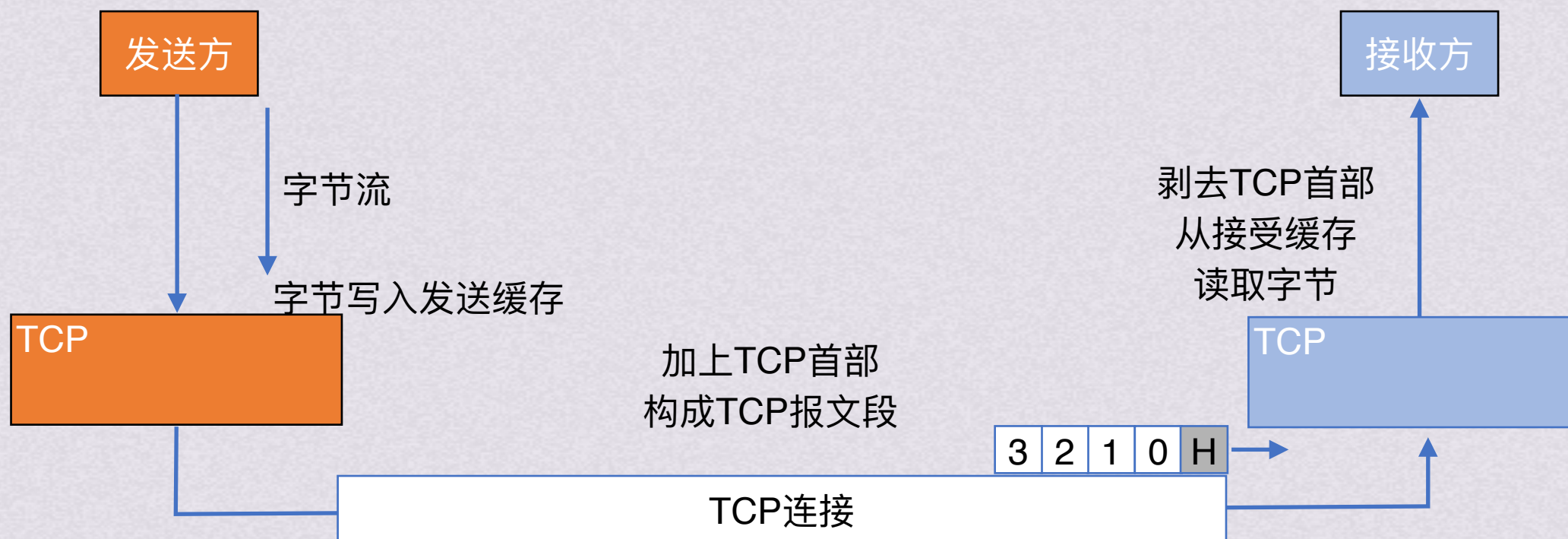
通过TCP连接传送的数据，无差错、不丢失、不重复，且按序到达

TCP面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流
TCP并不知道所传送的字节流的含义

TCP提供全双工通信

允许通信双方的应用进程在任何时候都能发送数据





TCP概述

TCP特点:

TCP是面向连接的传输层协议

在使用TCP之前必须建立连接，传送完后释放连接
每一条TCP连接必须是点对点的

TCP提供可靠交付的服务

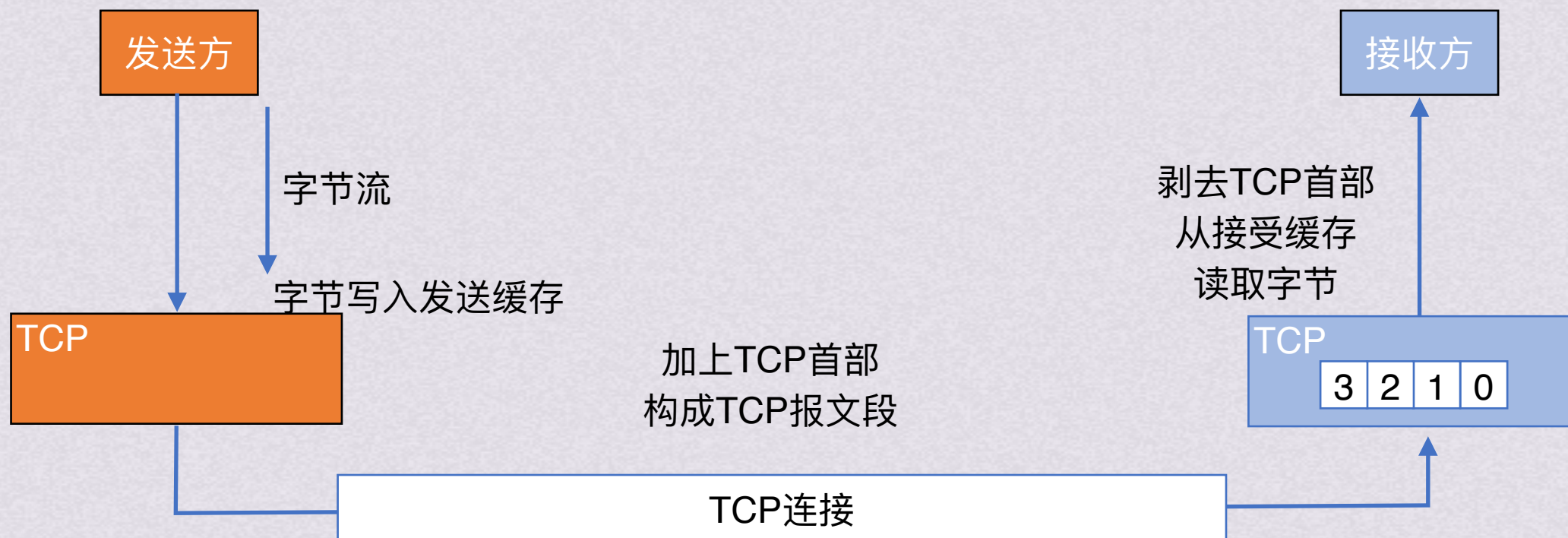
通过TCP连接传送的数据，无差错、不丢失、不重复，且按序到达

TCP面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流
TCP并不知道所传送的字节流的含义

TCP提供全双工通信

允许通信双方的应用进程在任何时候都能发送数据





TCP概述

TCP特点:

TCP是面向连接的传输层协议

在使用TCP之前必须建立连接，传送完后释放连接
每一条TCP连接必须是点对点的

TCP提供可靠交付的服务

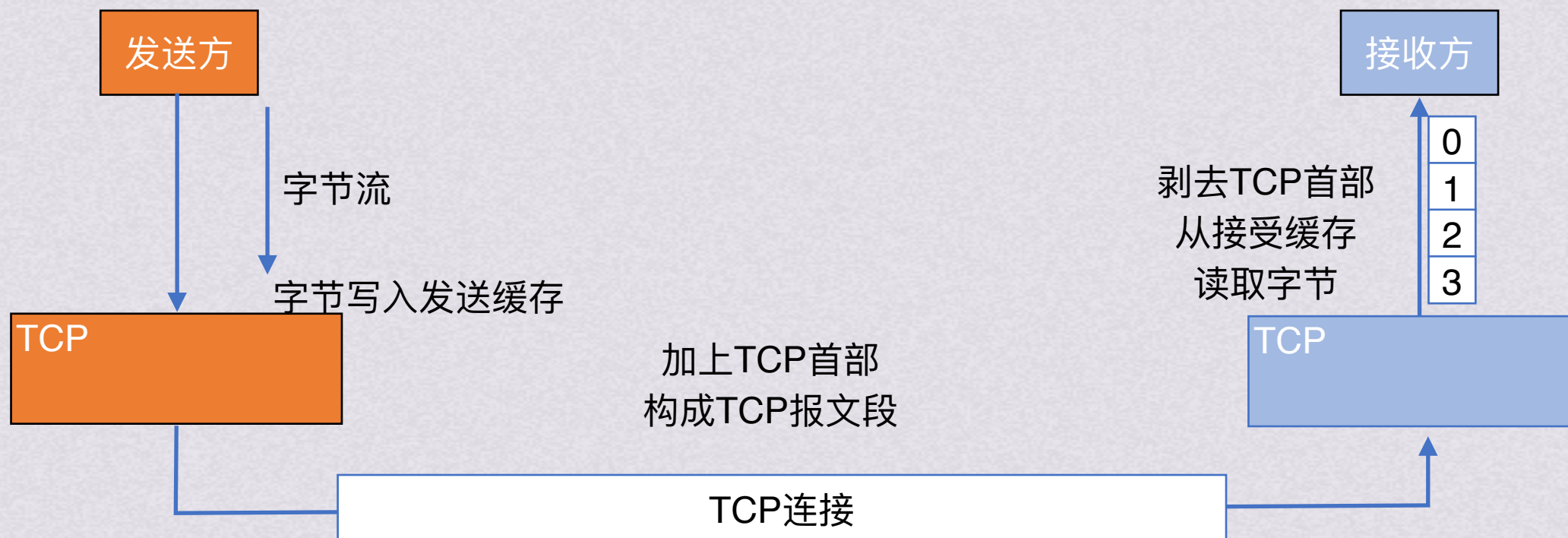
通过TCP连接传送的数据，无差错、不丢失、不重复，且按序到达

TCP面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流
TCP并不知道所传送的字节流的含义

TCP提供全双工通信

允许通信双方的应用进程在任何时候都能发送数据





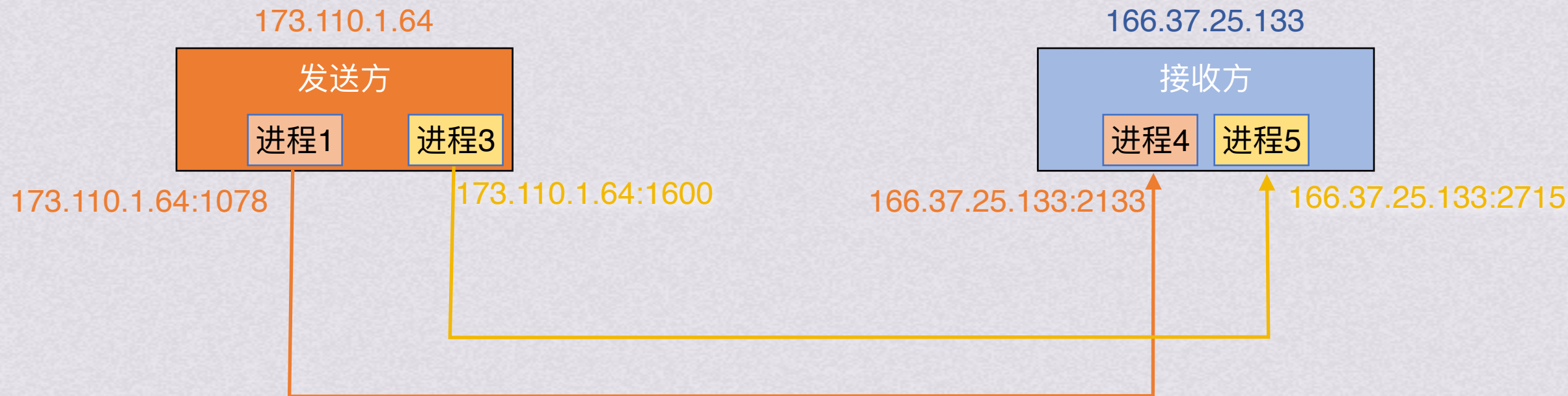
TCP概述



TCP的连接



TCP的连接



每一条TCP连接必须是点对点的
这里的“点”叫做套接字Socket

套接字Socket=(IP地址:端口号)

每一条TCP连接唯一地被通信两端的两个套接字所确定
同一个IP地址可以有多个不同的TCP连接，同一个端口号也可以出现在多个不同的TCP连接中

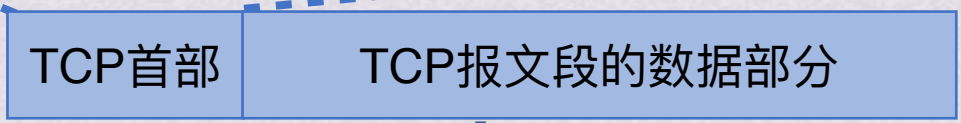
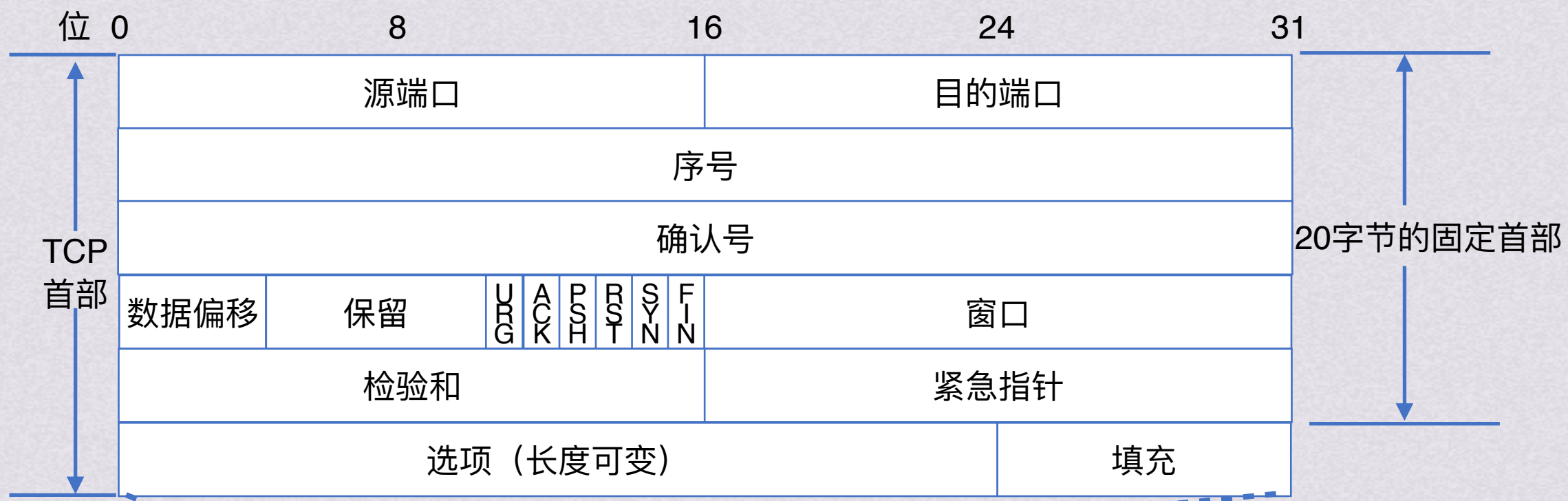


TCP的连接



TCP报文段的首部格式

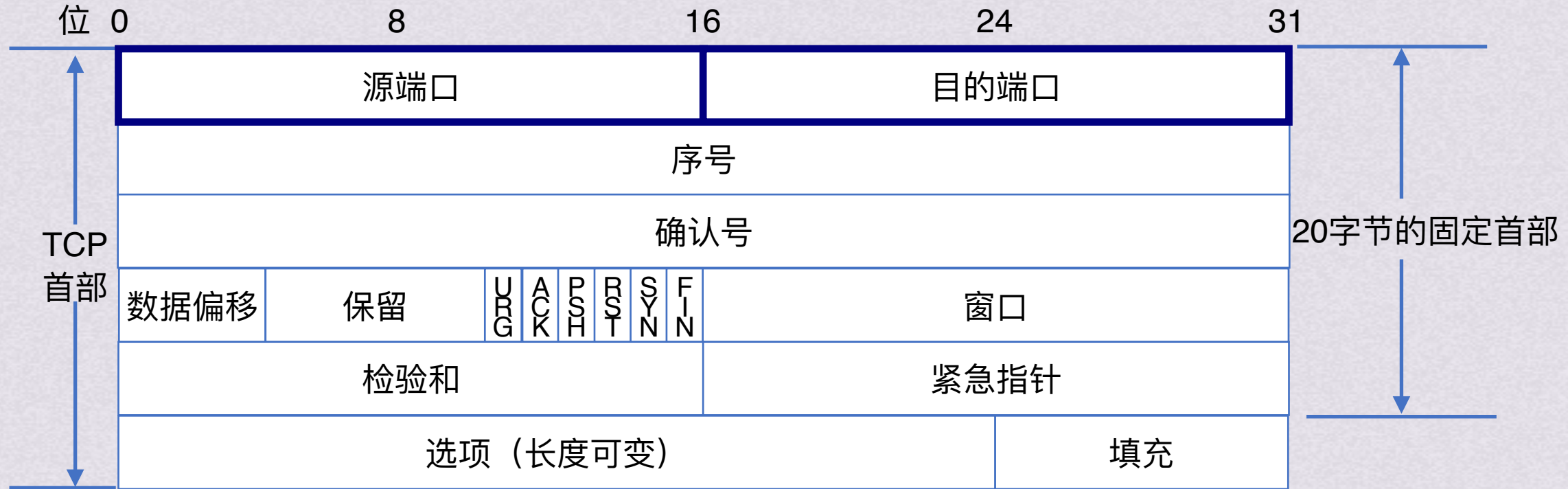
TCP报文段的首部格式



TCP虽然面向字节流，但TCP传送的数据单元是报文段

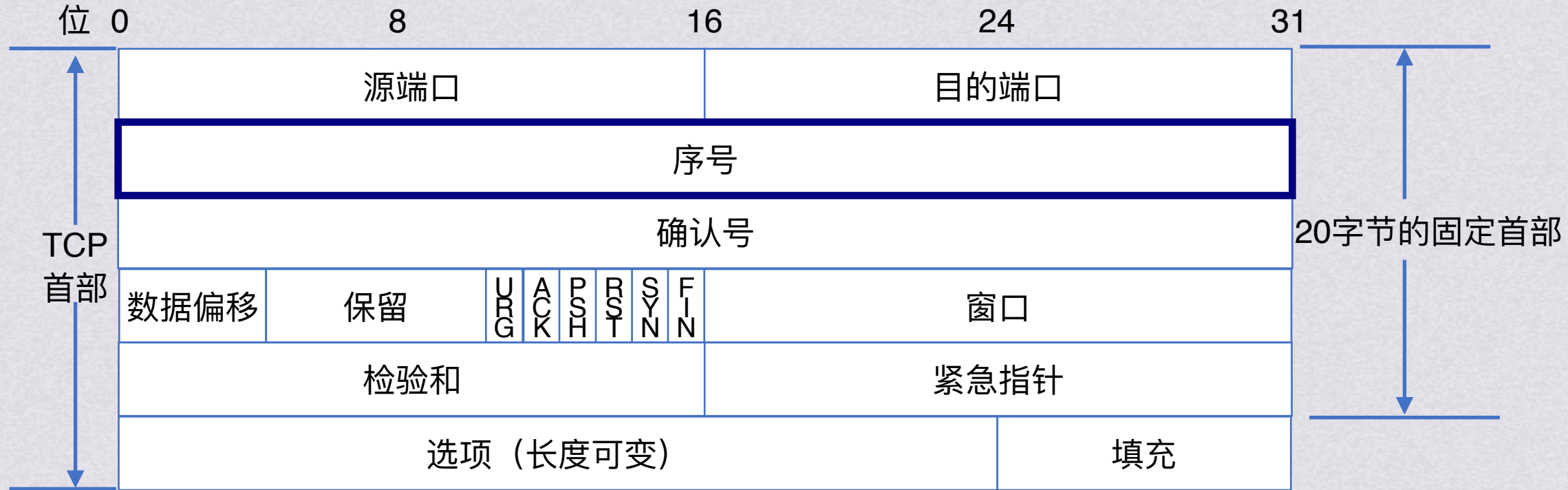


TCP报文段的首部格式



源端口和目的端口：各占2个字节，分别写入源端口号和目的端口号

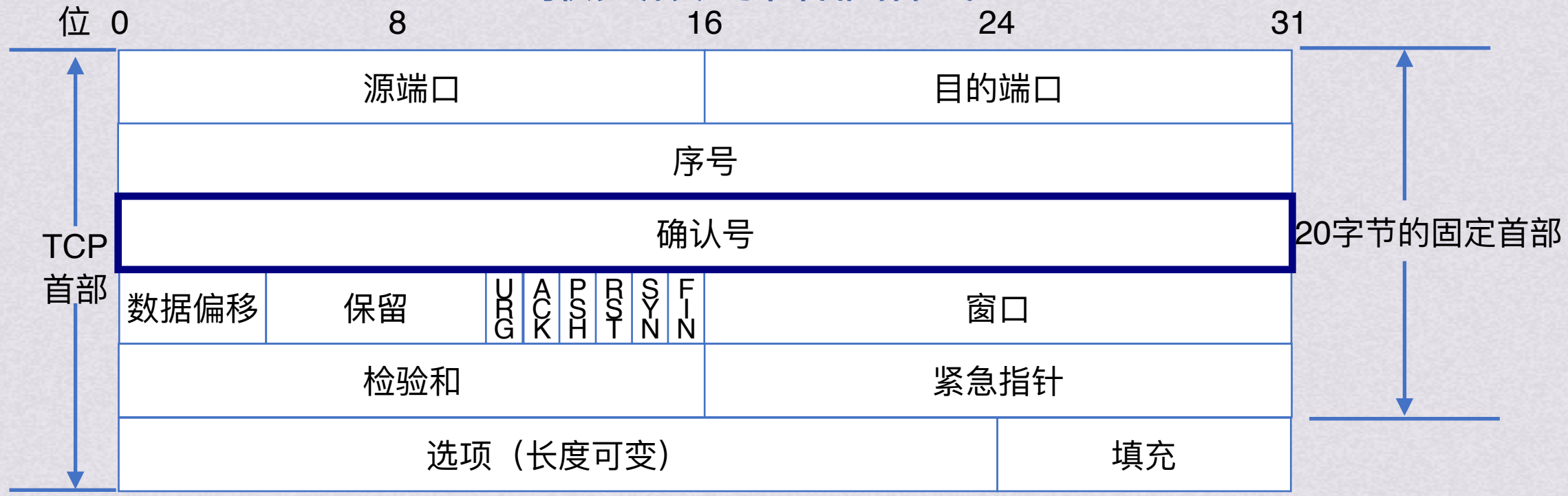
TCP报文段的首部格式



源端口和目的端口：各占2个字节，分别写入源端口号和目的端口号

序号：占4字节序号范围是 $[0, 2^{32} - 1]$ ，共 2^{32} 个序号序号增加到 $2^{32} - 1$ 后，下一个序号又回到0。在一个TCP连接中传送的字节流中，每一个字节按顺序编号。首部中的序号字段指的是本报文段所发送的数据的第一个字节的序号

TCP报文段的首部格式

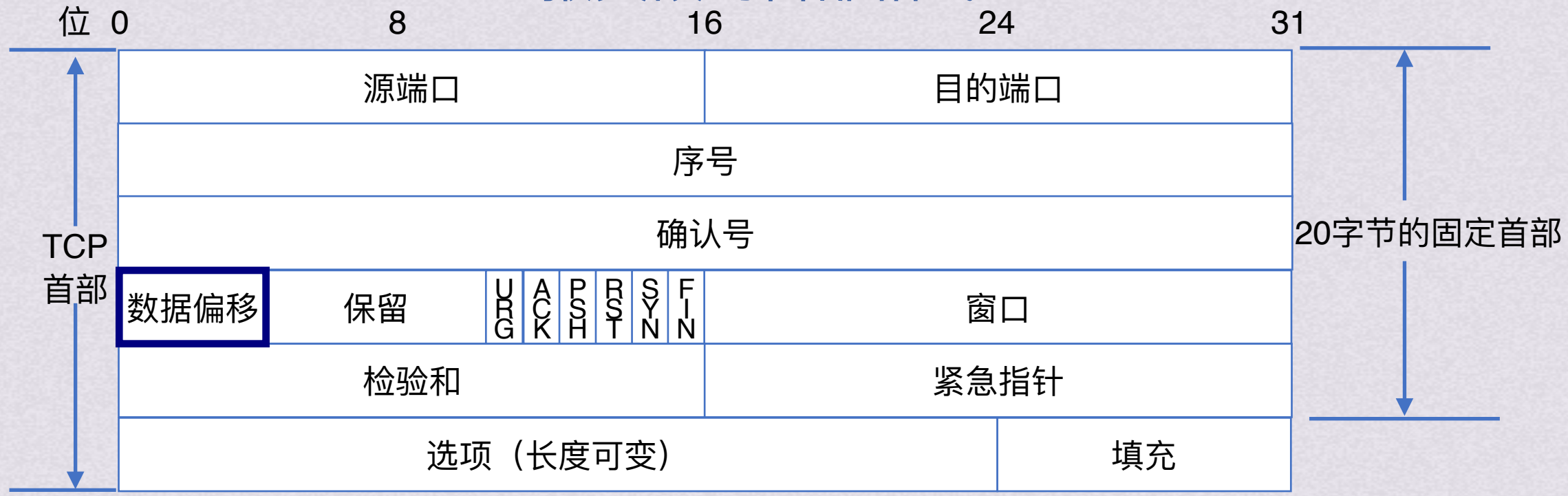


源端口和目的端口：各占2个字节，分别写入源端口号和目的端口号

序号：占4字节序号范围是 $[0, 2^{32} - 1]$ ，共 2^{32} 个序号序号增加到 $2^{32} - 1$ 后，下一个序号又回到0。在一个TCP连接中传送的字节流中，每一个字节按顺序编号。首部中的序号字段指的是本报文段所发送的数据的第一个字节的序号

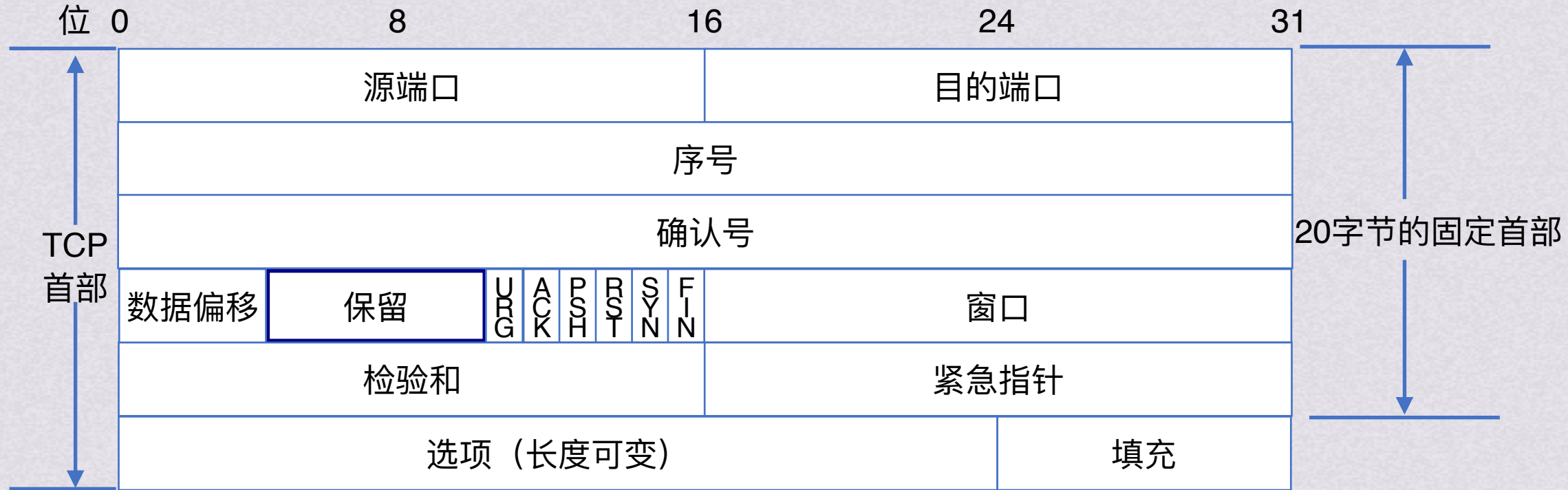
确认号：占4字节，是期望收到对方下一个报文段的第一个数据字节的序号若确认号=N，则表明：到序号N-1为止的所有数据都已经正确收到

TCP报文段的首部格式



数据偏移：占4位，它指出TCP报文段的数据起始处距离TCP报文段的起始处有多远。这个字段实际上是指出TCP报文段的首部长度数据偏移的单位是4B，所以数据偏移的最大值是15*4B=60B，这也是TCP首部的最大长度，即选项长度不能超过40B

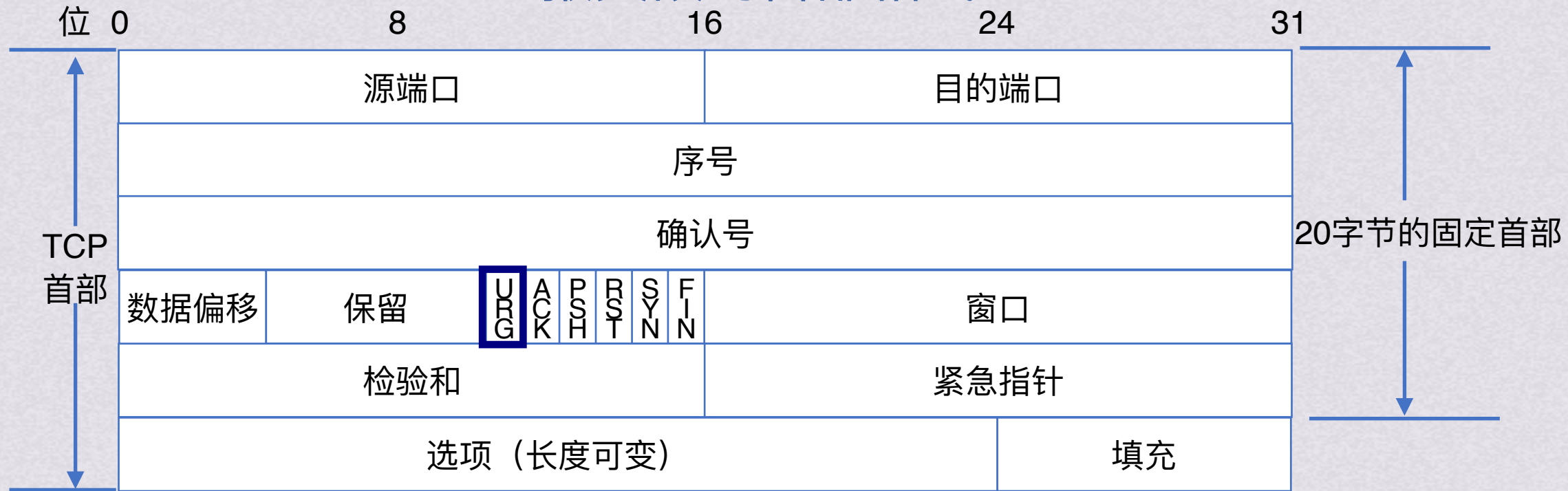
TCP报文段的首部格式



数据偏移：占4位，它指出TCP报文段的数据起始处距离TCP报文段的起始处有多远。这个字段实际上是指出TCP报文段的首部长度数据偏移的单位是4B，所以数据偏移的最大值是 $15 \times 4B = 60B$ ，这也是TCP首部的最大长度，即选项长度不能超过40B

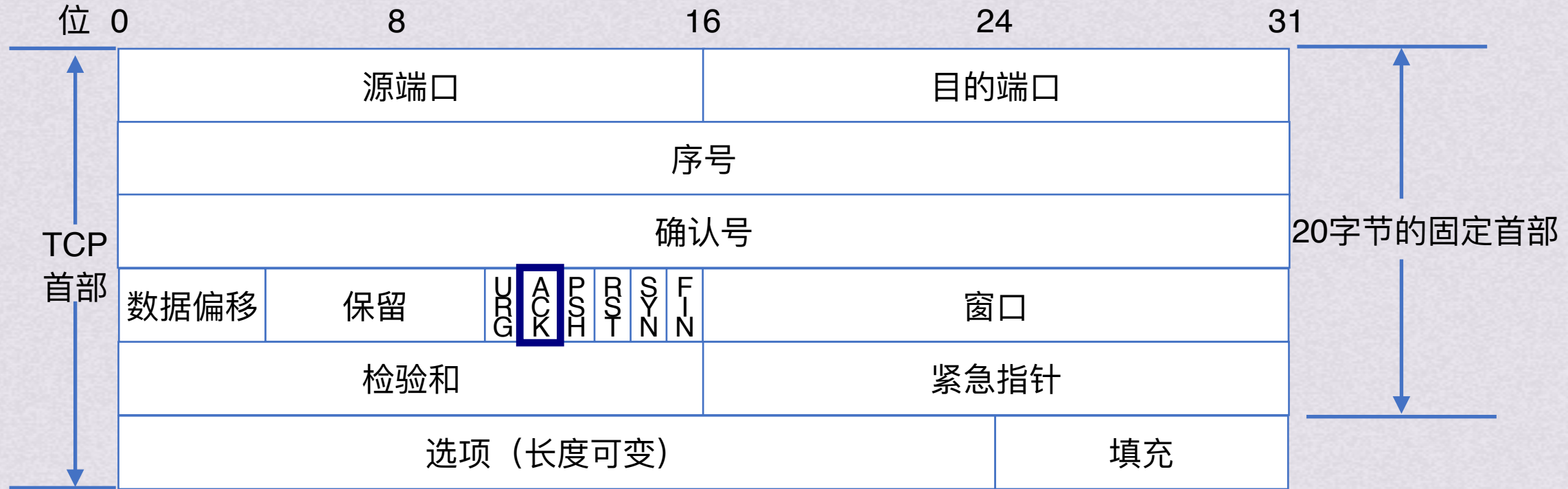
保留：占6位，保留为今后使用，目前置0

TCP报文段的首部格式



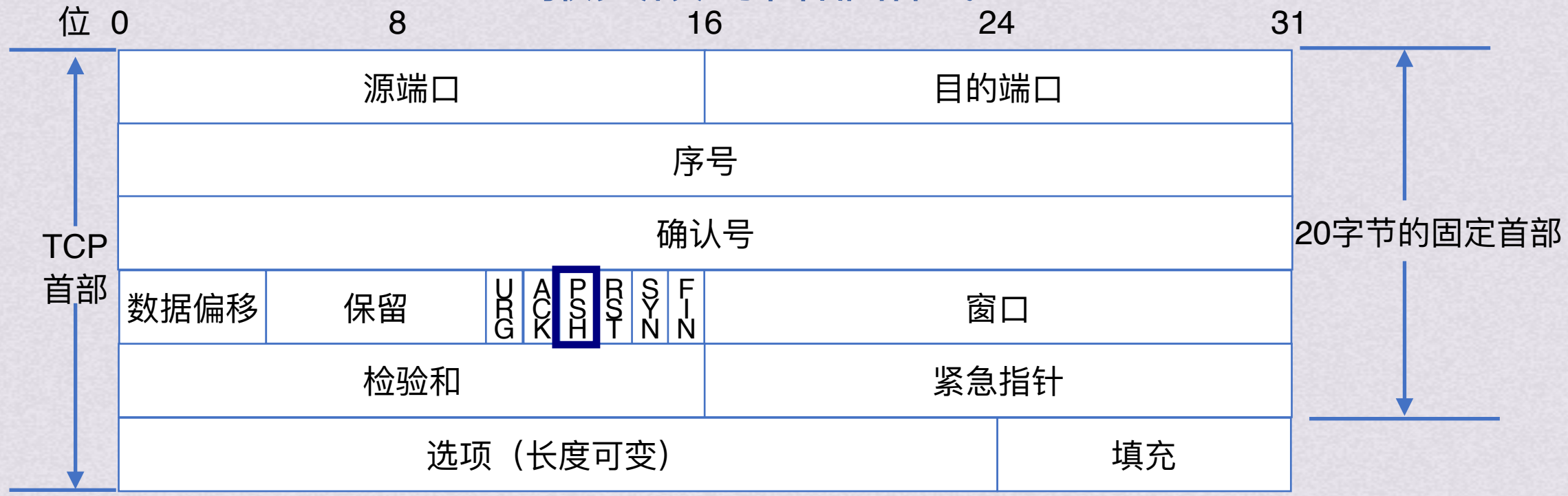
紧急URG: 当URG=1时，表明紧急指针字段有效它告诉系统此报文段中有紧急数据，应尽快传送，而不要按原来的排队顺序传送。于是发送方TCP就把紧急数据插入到本报文段数据的最前面，而在紧急数据后面的数据仍是普通数据，这时要和首部中的紧急指针字段配合使用

TCP报文段的首部格式



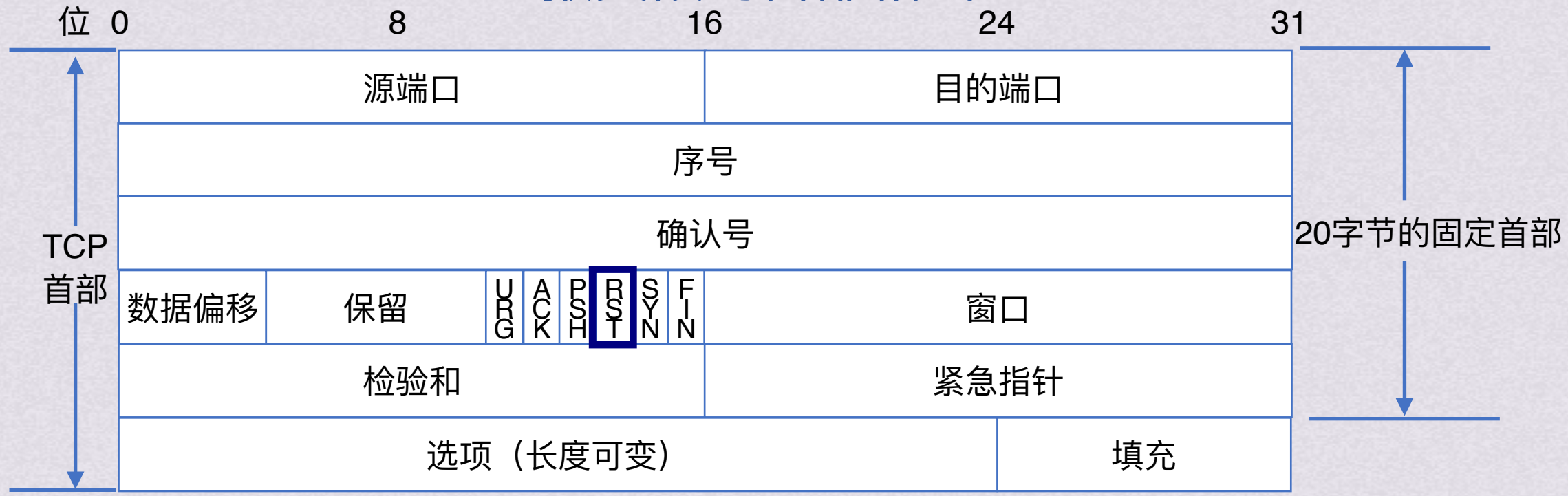
确认ACK：仅当ACK=1时确认号字段才有效当ACK=0时，确认号无效
TCP规定，连接建立后所有传送的报文段都必须把ACK置1

TCP报文段的首部格式



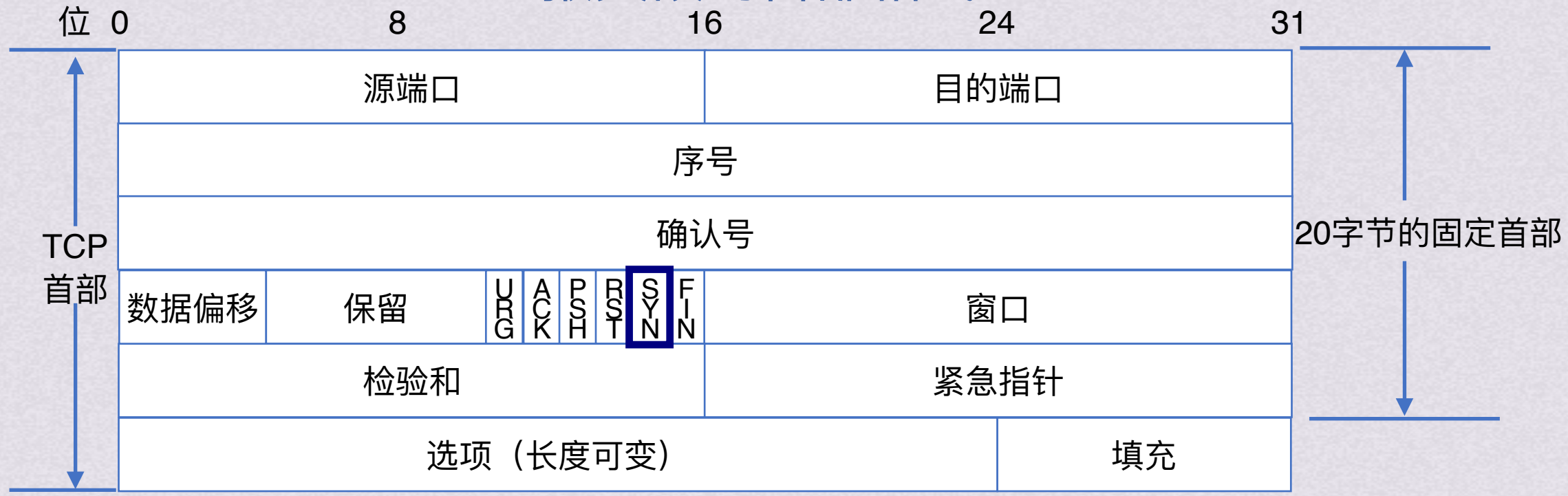
推送PSH：当两个应用进程进行交互式的通信时，有时一端的应用进程希望在键入一个命令后立刻收到对方的响应这时，发送方把PSH置1，并立即创建一个报文段发送出去接收方TCP收到PSH=1的报文段，就尽快地交付接收应用进程，而不再等到整个缓存都填满了再向上交付

TCP报文段的首部格式



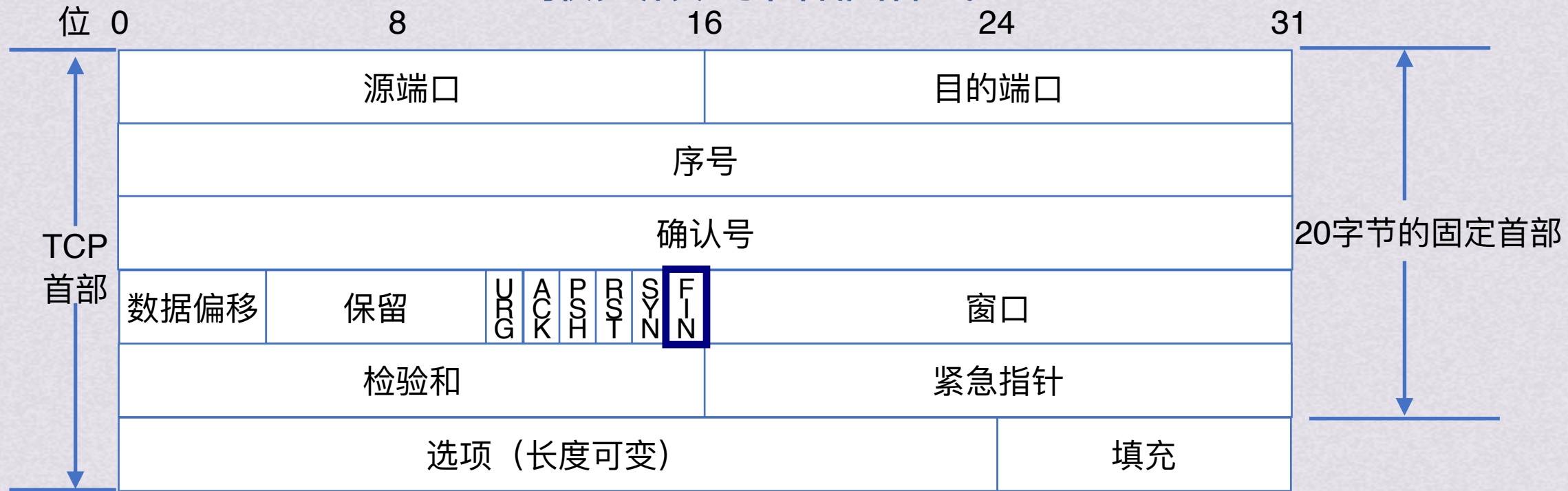
复位RST: 当RST=1时, 表明TCP连接中出现严重差错, 必须释放连接, 然后再重新建立运输连接。将RST置1还用来拒绝一个非法的报文段或拒绝打开一个连接

TCP报文段的首部格式

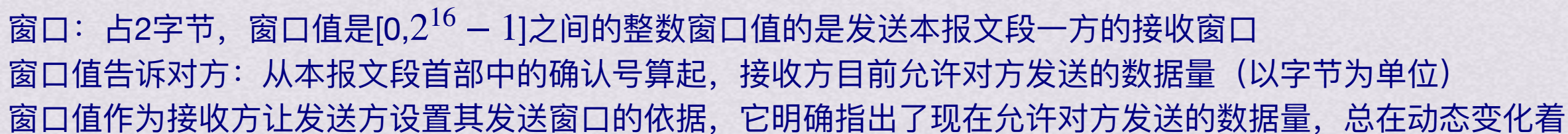


同步SYN：在连接建立时用来同步序号当SYN=1而ACK=0时，表明这是一个连接请求报文段对方若同意建立连接，则应在响应的报文段中使SYN=1和ACK=1。因此，SYN置1就表示这是一个连接请求或连接接受报文

TCP报文段的首部格式

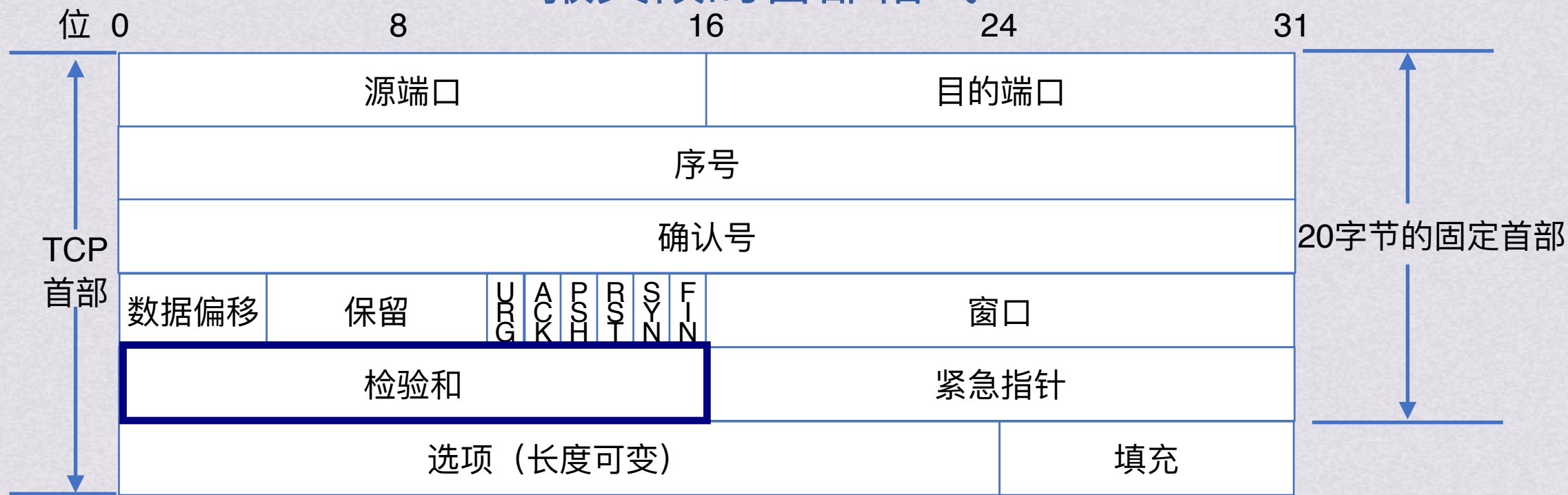


终止FIN：用来释放一个连接当FIN=1时，表明此报文段的发送方的数据已发送完毕，并要求释放运输连接



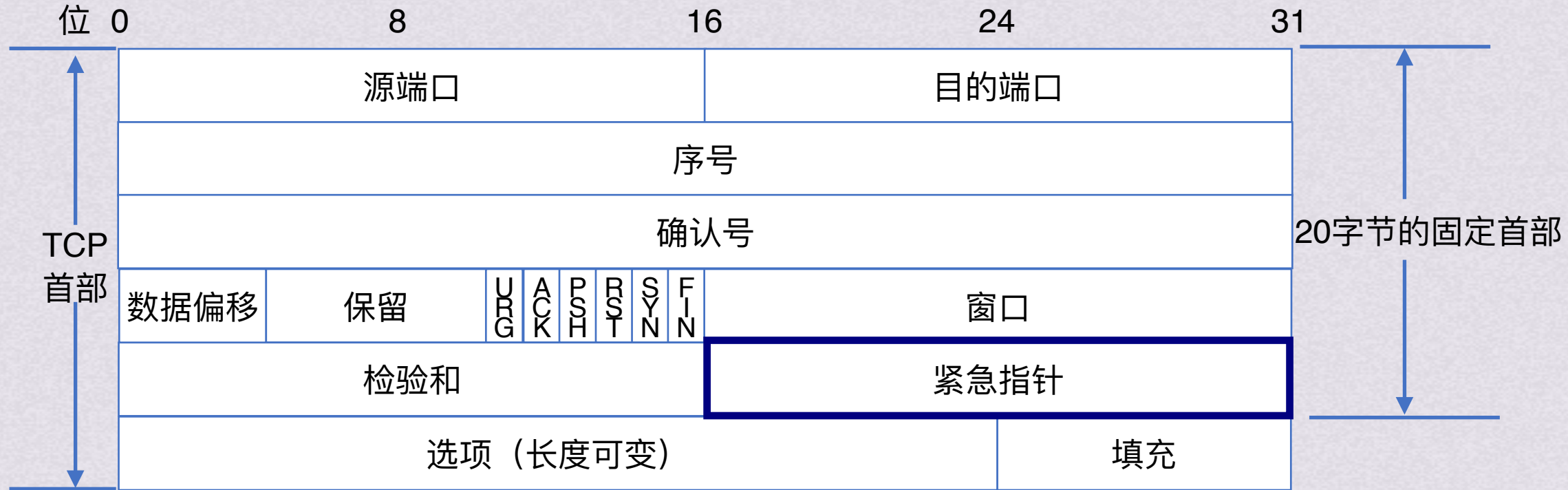


TCP报文段的首部格式



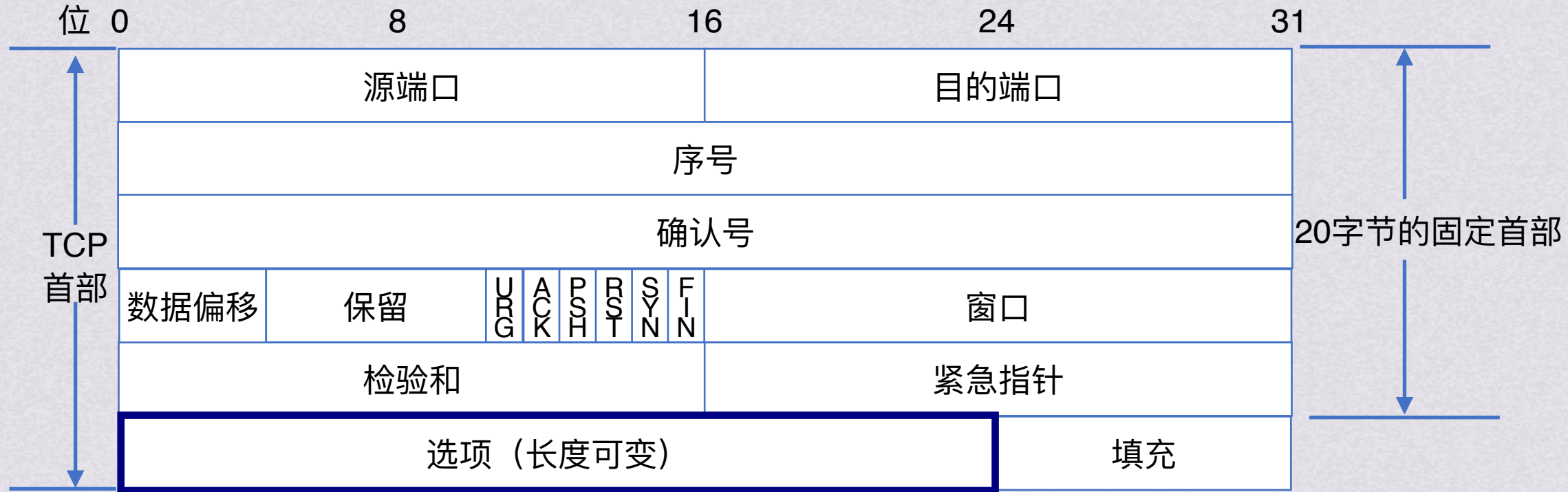
检验和：占2字节检验的范围包括数据和首部两个部分和UDP一样，TCP在计算检验和时也要在前面添加伪首部，占12字节伪首部的格式和UDP一样，但应该把第四个字段中的17改成6表示TCP接收方收到报文段后，仍要加上伪首部来计算检验和

TCP报文段的首部格式



紧急指针：占2字节，紧急指针仅在URG=1时才有意义，它指出本报文段中的紧急数据的字节数。因此，紧急指针指出了紧急数据末尾在报文段中的位置

TCP报文段的首部格式

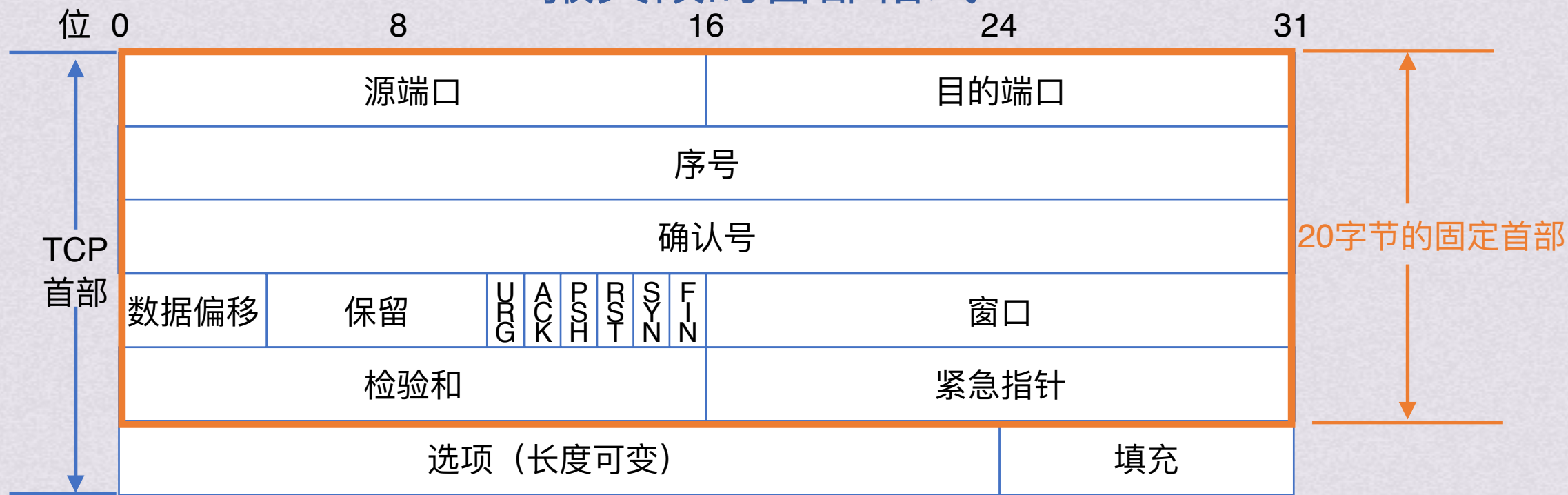


选项：长度可变，最长40B，最后的填充字段仅为使整个TCP首部长度的4B的整数倍

TCP起初只规定了一种选项，即最大报文长度MSS，MSS是每一个TCP报文段中的数据字段的最大长度
所以MSS并不是整个TCP报文段的最大长度，而是“TCP报文段长度减去TCP首部长度的”



TCP报文段的首部格式



TCP报文段首部的前20个字节是固定的，后面有4n字节是根据需要而增加的选项（n是整数）
因此，TCP首部的最小长度是20字节



TCP报文段的首部格式



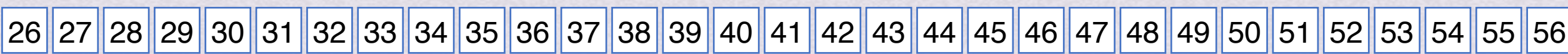
TCP可靠传输的实现



TCP可靠传输的实现

1.序号

TCP连接传送的数据流按照字节编号，TCP头部中序号字段的值就填本报文段所发送的数据第一个字节的编号

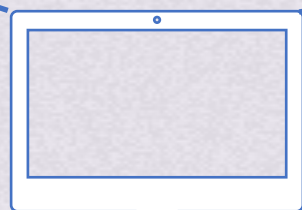


第一个报文段数据

第二个报文段数据

第三个报文段数据

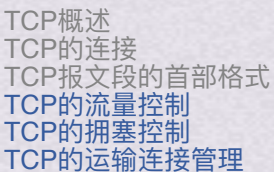
A的窗口情况



A



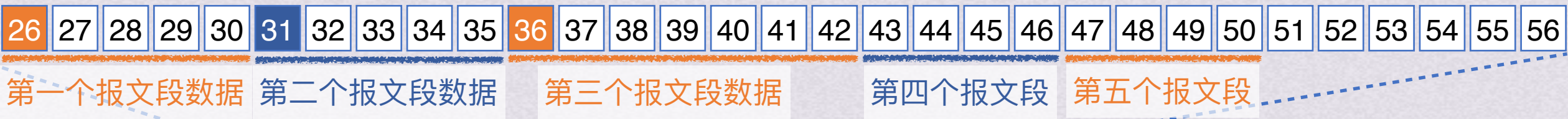
B



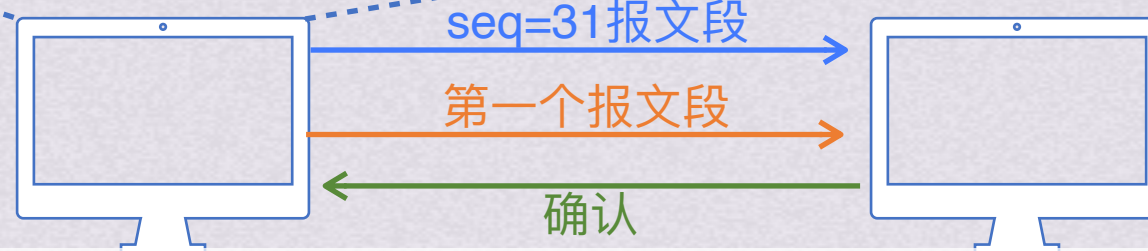
TCP可靠传输的实现

3.重传

TCP连接传送的数据流按照字节编号，TCP头部中序号字段的值就填本报文段所发送的数据第一个字节的编号



A的窗口情况



TCP规定当发送方收到对同一个报文段的3个冗余ACK时，就可以认为跟在这个被确认报文段之后的报文段已经丢失。这时发送方会立刻对31号报文段重传，这种技术叫做快重传，我们也会在以后仔细学习。



正确收到第一个报文段,回复ack=31

报文段丢失

31缺失,ack=31

31缺失,ack=31 31缺失,ack=31

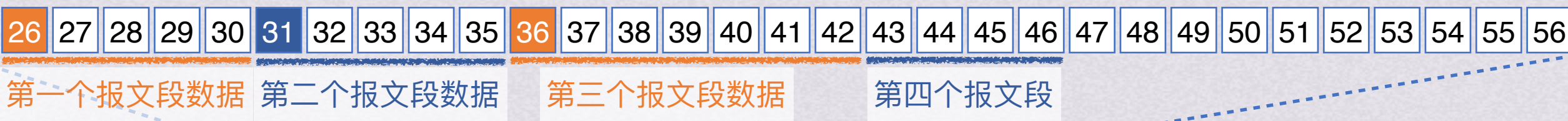
TCP默认使用累计确认，也就是TCP只确认数据流中第一个丢失字节为止的字节



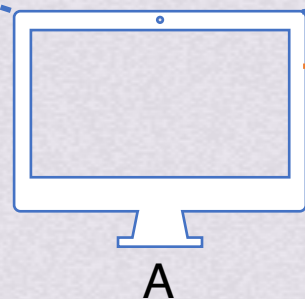
TCP可靠传输的实现

3.重传

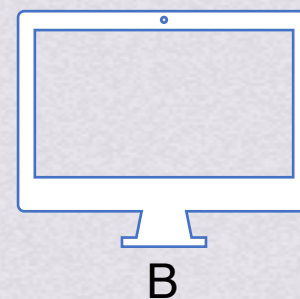
TCP连接传送的数据流按照字节编号，TCP头部中序号字段的值就填本报文段所发送的数据第一个字节的编号



A的窗口情况



第一个报文段 丢失



还有第二种情况也会触发重传：超时

在一定时间内都没有收到B对该报文段的确认时，就需要重传这一报文段



TCP可靠传输的实现

注意：

- 1.虽然A的发送窗口是根据B的接收窗口设置的，但同一时刻，A的发送窗口并不总是和B的接收窗口一样大这是因为网络传送窗口值需要经历一定的时间滞后另外，发送方A还会根据网络的拥塞情况调整自己的发送窗口数值这一点和链路层的滑动窗口是不一样的，链路层的ARQ中滑动窗口的大小总是固定不变的
- 2.对于不按序到达的数据应如何处理，TCP标准并无明确规定如果接收方把不按序到达的数据一律丢弃，那么接收窗口的管理将会比较简单，但这样对网络资源利用不利TCP通常是将不按序到达的数据先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再按序交付上层的应用进程
- 3.TCP要求接收方必须有累积确认的功能，接收方可以在合适的时候发送确认，也可以在自己有数据要发送的时候把确认信息捎带上



TCP可靠传输的实现

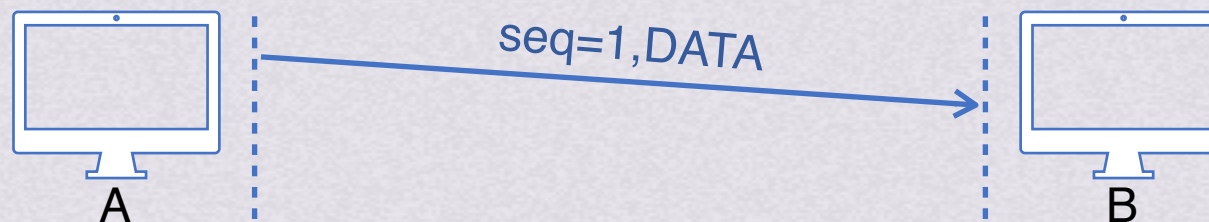
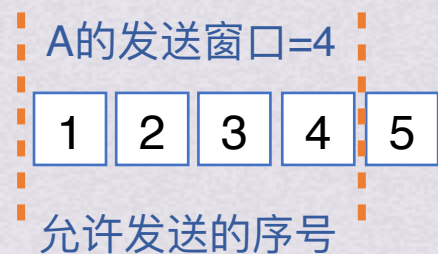


TCP的流量控制



TCP的流量控制

流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制

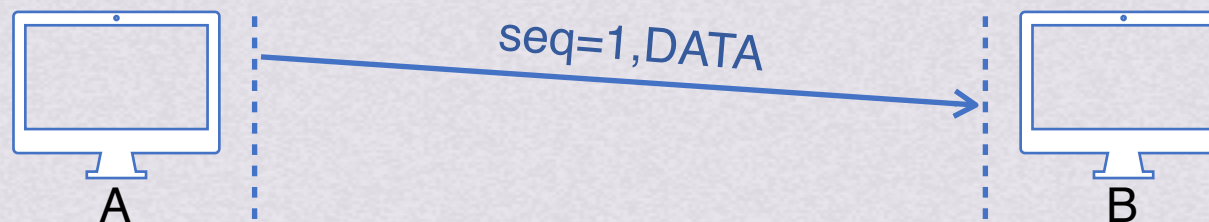
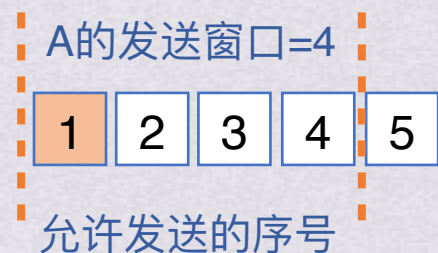


假设在连接建立时，B告知A接收窗口rwnd=4



TCP的流量控制

流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制

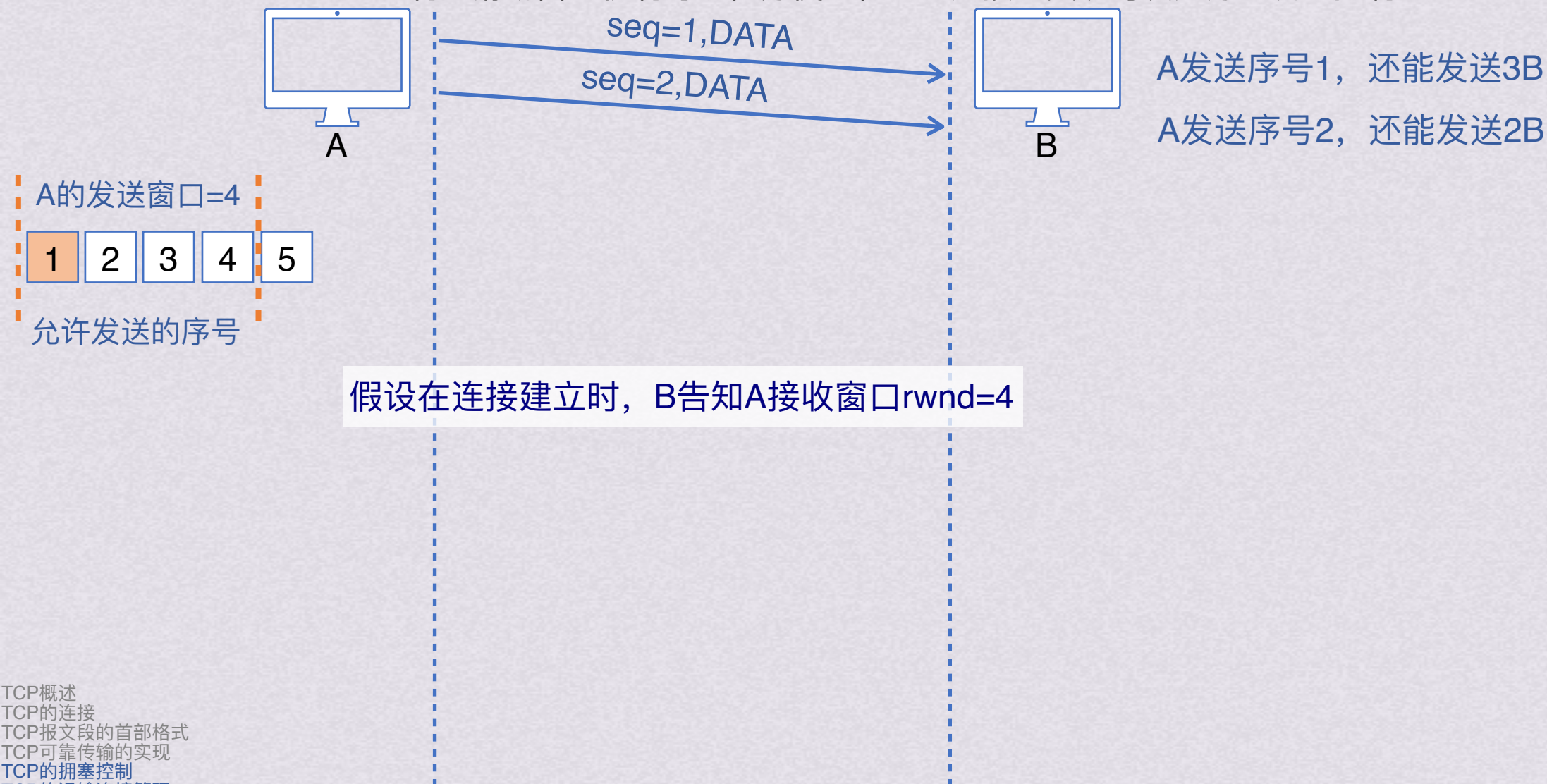


假设在连接建立时，B告知A接收窗口rwnd=4



TCP的流量控制

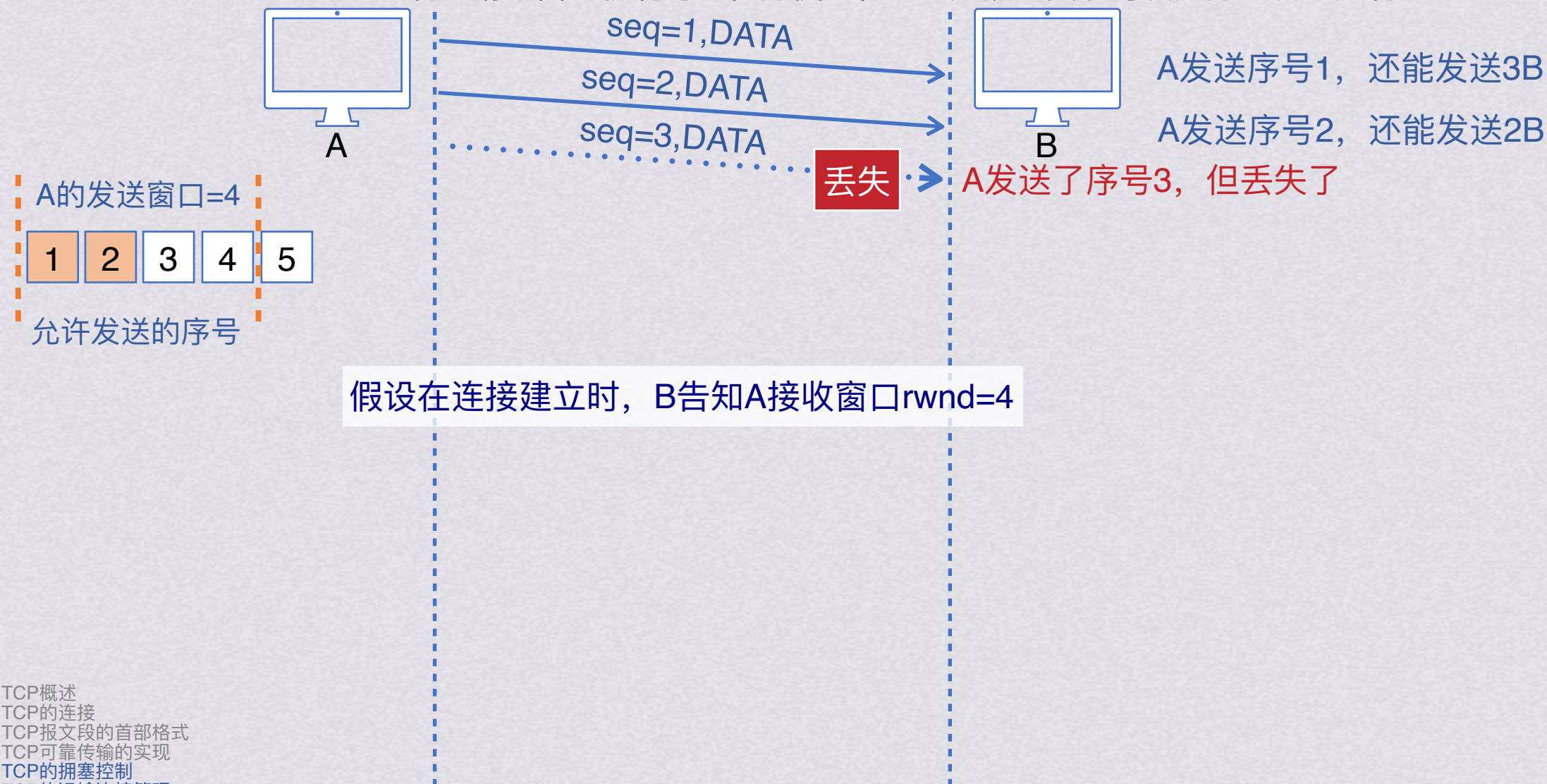
流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制





TCP的流量控制

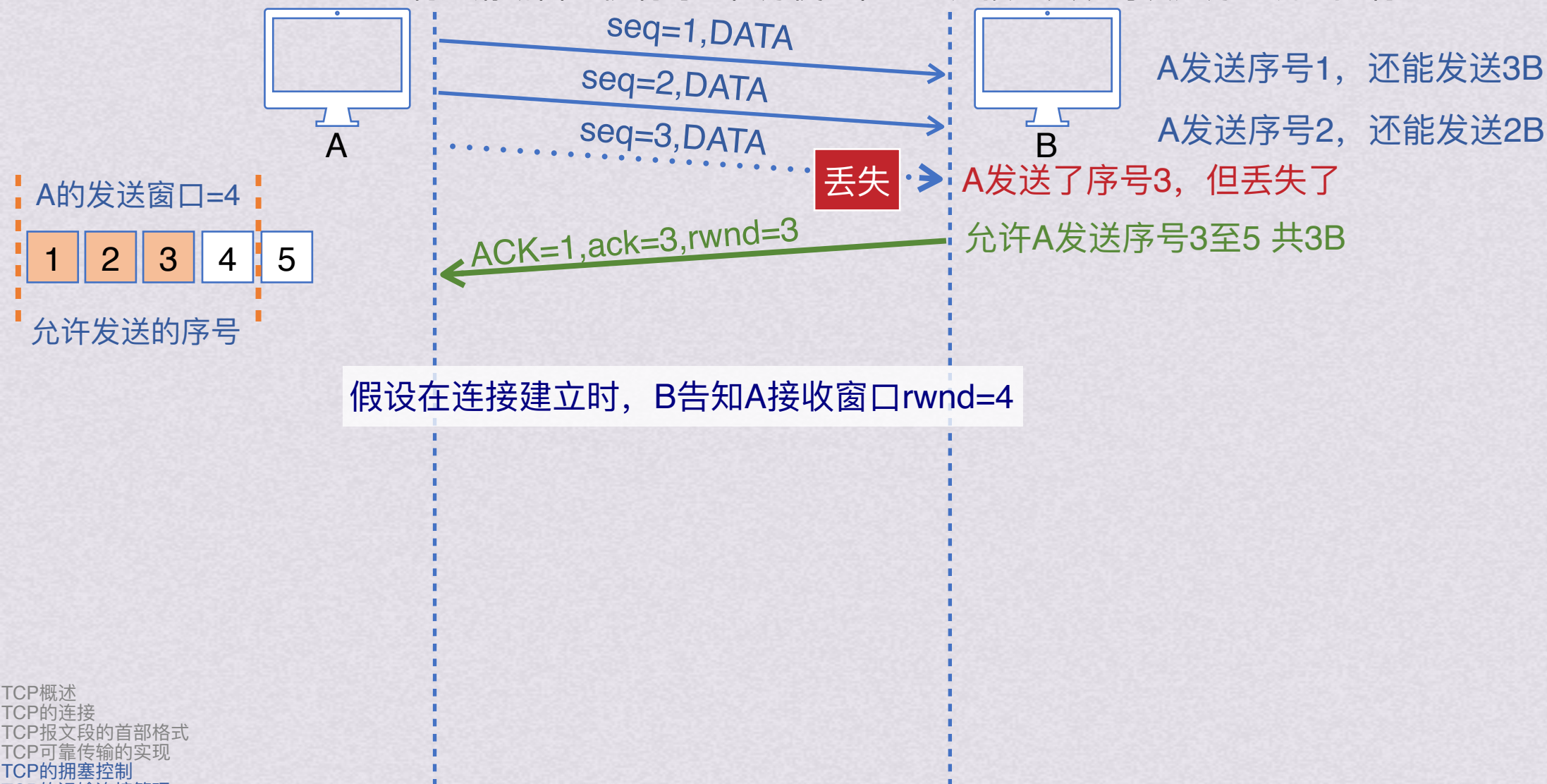
流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制





TCP的流量控制

流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制





TCP的流量控制

流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制



假设在连接建立时，B告知A接收窗口rwnd=4



seq=1, DATA

seq=2, DATA

seq=3, DATA

丢失

ACK=1, ack=3, rwnd=3

seq=4, DATA

A发送序号1, 还能发送3B

A发送序号2, 还能发送2B

A发送了序号3, 但丢失了

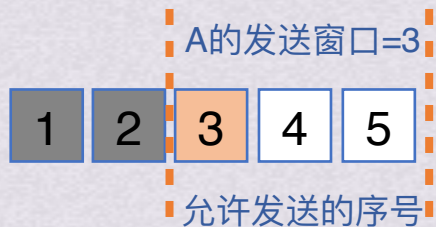
允许A发送序号3至5 共3B

A发送序号4, 还能发送1B新数据

窗口=3

5

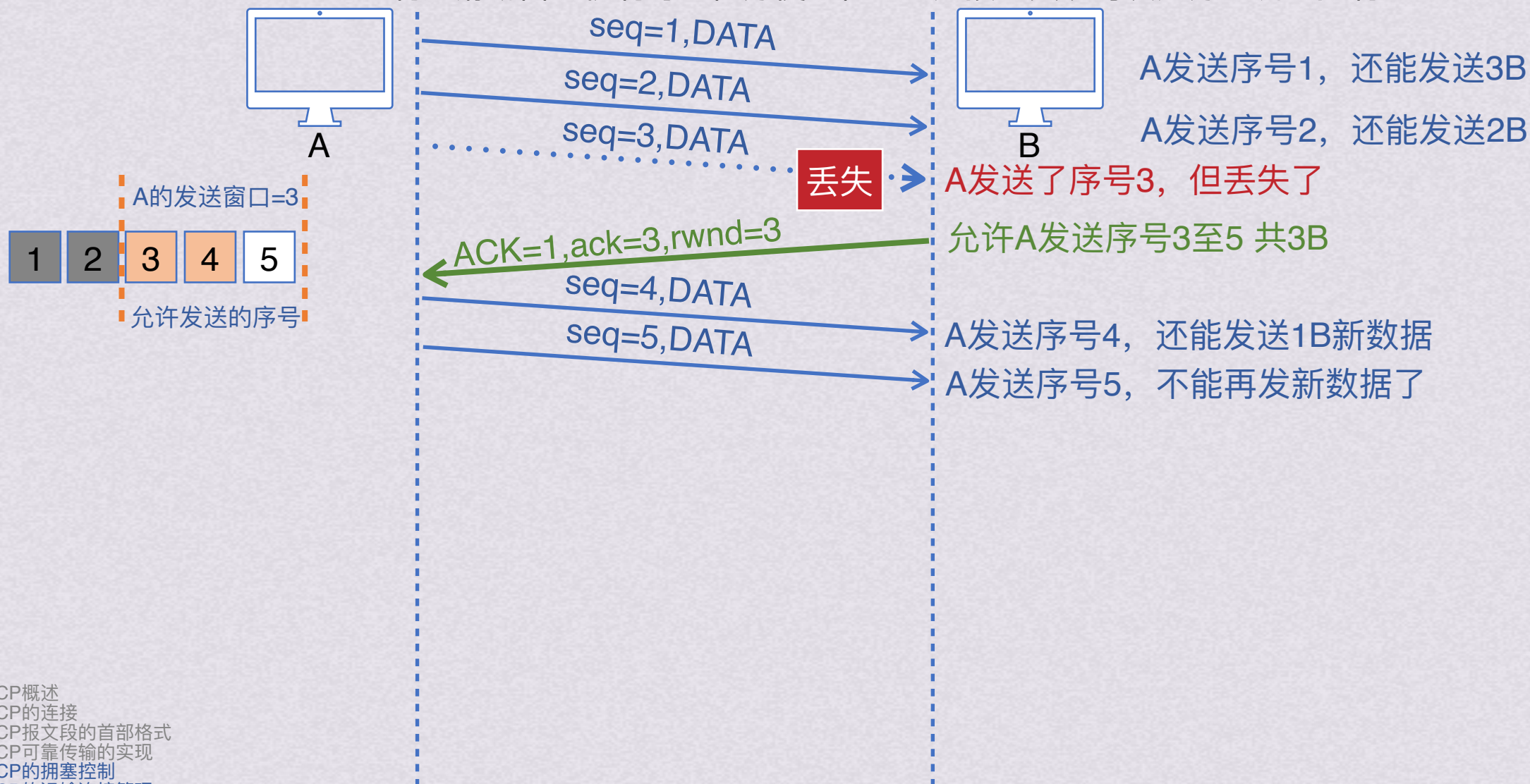
的序号





TCP的流量控制

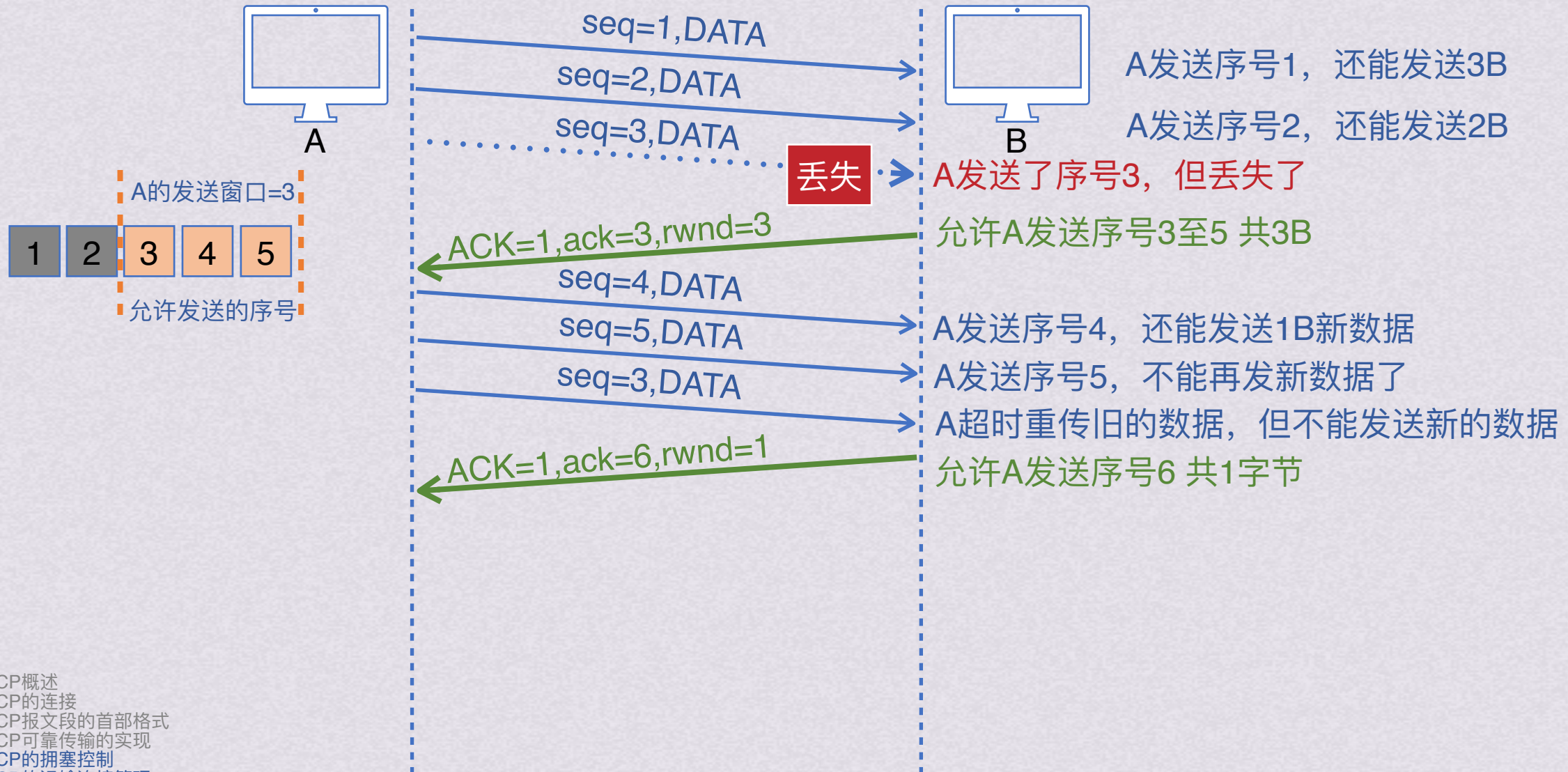
流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制





TCP的流量控制

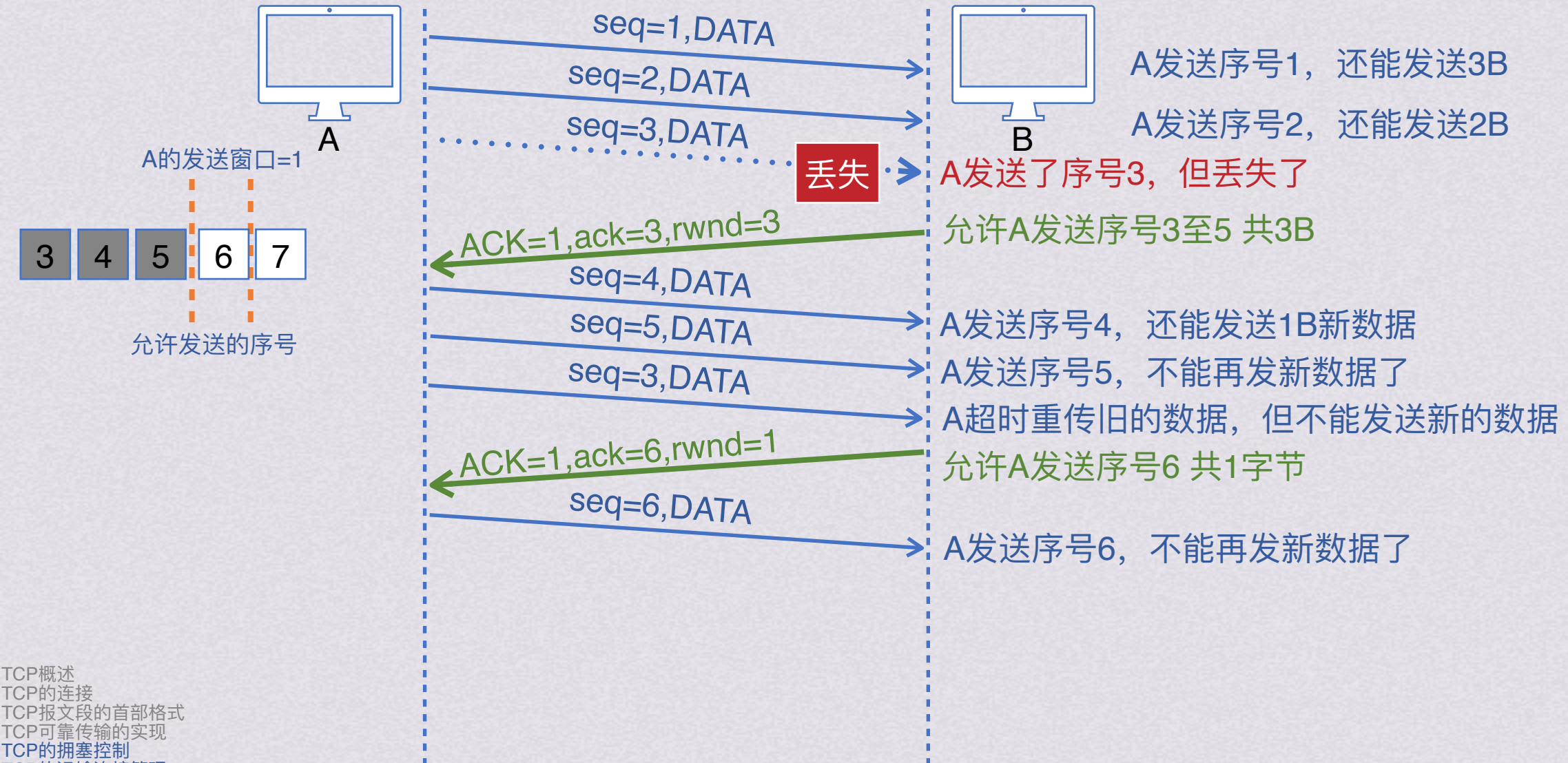
流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制





TCP的流量控制

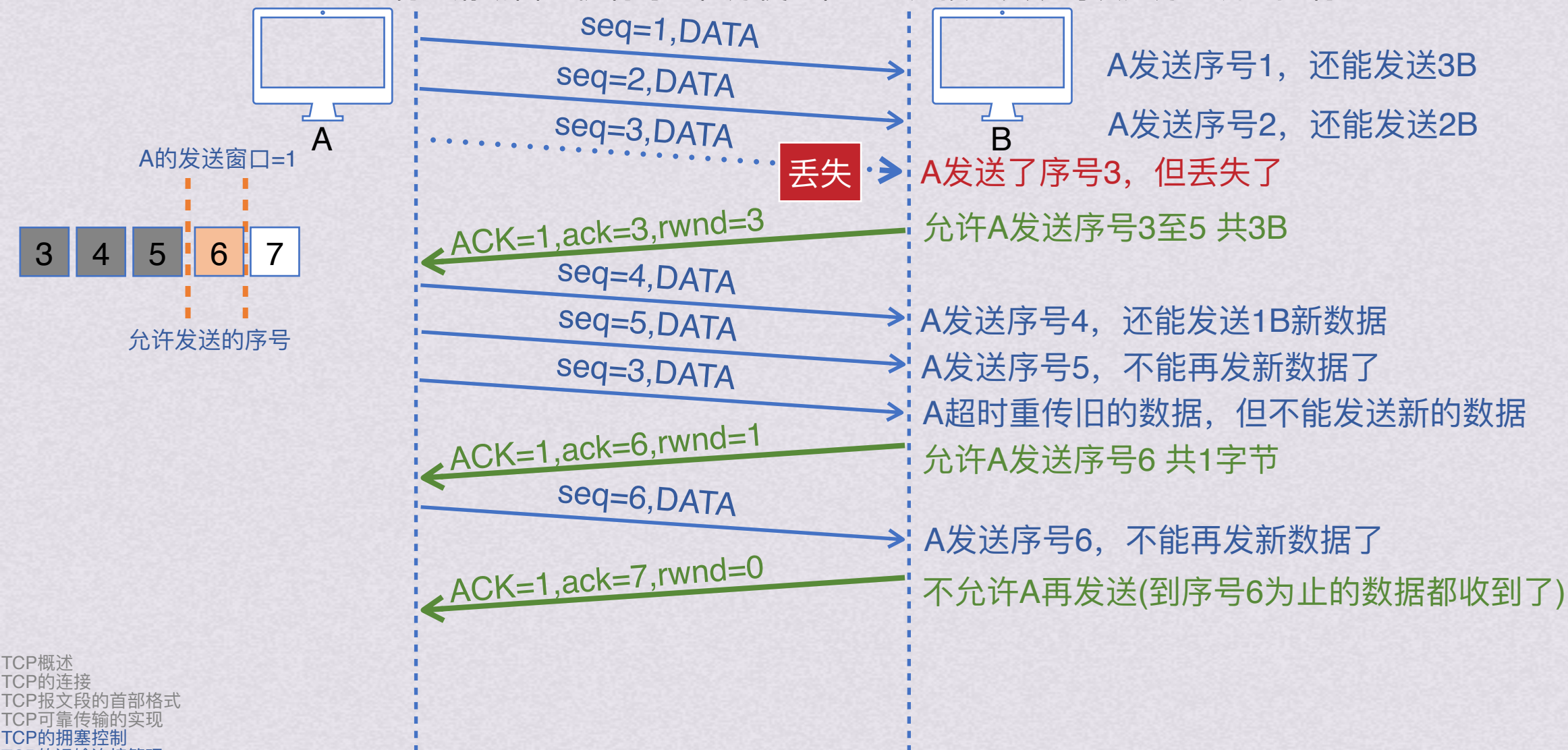
流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制





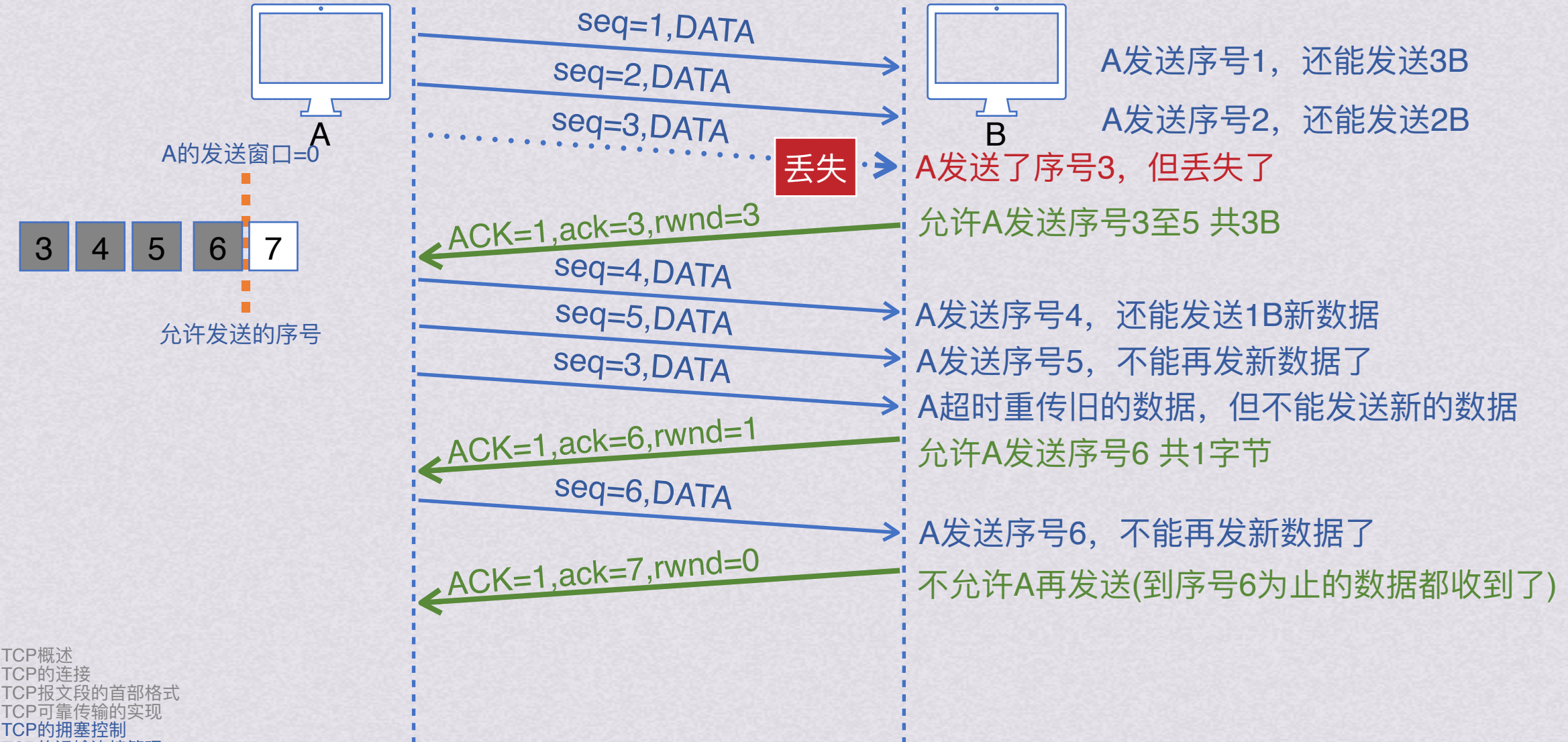
TCP的流量控制

流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制



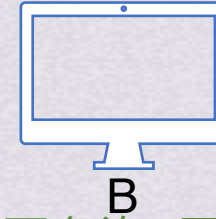
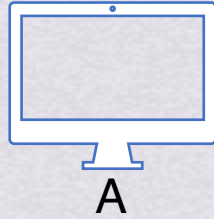
TCP的流量控制

流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制

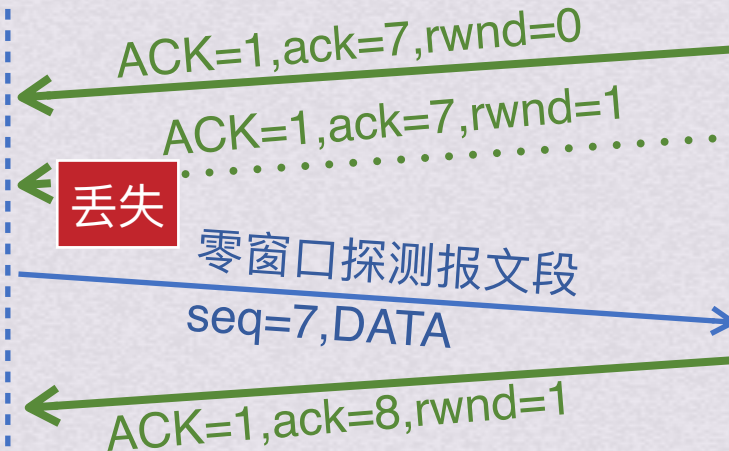


TCP的流量控制

流量控制的含义就是让发送方的发送速率不要太快，要让接收方来得及接收
利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制



为了解决这个问题，TCP为每个连接设有一持续计时器收到零窗口通知就启动时间到期时发送一个零窗口探测报文段(携带1B数据)，对方确认这个探测报文段时给出现在的窗口值



不允许A再发送(到序号6为止的数据都收到了)

假设此时B的接受缓存又有了存储空间

此时B一直等A的数据

A一直等B非0窗口调整通知

出现了互相等待的死锁局面



TCP的流量控制

利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制

TCP接收方利用自己的接收窗口的大小来限制发送方发送窗口的大小

TCP发送方收到接收方的零窗口通知后，应启动持续计时器持续计时器超时后，向接收方发送零窗口探测报文



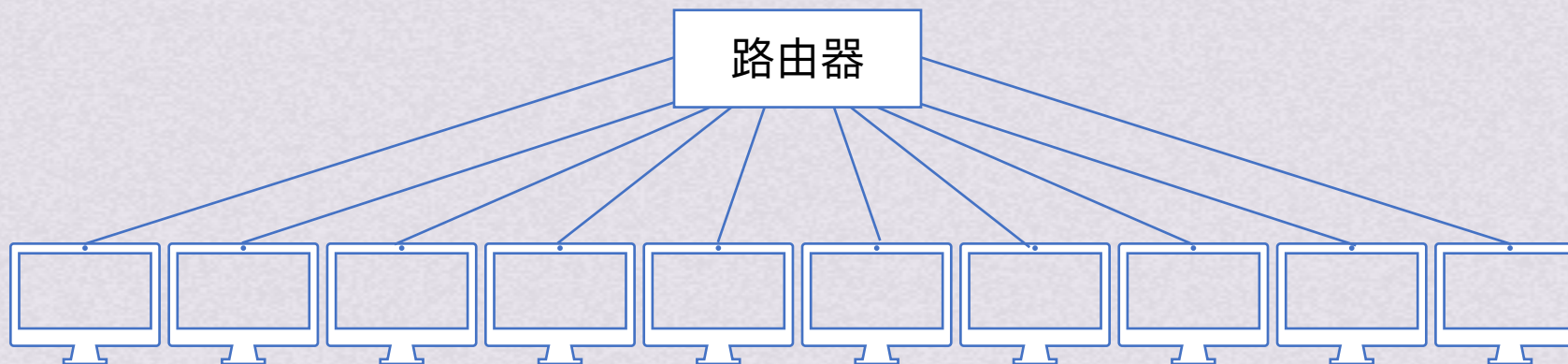
TCP的流量控制



TCP的拥塞控制



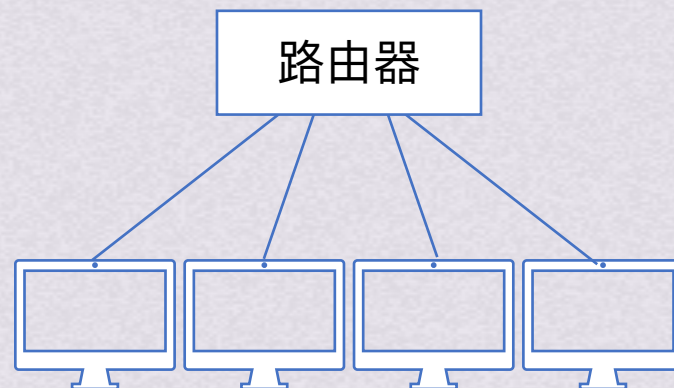
TCP的拥塞控制



开环控制：在设计网络时事先将发生拥塞的有关因素考虑周到，力求网络在工作时不产生拥塞但一旦整个系统运行起来，就不再中途进行改正了

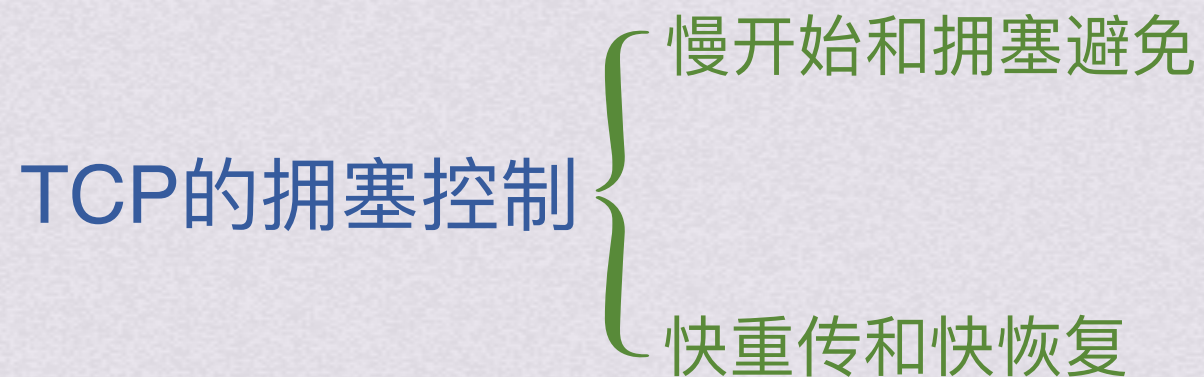


TCP的拥塞控制



开环控制：在设计网络时事先将发生拥塞的有关因素考虑周到，力求网络在工作时不产生拥塞但一旦整个系统运行起来，就不再中途进行改正了

闭环控制：基于反馈环路的概念，主要有以下几种措施：
监测网络系统以便检测到拥塞在何时、何处发生
把拥塞发生的信息传送到可采取行动的地方
调整网络系统的运行以解决出现的问题



假定：

- 1数据是单方向传送的，对方只传送确认报文但实际上TCP是双向传输的
- 2接收方总是有足够大的缓存空间，因而发送窗口的大小由网络的拥塞程度来决定

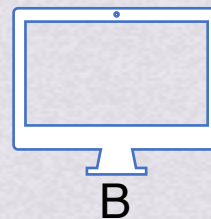
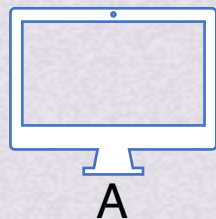


TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



发送窗口= $\min\{\text{拥塞窗口}, \text{缓存空间}\}$

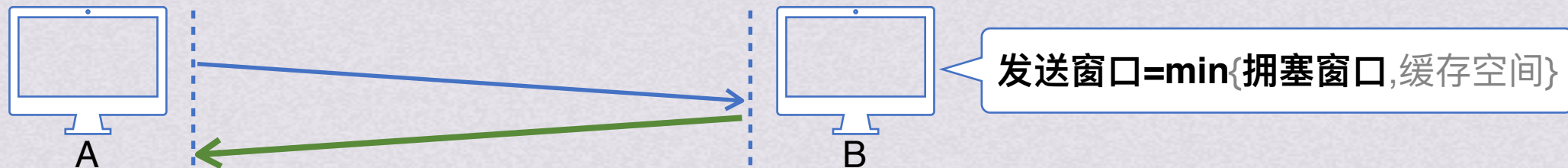


TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



网络阻塞时, 路由器会把来不及处理而又排不上队的分组丢弃
只要发送方没有按时收到接收方的确认报文, 也就是说出现了超时, 就可以估计网络某处出现了拥塞

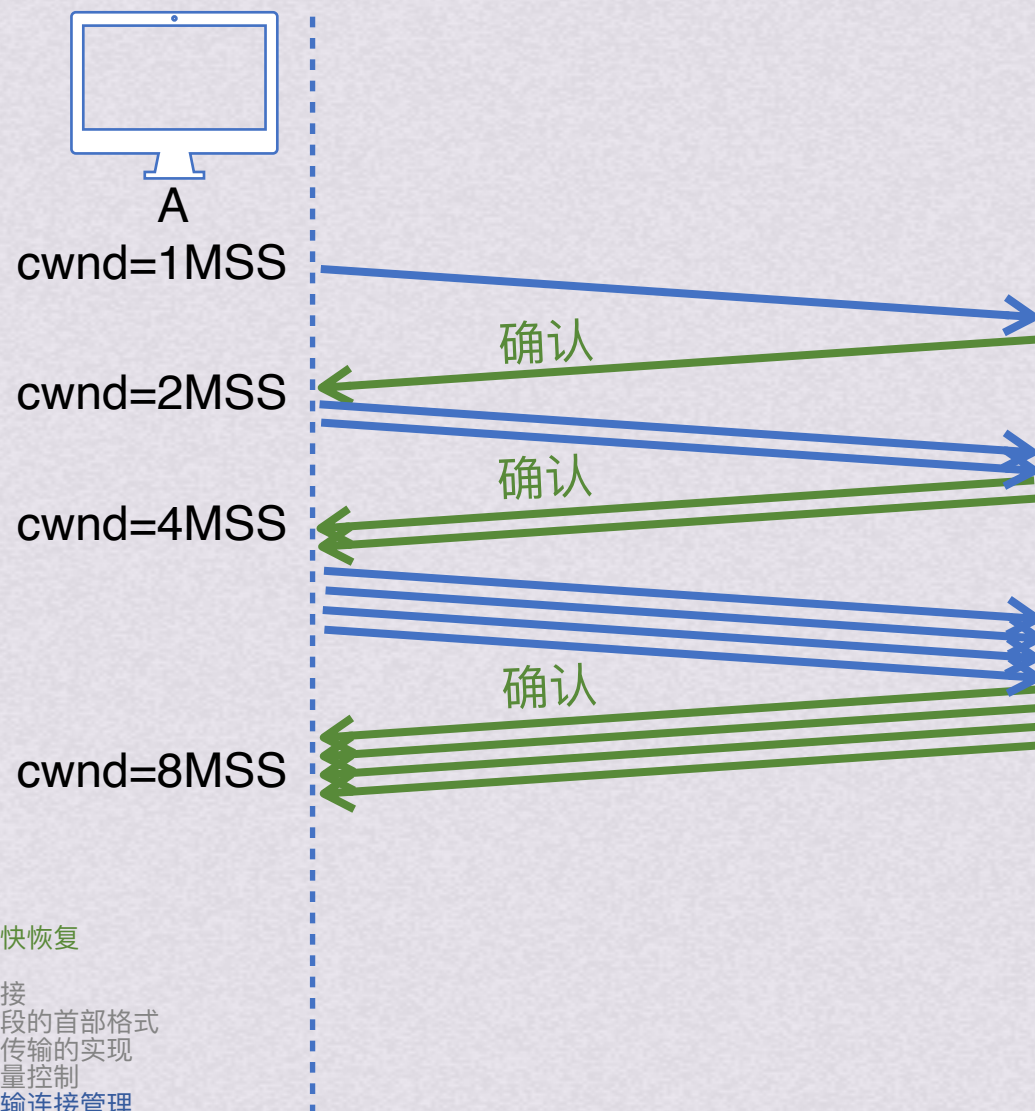


TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



发送窗口=min{拥塞窗口,缓存空间}

慢开始算法的思路是: 当主机在已建立的TCP连接上开始发送数据时, 并不清楚网络当前的负荷情况

如果立即把大量数据注入网络, 有可能会引起网络拥塞

应该由小到大逐渐增大拥塞窗口数值

在刚刚开始发送报文段时, 先把初始拥塞窗口cwnd设置为1个最大报文段MSS的数值。在每收到一个对新的报文段的确认后, 就可以把拥塞窗口增加一个MSS的数值

注意: TCP是用字节数作为窗口大小的单位

但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位

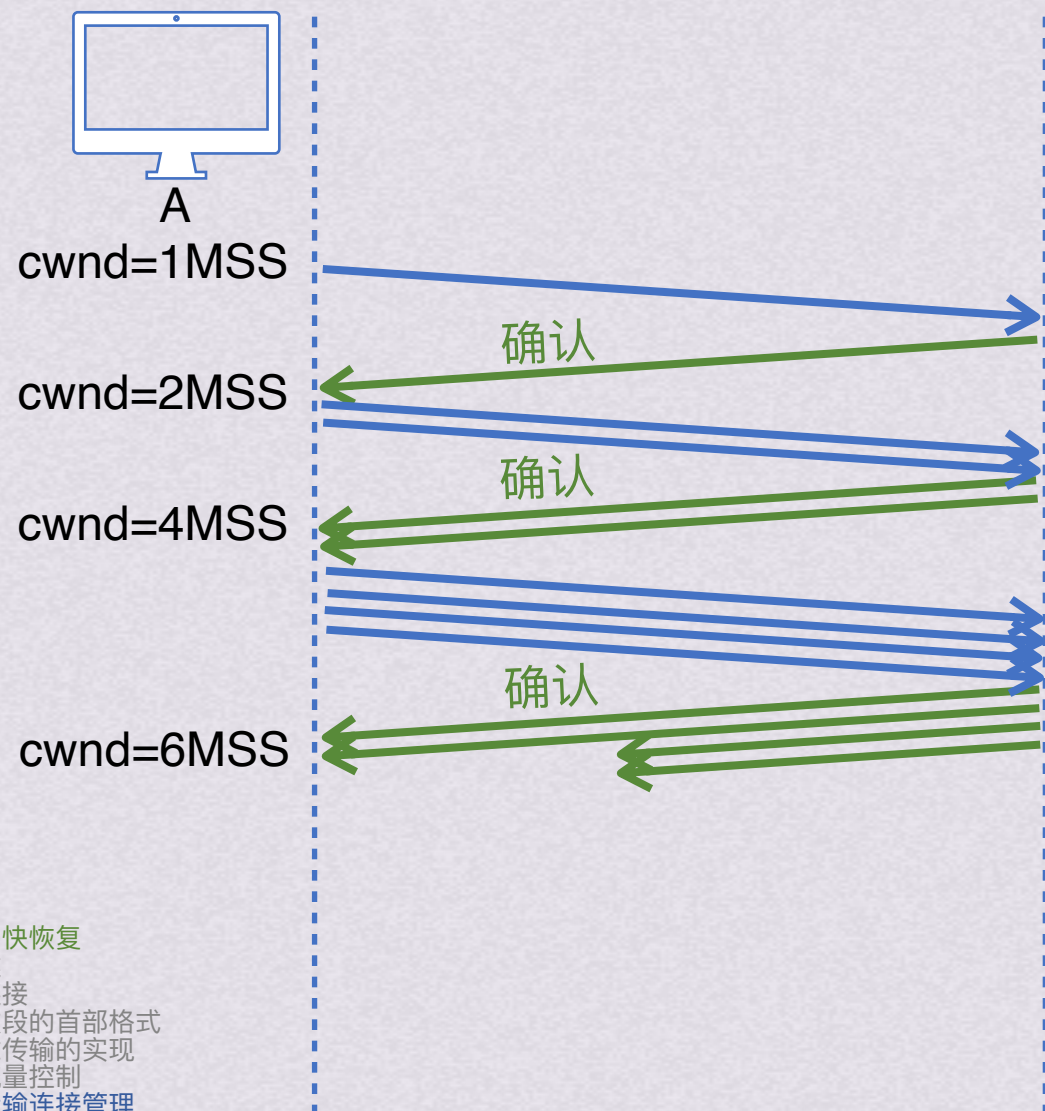


TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



发送窗口= $\min\{\text{拥塞窗口}, \text{缓存空间}\}$

慢开始算法的思路是: 当主机在已建立的TCP连接上开始发送数据时, 并不清楚网络当前的负荷情况

如果立即把大量数据注入网络, 有可能会引起网络拥塞

应该由小到大逐渐增大拥塞窗口数值

在刚刚开始发送报文段时, 先把初始拥塞窗口cwnd设置为1个最大报文段MSS的数值。在每收到一个对新的报文段的确认后, 就可以把拥塞窗口增加一个MSS的数值

注意: TCP是用字节数作为窗口大小的单位

但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位



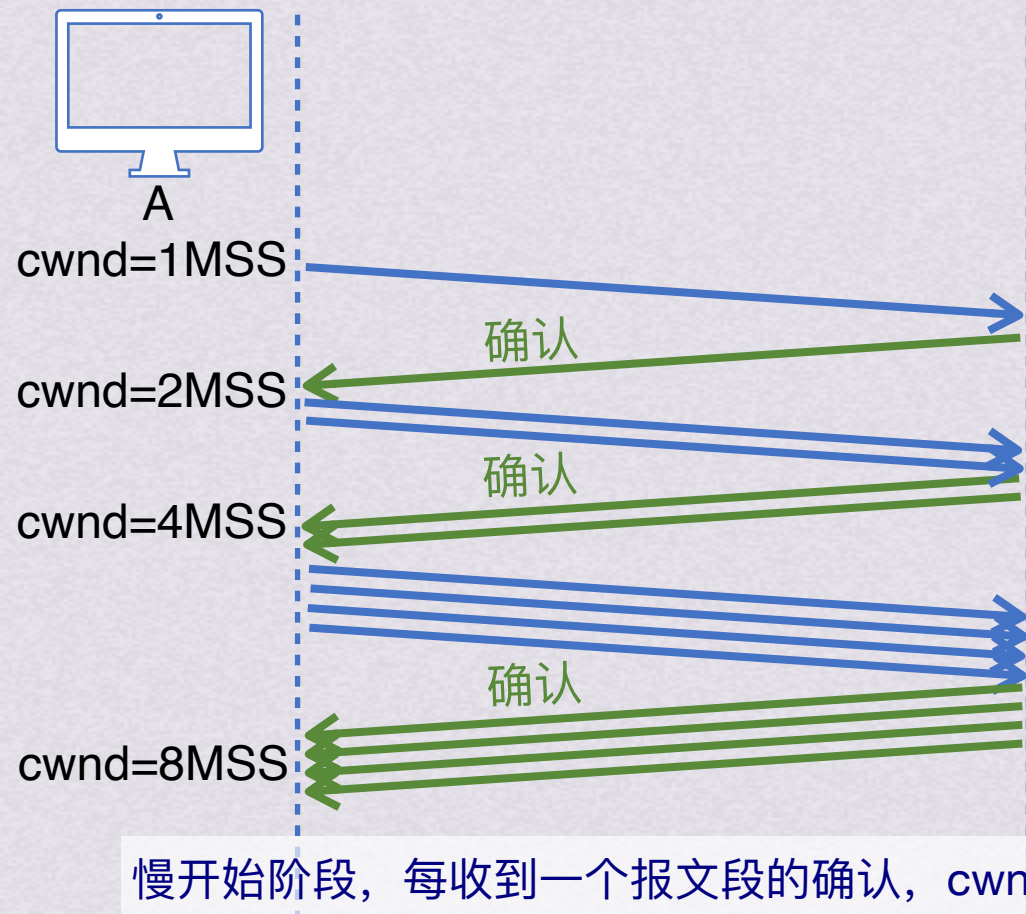
TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口

发送窗口= $\min\{\text{拥塞窗口}, \text{缓存空间}\}$



慢开始算法的思路是: 当主机在已建立的TCP连接上开始发送数据时, 并不清楚网络当前的负荷情况

如果立即把大量数据注入网络, 有可能会引起网络拥塞

应该由小到大逐渐增大拥塞窗口数值

在刚刚开始发送报文段时, 先把初始拥塞窗口cwnd设置为1个最大报文段MSS的数值。在每收到一个对新的报文段的确认后, 就可以把拥塞窗口增加一个MSS的数值

注意: TCP是用字节数作为窗口大小的单位

但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位

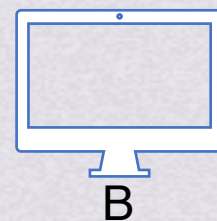
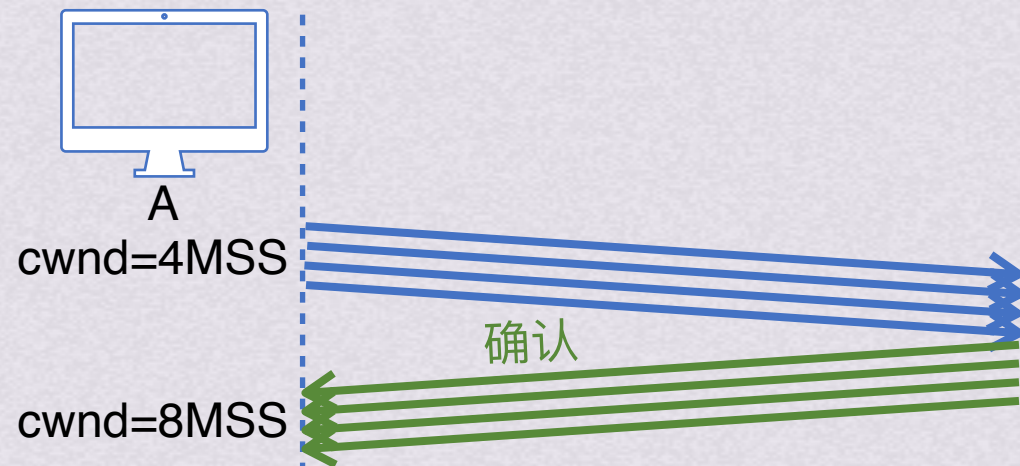


TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



发送窗口= $\min\{\text{拥塞窗口}, \text{缓存空间}\}$

慢开始算法的思路是: 当主机在已建立的TCP连接上开始发送数据时, 并不清楚网络当前的负荷情况

如果立即把大量数据注入网络, 有可能会引起网络拥塞

应该由小到大逐渐增大拥塞窗口数值

为了防止cwnd增长过大引起网络拥塞, 还需要设置一个慢开始门限ssthresh, 用法如下:

当 $cwnd < ssthresh$ 时, 使用慢开始算法

当 $cwnd > ssthresh$ 时, 停止使用慢开始算法, 改用拥塞避免算法

当 $cwnd = ssthresh$ 时, 二者都可以使用

拥塞避免算法的目的是让cwnd缓慢增大, 思路如下: 每经过一个RTT, 发送方的拥塞窗口cwnd就加1, 按线性规律缓慢增长

注意: TCP是用字节数作为窗口大小的单位

但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位

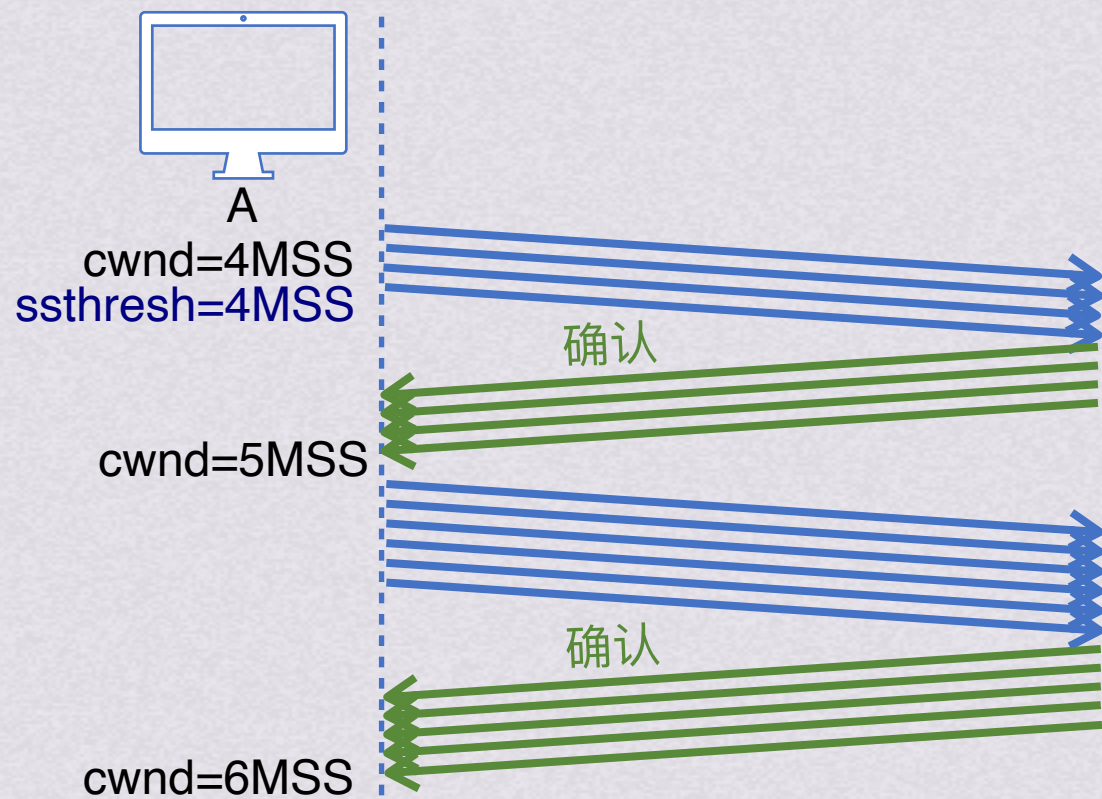


TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



发送窗口 = $\min\{\text{拥塞窗口}, \text{缓存空间}\}$

慢开始算法的思路是: 当主机在已建立的TCP连接上开始发送数据时, 并不清楚网络当前的负荷情况

如果立即把大量数据注入网络, 有可能会引起网络拥塞
应该由小到大逐渐增大拥塞窗口数值

为了防止cwnd增长过大引起网络拥塞, 还需要设置一个慢开始门限 $ssthresh$, 用法如下:

当 $cwnd < ssthresh$ 时, 使用慢开始算法

当 $cwnd > ssthresh$ 时, 停止使用慢开始算法, 改用拥塞避免算法

当 $cwnd = ssthresh$ 时, 二者都可以使用

拥塞避免算法的目的是让cwnd缓慢增大, 思路如下: 每经过一个RTT, 发送方的拥塞窗口cwnd就加1, 按线性规律缓慢增长

注意: TCP是用字节数作为窗口大小的单位

但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位



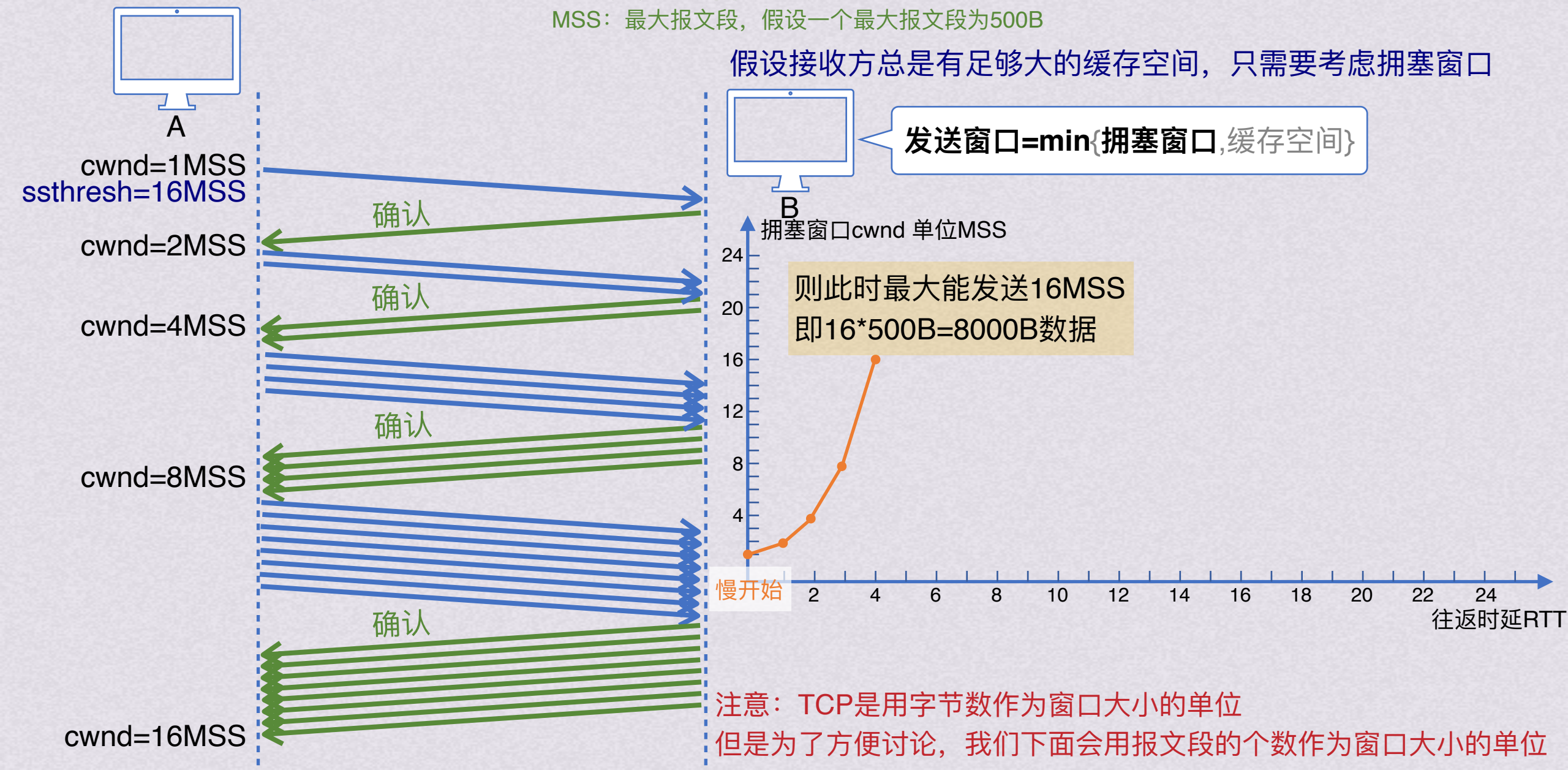
TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口

发送窗口 = $\min\{\text{拥塞窗口}, \text{缓存空间}\}$





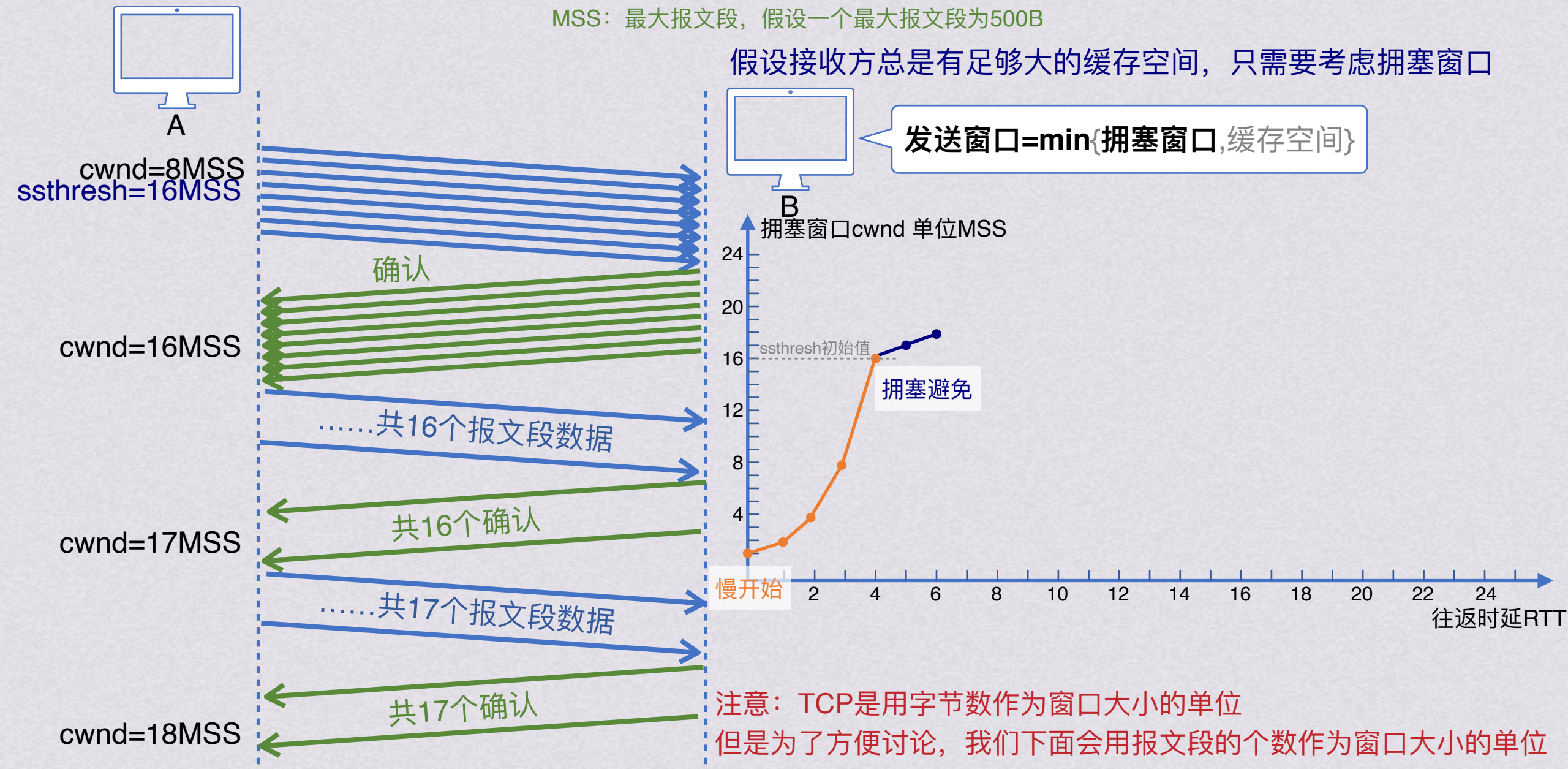
TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口

发送窗口 = $\min\{\text{拥塞窗口}, \text{缓存空间}\}$



注意: TCP是用字节数作为窗口大小的单位
但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位

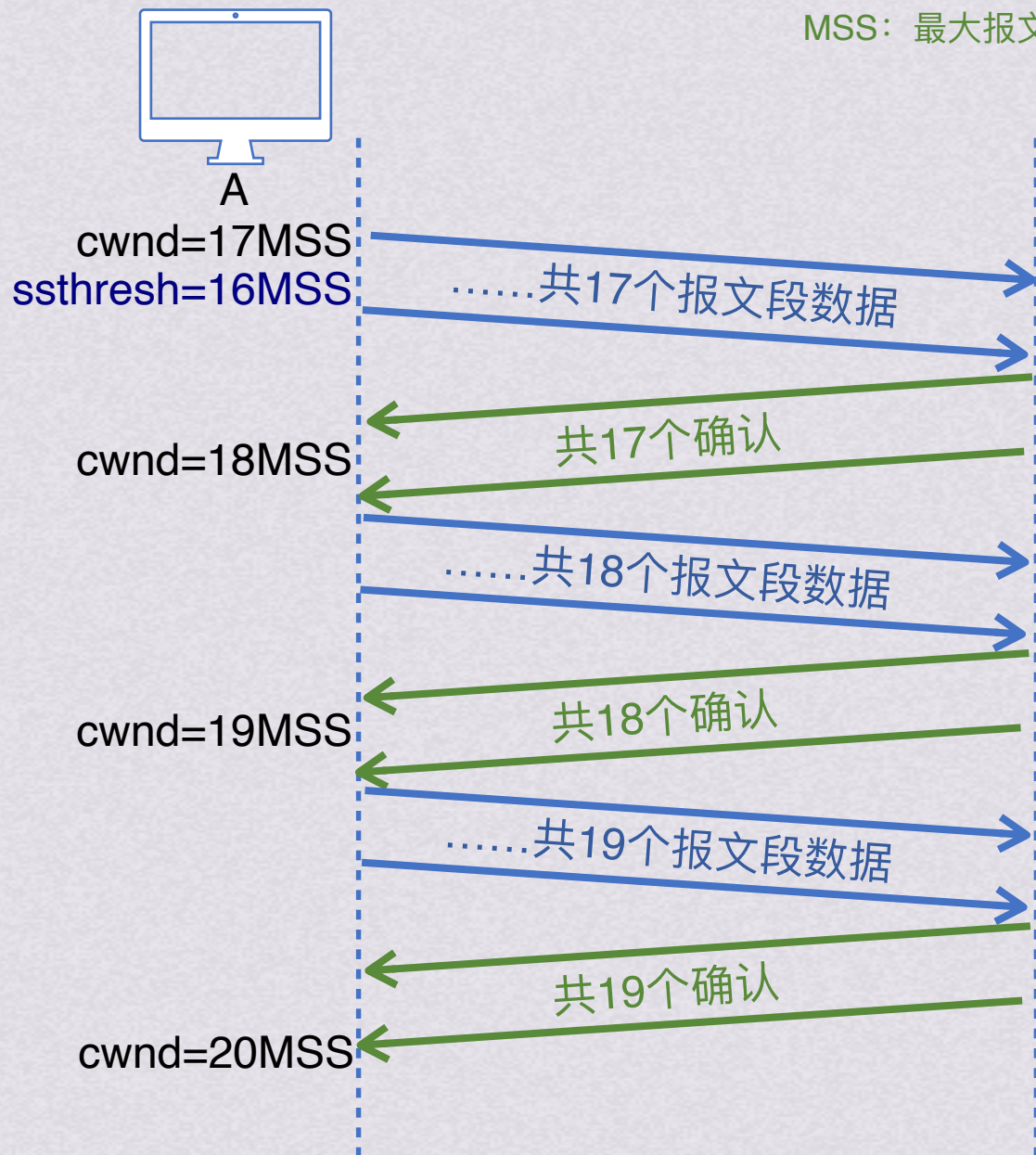


TCP的拥塞控制

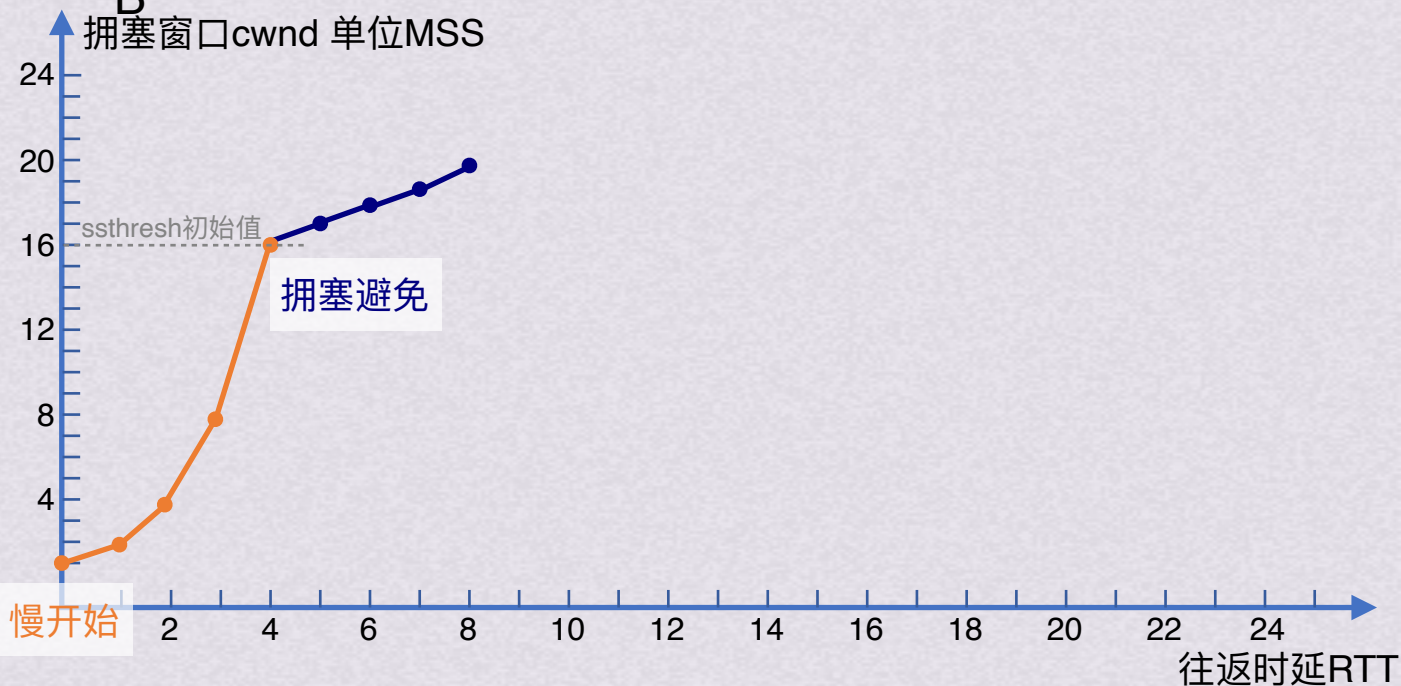
慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



发送窗口=min{拥塞窗口,缓存空间}



注意: TCP是用字节数作为窗口大小的单位
但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位

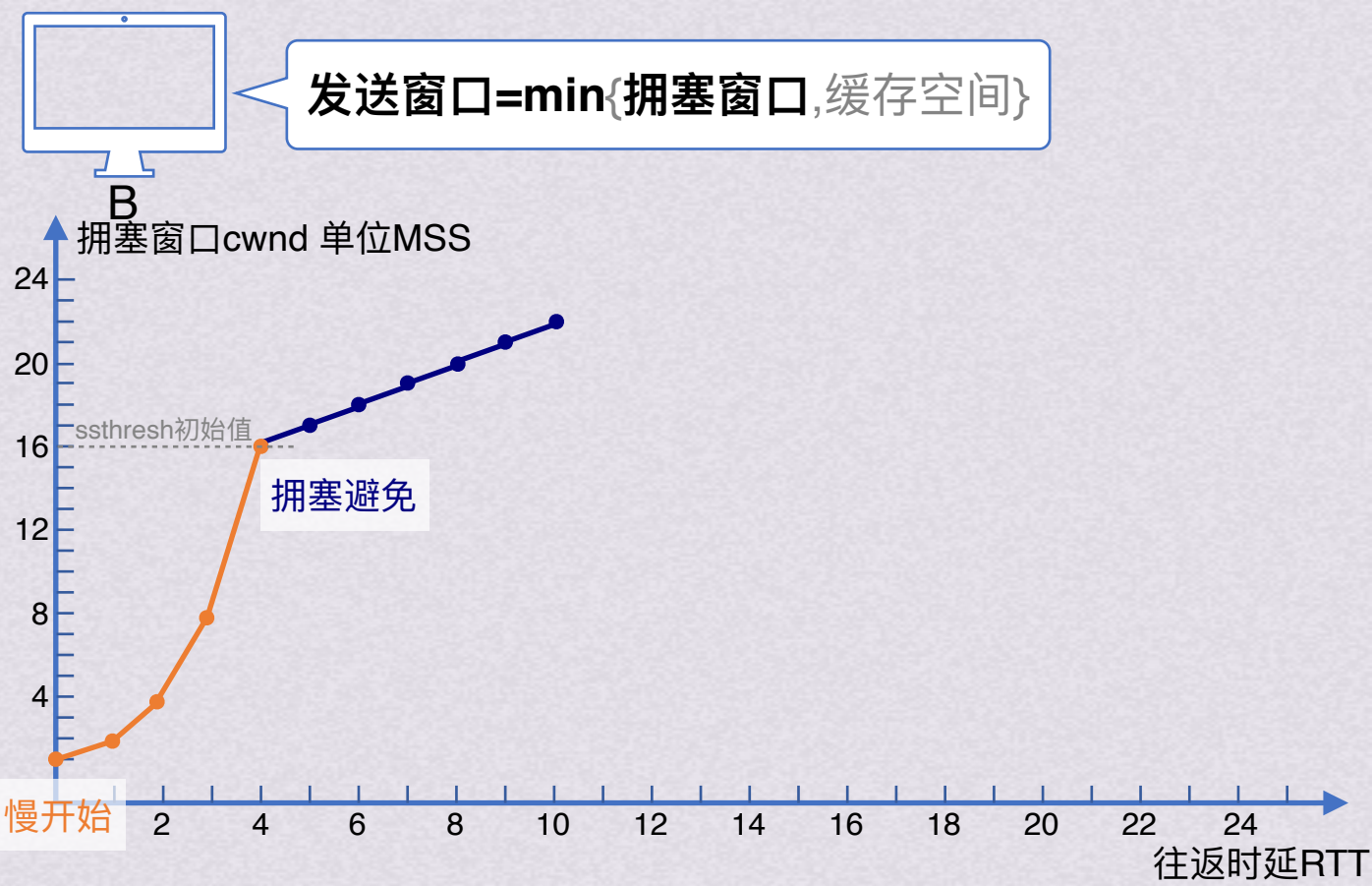
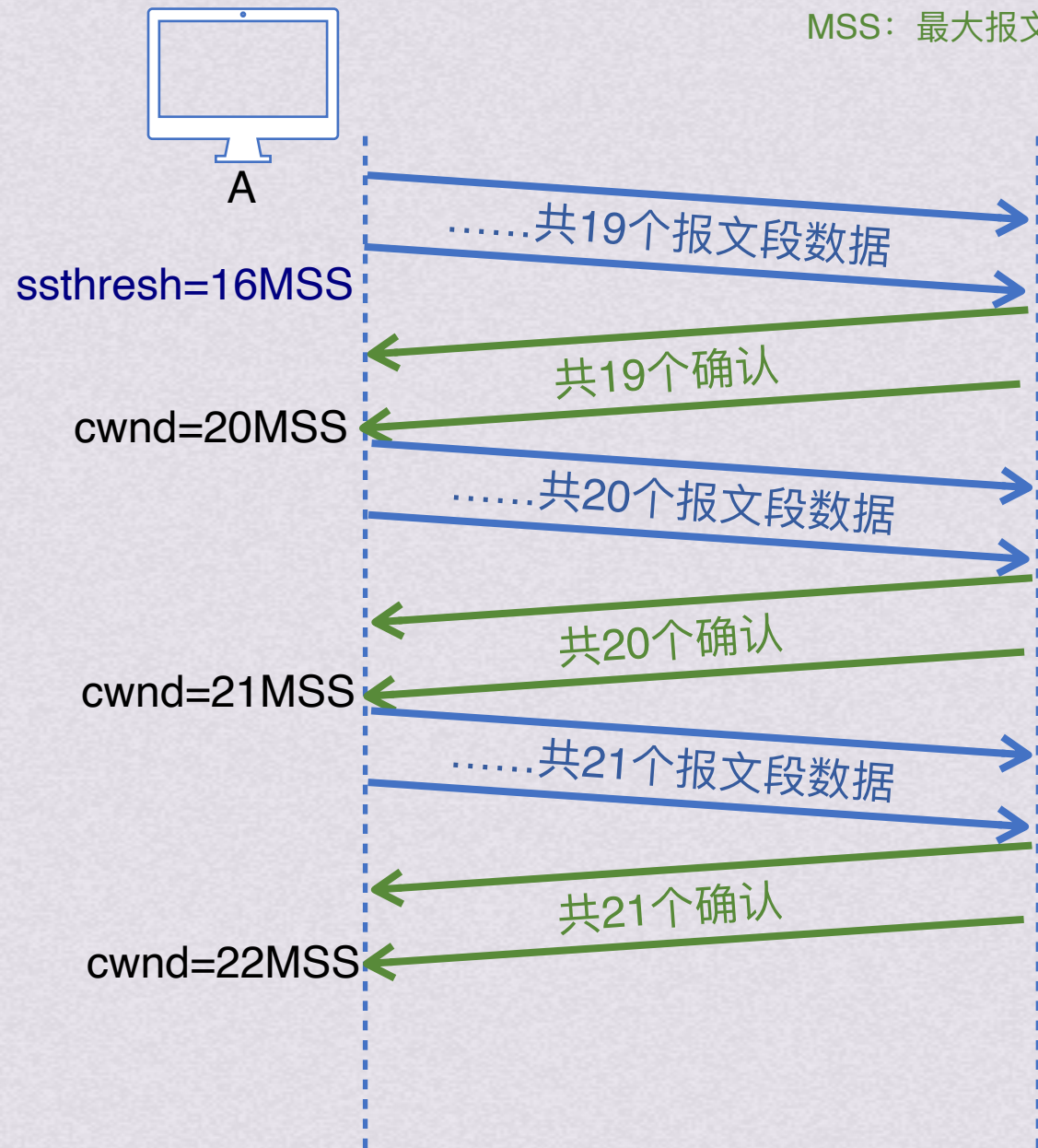


TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



注意: TCP是用字节数作为窗口大小的单位
但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位



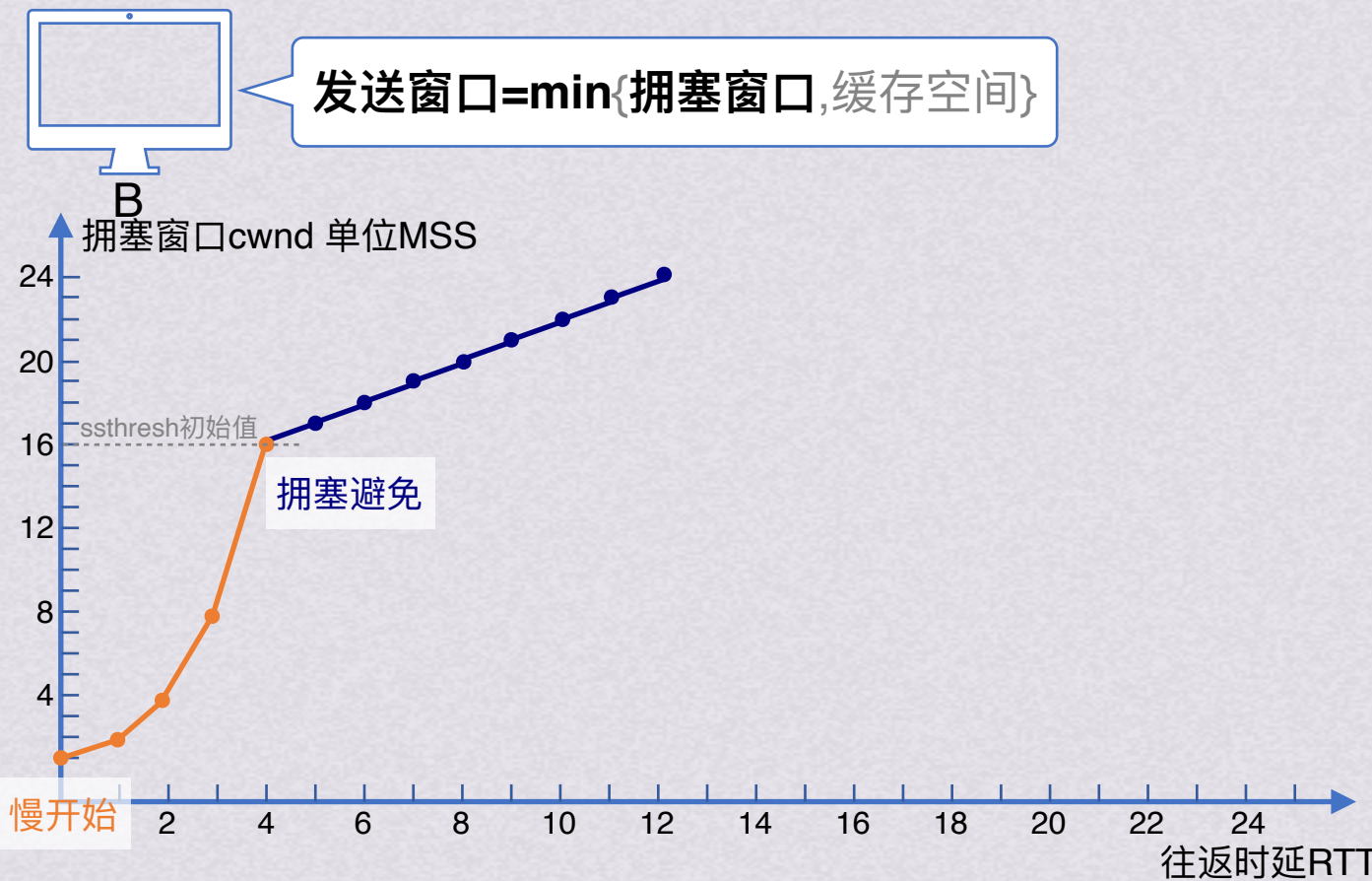
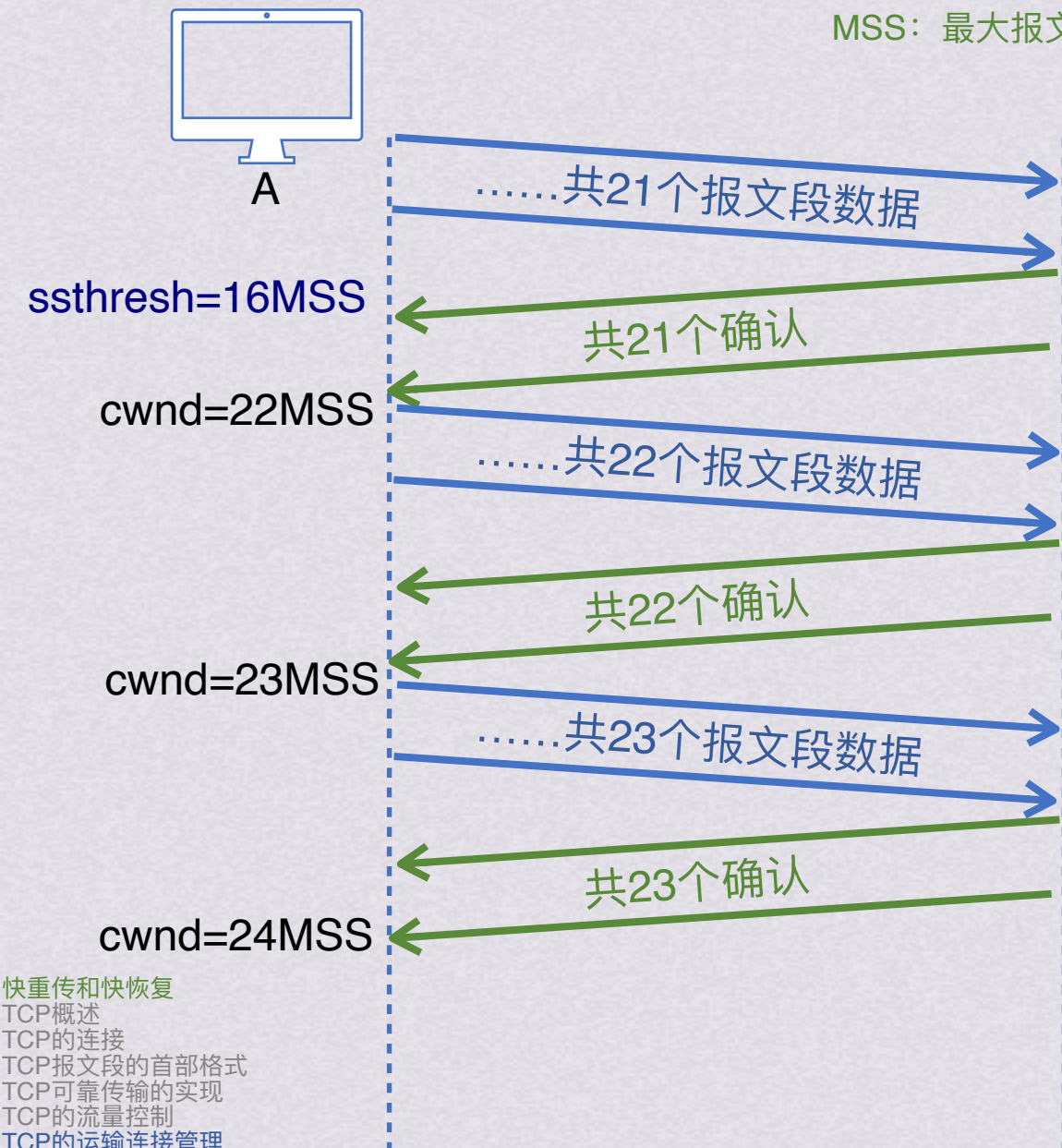
TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口

发送窗口 = $\min\{\text{拥塞窗口}, \text{缓存空间}\}$



注意: TCP是用字节数作为窗口大小的单位
但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位

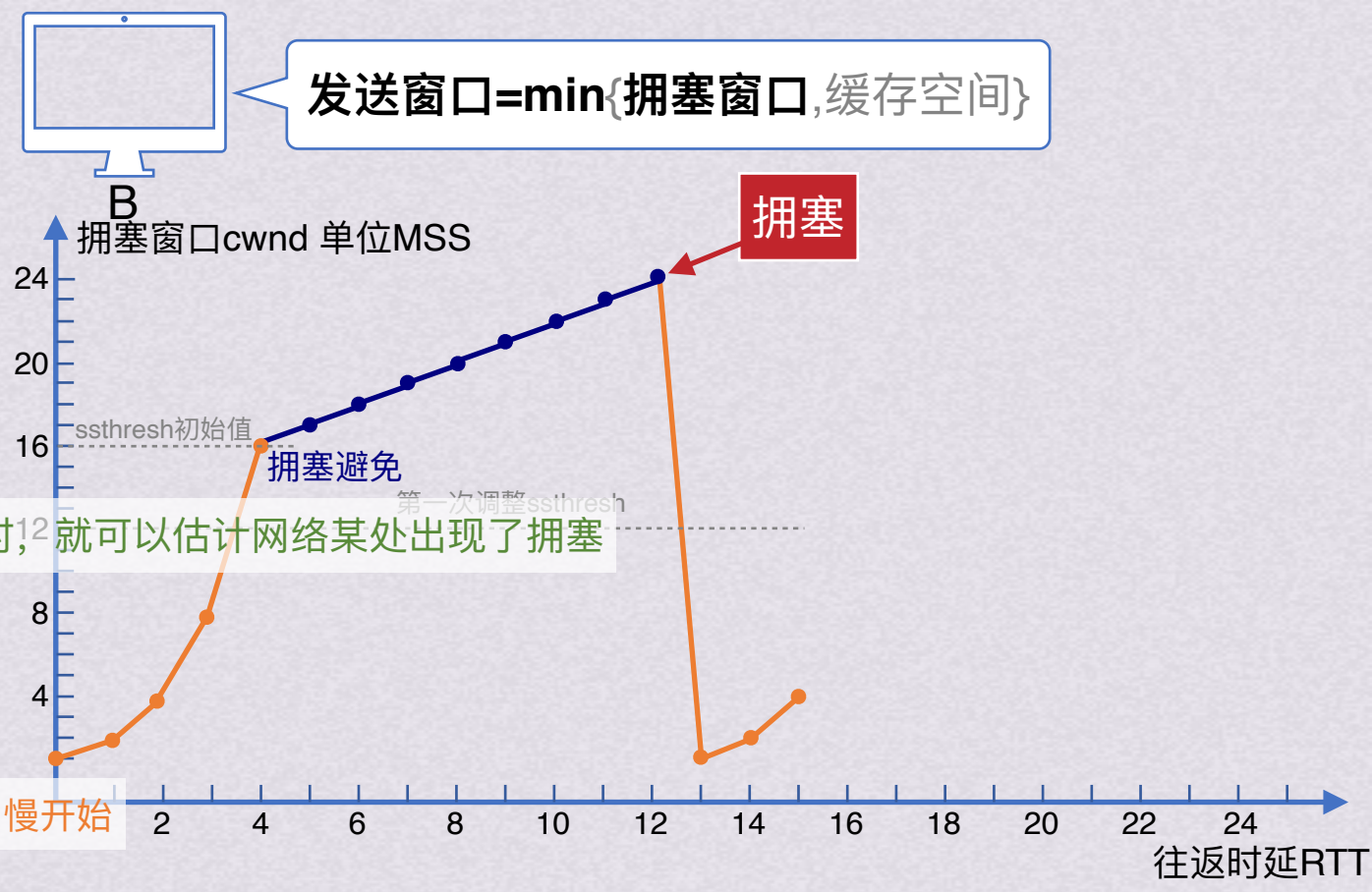
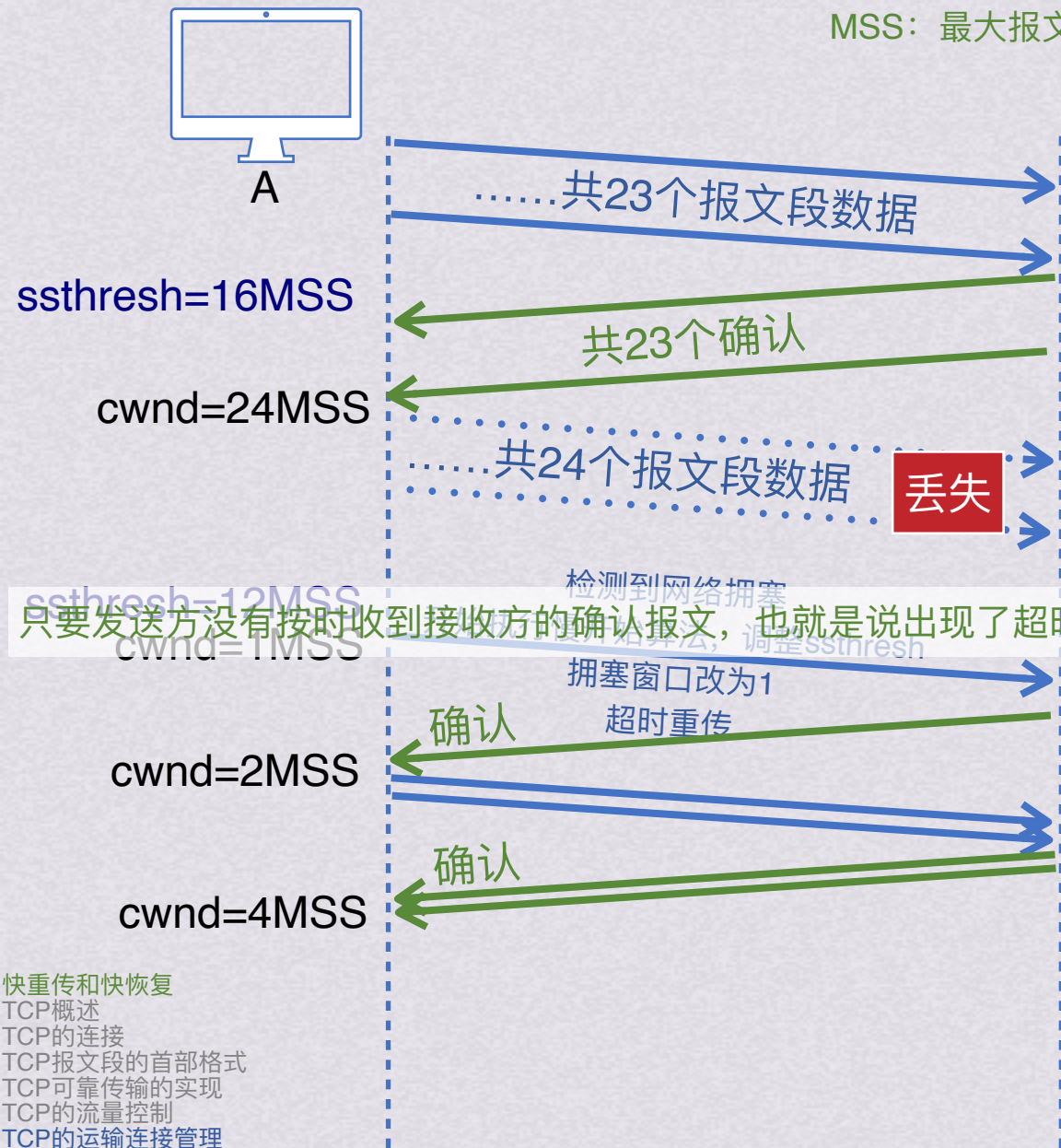


TCP的拥塞控制

慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



注意: TCP是用字节数作为窗口大小的单位
但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位

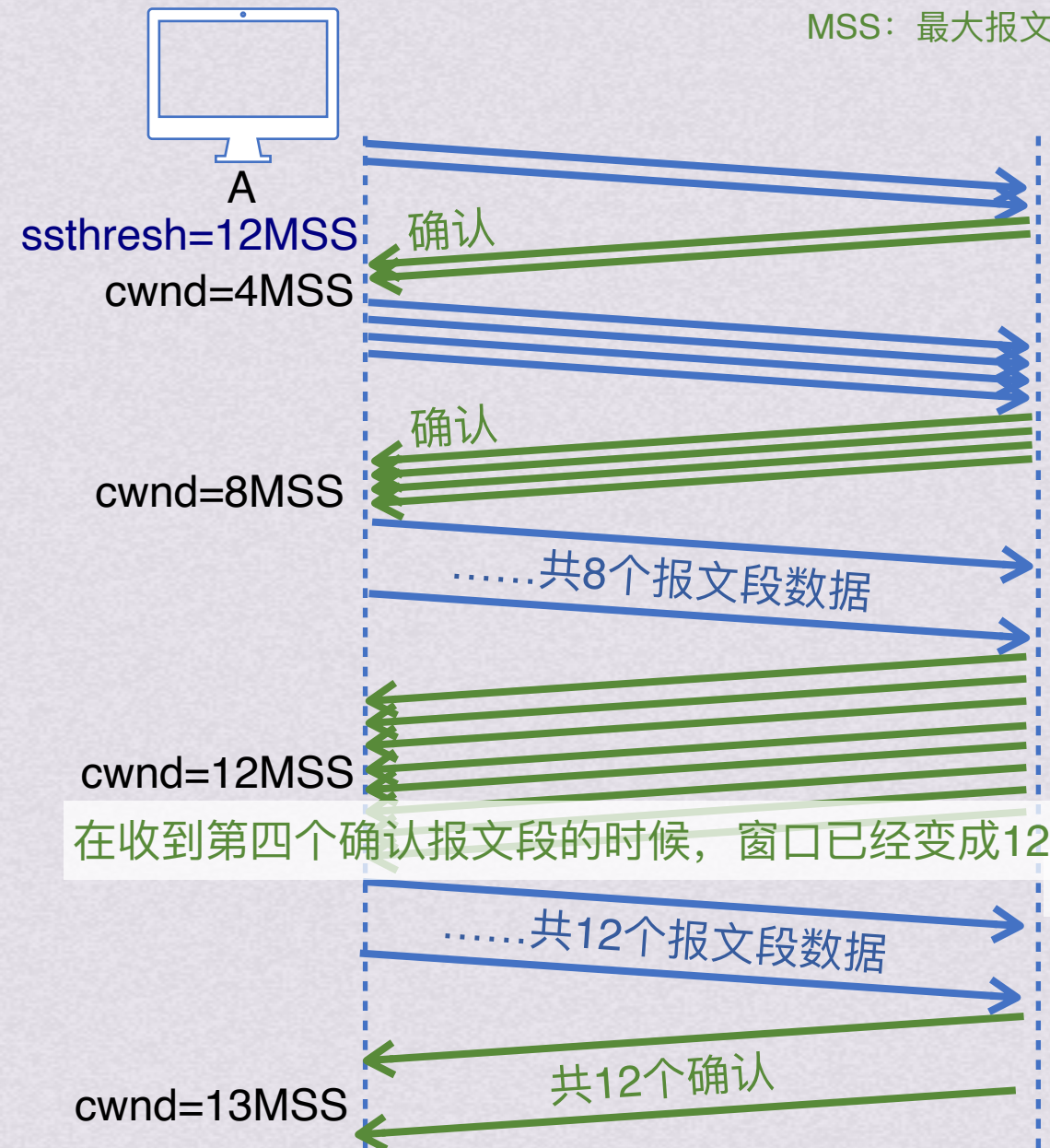


TCP的拥塞控制

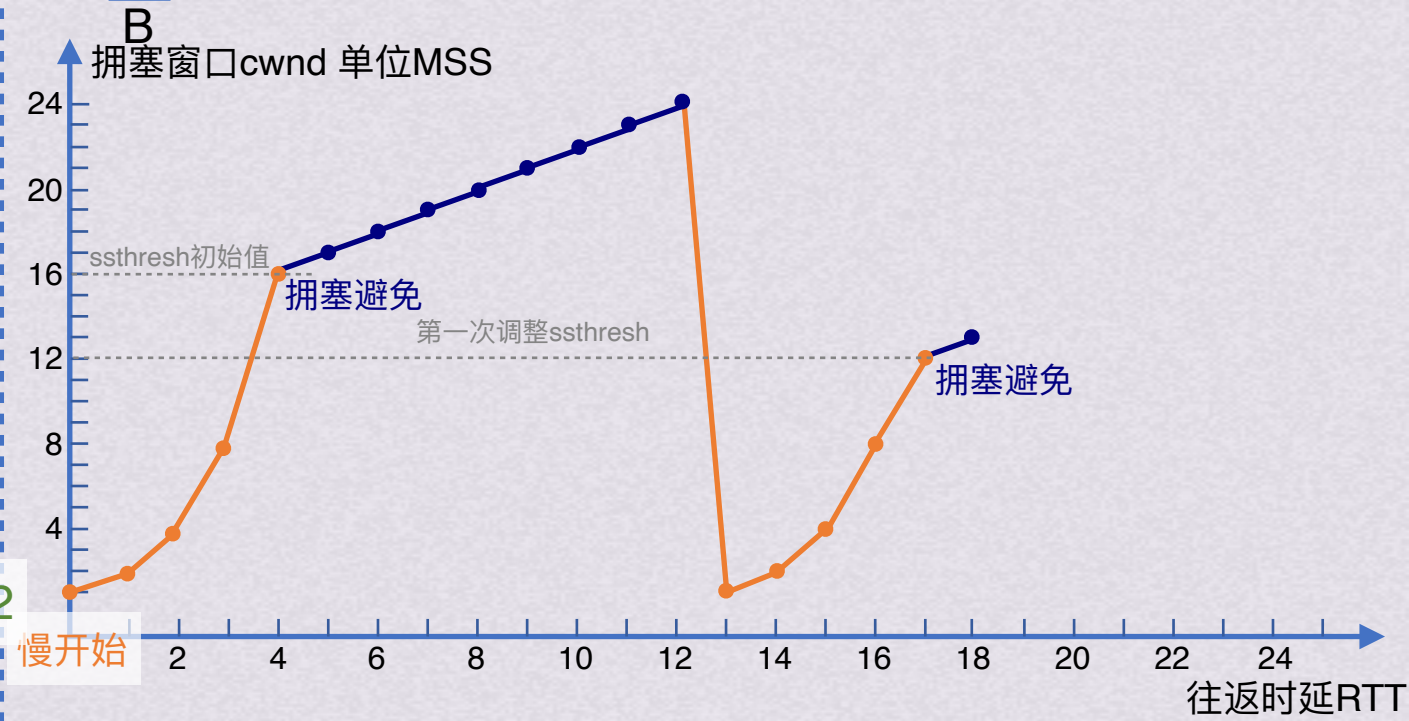
慢开始和拥塞避免

MSS: 最大报文段, 假设一个最大报文段为500B

假设接收方总是有足够大的缓存空间, 只需要考虑拥塞窗口



发送窗口=min{拥塞窗口,缓存空间}



注意: TCP是用字节数作为窗口大小的单位
但是为了方便讨论, 我们下面会用报文段的个数作为窗口大小的单位



TCP的拥塞控制

慢开始和拥塞避免

[2009]一个TCP连接总以1KB的最大段长发送TCP段，发送方有足够多的数据要发送，当拥塞窗口为16KB时发生了超时，若接下来的4个RTT时间内的TCP段的传输都是成功的，则当第4个RTT时间内发送的所有TCP段都得到肯定应答时，拥塞窗口大小是

A.7KB

B.8KB

C.9KB

D.16KB



TCP的拥塞控制

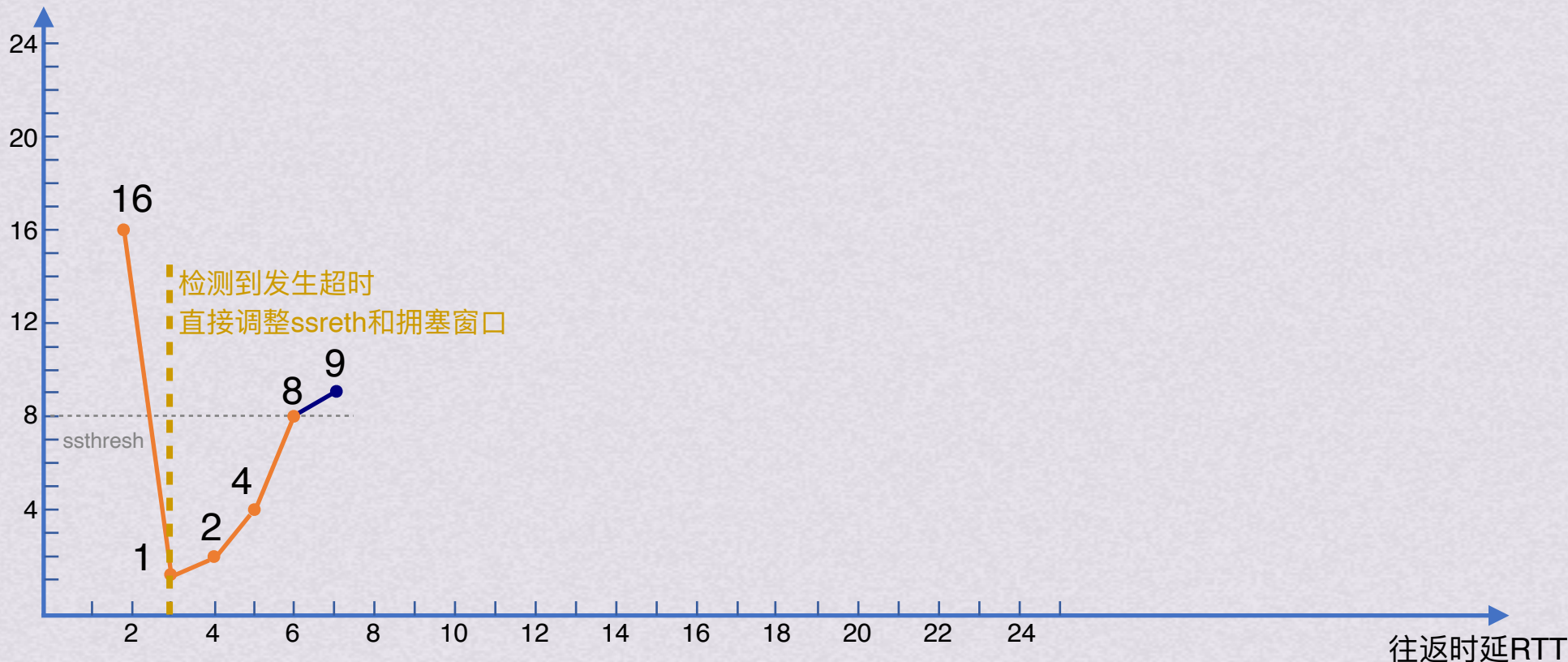
慢开始和拥塞避免

[2009]一个TCP连接总以1KB的最大段长发送TCP段，发送方有足够多的数据要发送，当拥塞窗口为16KB时发生了超时，若接下来的4个RTT时间内的TCP段的传输都是成功的，则当第4个RTT时间内发送的所有TCP段都得到肯定应答时，拥塞窗口大小是**C.9KB**

A.7KB

B.8KB

D.16KB





TCP的拥塞控制

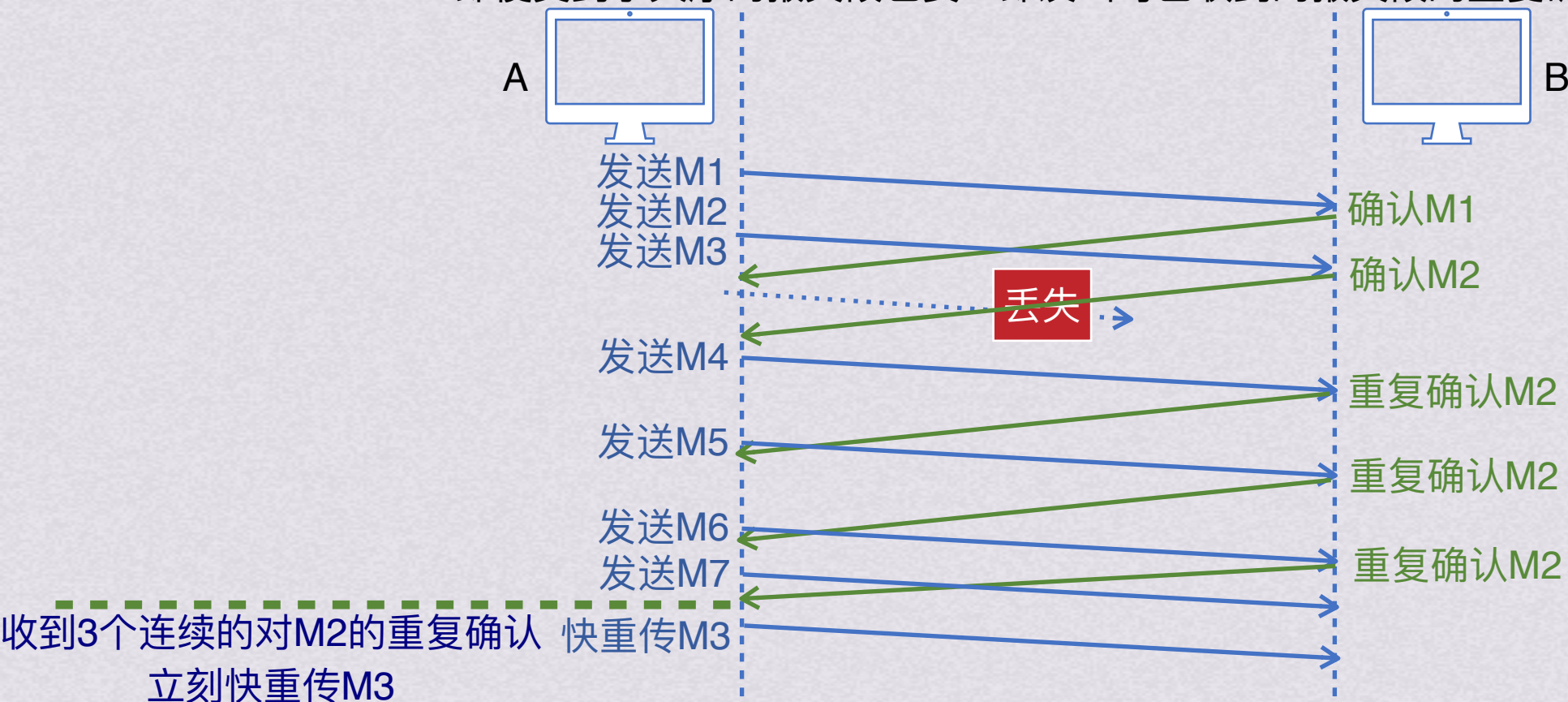
快重传和快恢复



TCP的拥塞控制

快重传和快恢复

快重传算法首先要求接收方不要等待自己发送数据时才进行捎带确认，而是要立即发送确认
即使受到了失序的报文段也要立即发出对已收到的报文段的重复确认

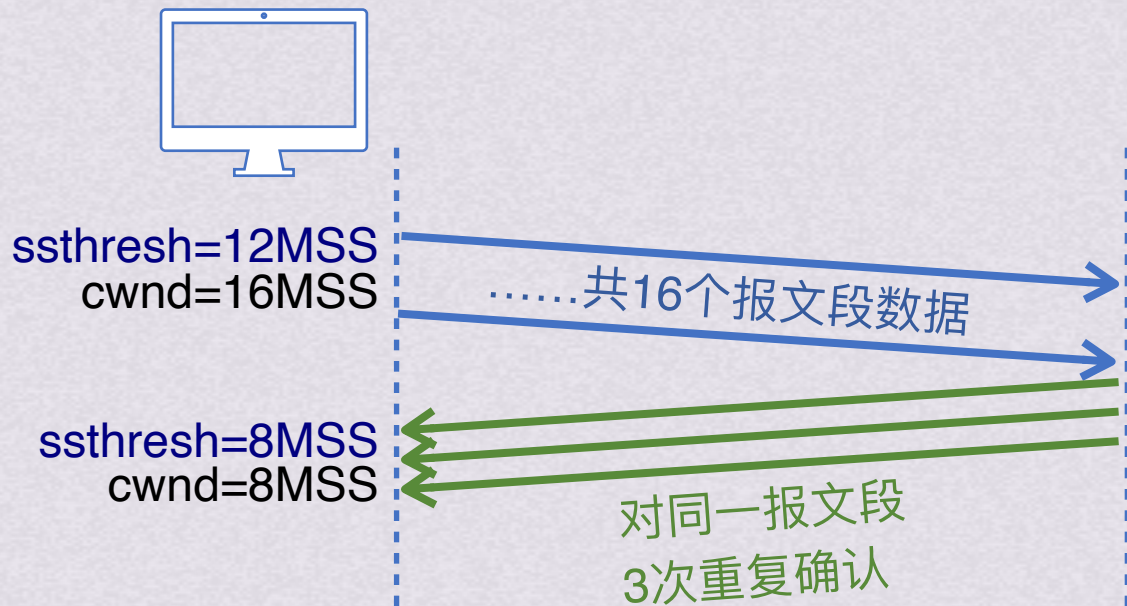


简单来说，快重传就是在收到3次重复确认后，会立刻对该报文进行重传



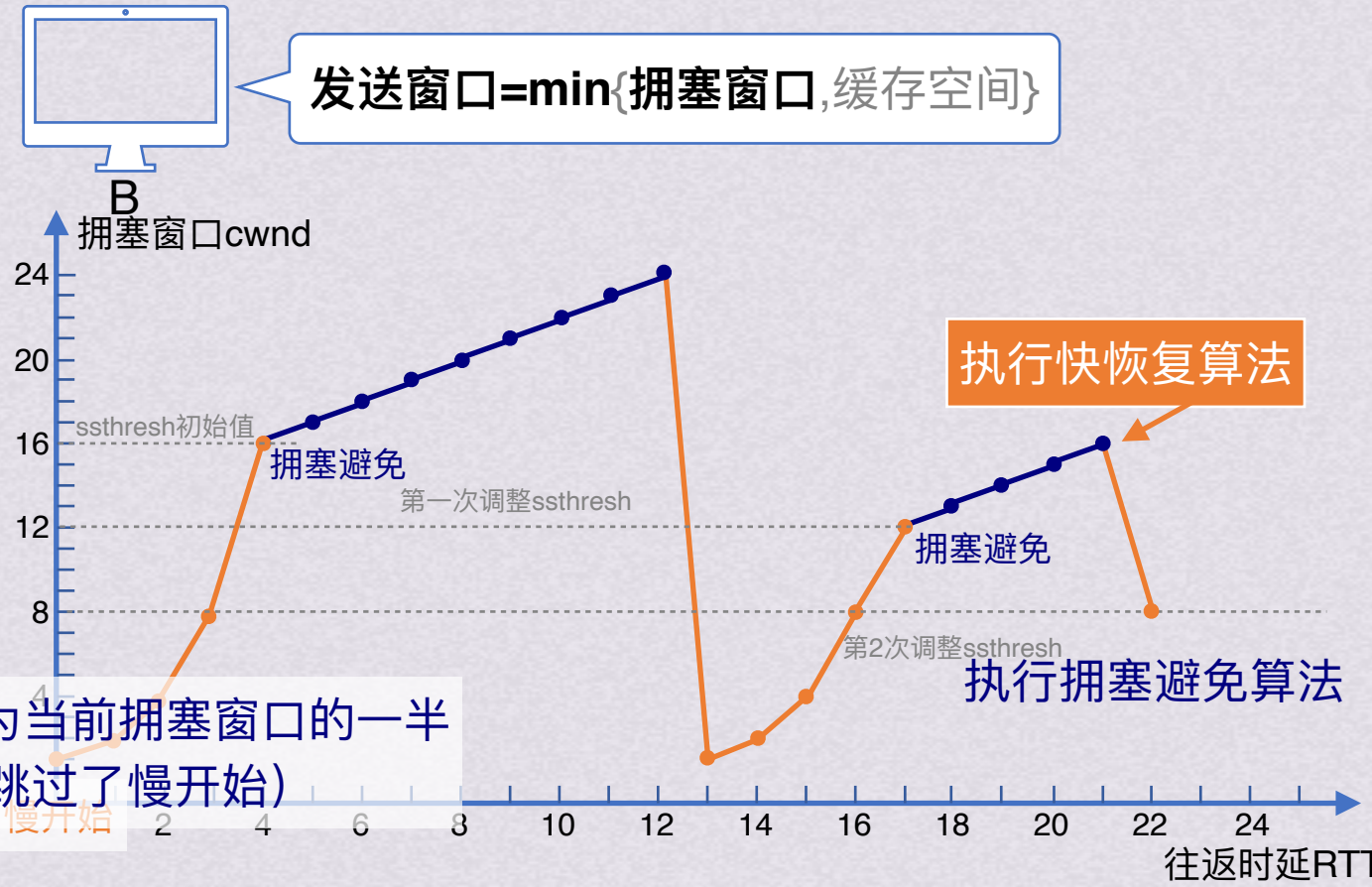
TCP的拥塞控制

快重传和快恢复



假设接收方总是有足够大的缓存空间，只需要考虑拥塞窗口

发送窗口= $\min\{\text{拥塞窗口}, \text{缓存空间}\}$



快恢复就是在收到3次重复确认后，将ssthresh调整为当前拥塞窗口的一半并且让cwnd=ssthresh，直接执行拥塞避免（跳过了慢开始）

注意：TCP是用字节数作为窗口大小的单位
但是为了方便讨论，我们下面会用报文段的个数作为窗口大小的单位

- 慢开始和拥塞避免
- TCP概述
- TCP的连接
- TCP报文段的首部格式
- TCP可靠传输的实现
- TCP的流量控制
- TCP的运输连接管理



TCP的拥塞控制

快重传和快恢复

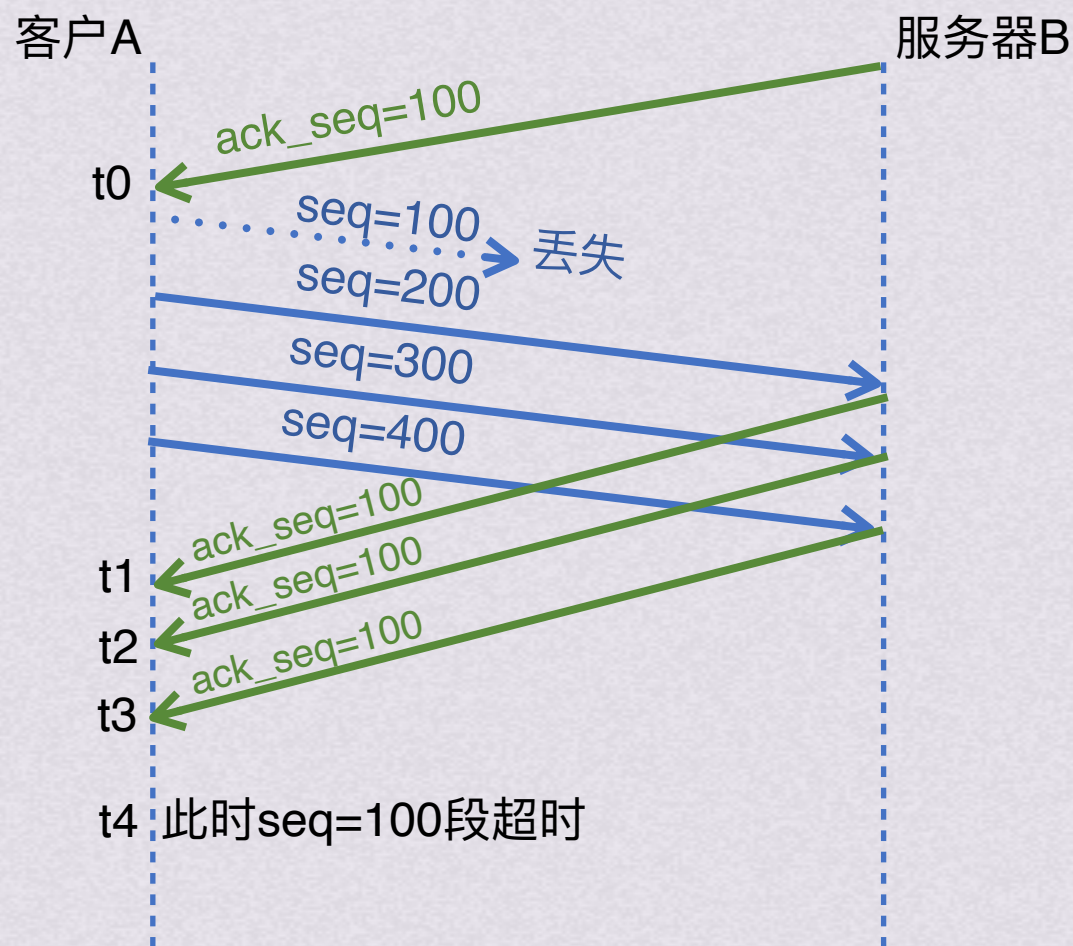
[2019]某客户通过一个TCP连接向服务器发送数据的部分过程如下图所示。客户在t0时刻第一次收到确认序列号ack_seq=100的段，并发送序列号seq=100的段，但发生丢失。若TCP支持快速重传，则TCP发送seq=100的段的时刻是

A.t1

B.t2

C.t3

D.t4





TCP的拥塞控制

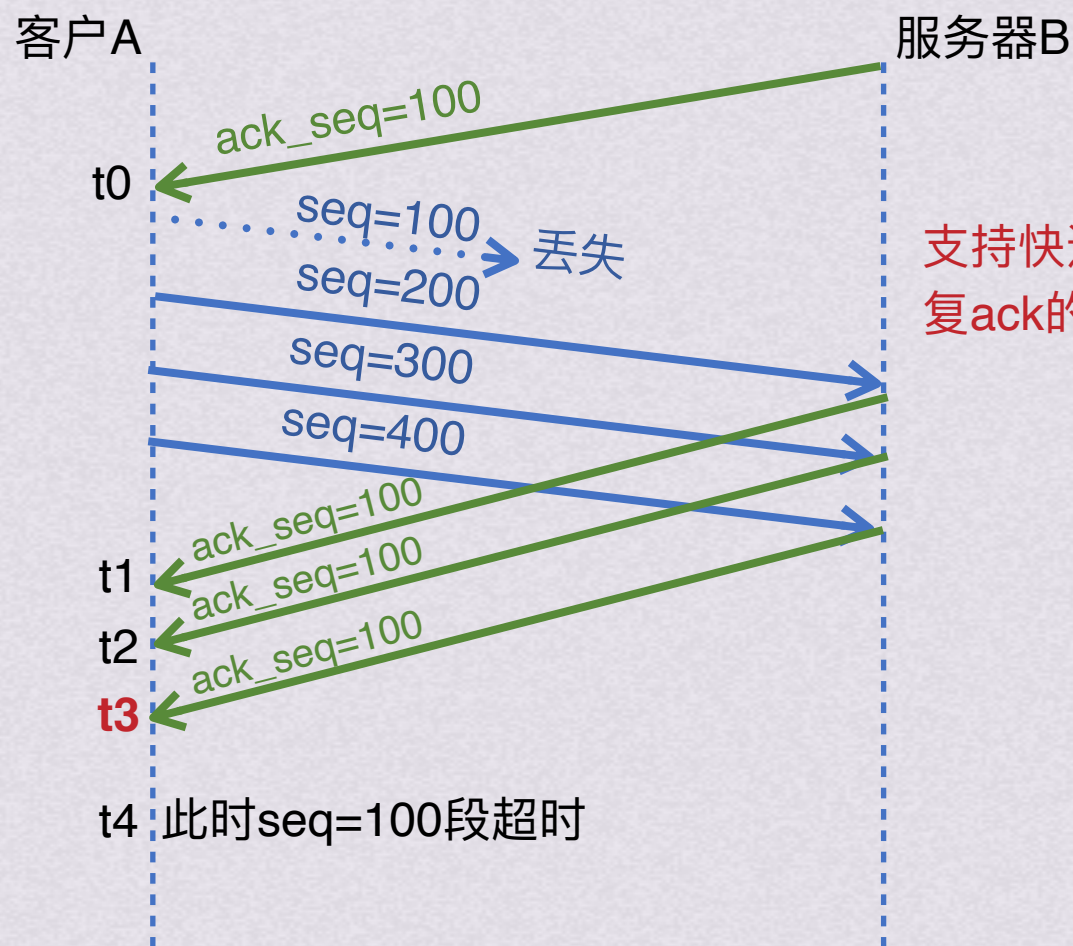
快重传和快恢复

[2019]某客户通过一个TCP连接向服务器发送数据的部分过程如下图所示。客户在 t_0 时刻第一次收到确认序列号 $ack_seq=100$ 的段，并发送序列号 $seq=100$ 的段，但发生丢失。若TCP支持快速重传，则TCP发送 $seq=100$ 的段的时刻是 **C.t3**

A.t1

B.t2

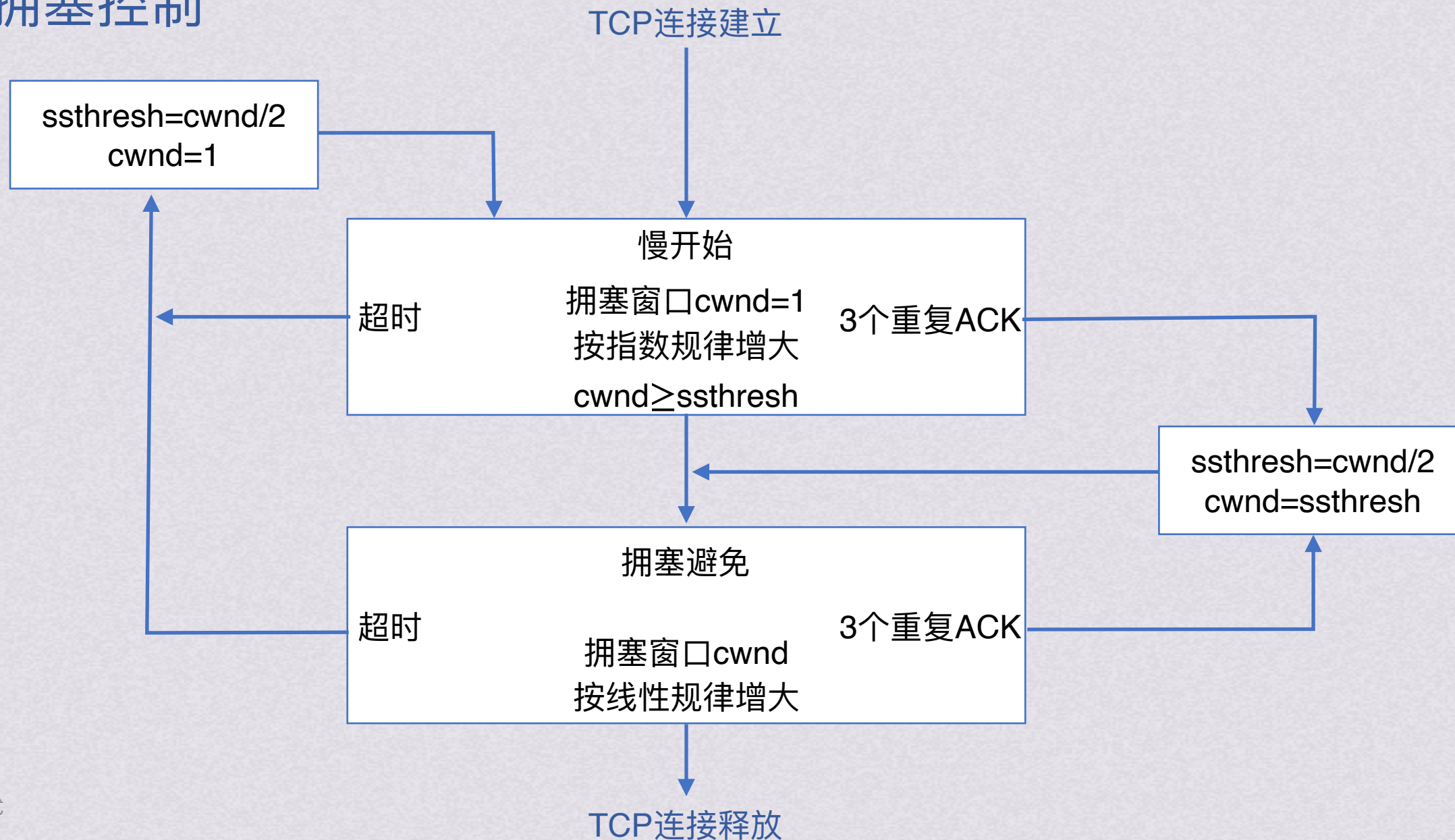
D.t4



支持快速重传的TCP会在连续收到3个重复ack的时候重传丢失报文



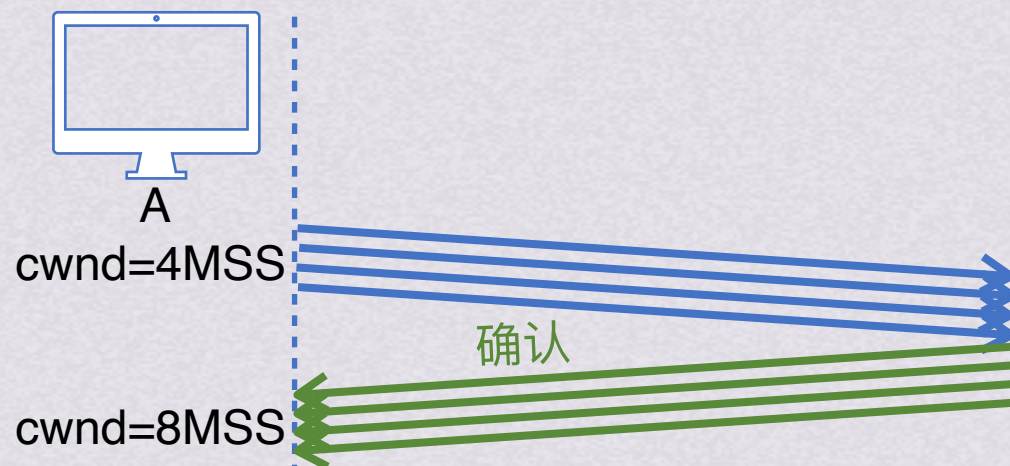
TCP的拥塞控制





TCP的拥塞控制

假设接收方总是有足够大的缓存空间，只需要考虑拥塞窗口



发送窗口= $\min\{\text{拥塞窗口}, \text{缓存空间}\}$

实际上，接收方的缓存空间总是有限的接收方根据自己的接收能力设定了接收窗口 $rwnd$ ，并把这个窗口值写入TCP首部中的窗口字段，发给发送方

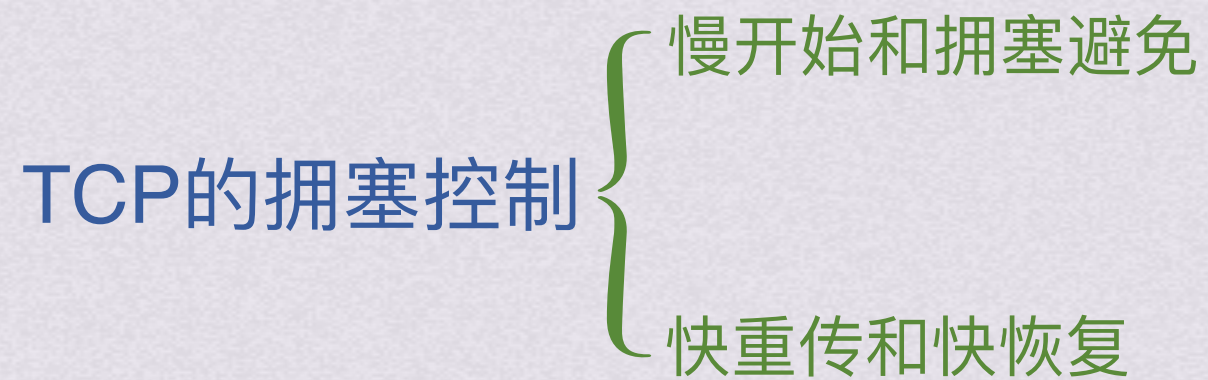
因此，如果要把这个 $rwnd$ 和拥塞情况同时考虑，发送方窗口的上限值= $\min[rwnd, cwnd]$

当 $rwnd < cwnd$ 时，接收方的接收能力限制发送窗口的最大值

当 $cwnd < rwnd$ 时，则是网络的拥塞程度限制发送方窗口的最大值

注意：TCP是用字节数作为窗口大小的单位

但是为了方便讨论，我们下面会用报文段的个数作为窗口大小的单位





TCP的运输连接管理





TCP的运输连接管理

连接建立

TCP连接的建立采用客户服务器方式主动发起连接建立的应用进程叫作客户，而被动等待连接建立的应用进程叫作服务器

A的报文和B的报文seq可以不一样，因为tcp是全双工，他们在互相发数据并且捎带确认，所以有独立的序号池

主动打开

CLOSED

SYN报文段不携带数据
但消耗一个序号

确认报文段不携带数据
但消耗一个序号

最后一次握手报文段
可以携带数据
不携带则不消耗序号

SYN-
SENT

ESTAB-
LISHED

SYN=1, seq=x

SYN=1, ACK=1, seq=y, ack=x+1

ACK=1, seq=x+1, ack=y+1

A的seq和B的ack是有关系的
而B的seq和A的ack是有关系的

数据传送

CLOSED

被动打开

LISTEN

SYN-
RCVD

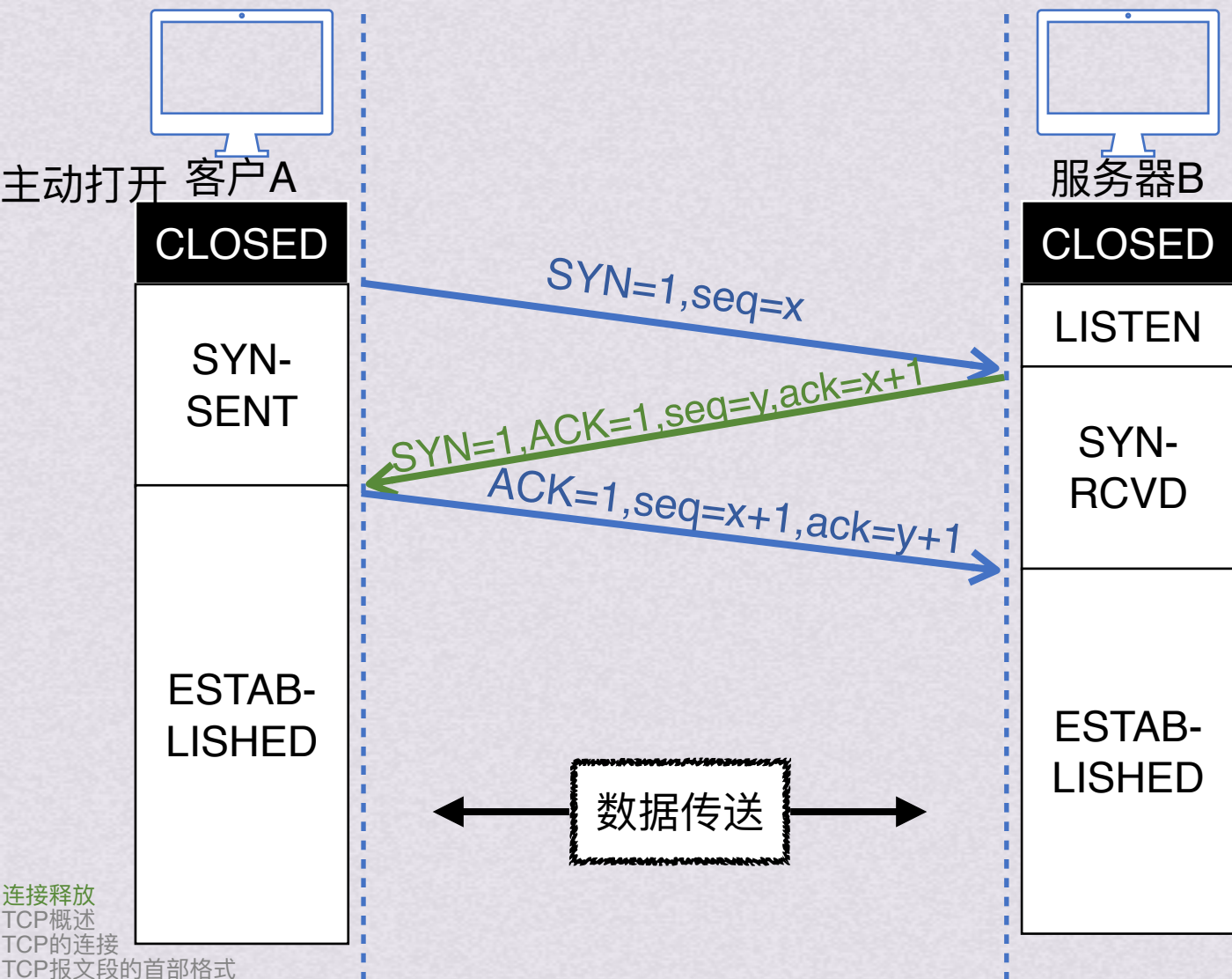
ESTAB-
LISHED



TCP的运输连接管理

连接建立

TCP连接的建立采用客户服务器方式主动发起连接建立的应用进程叫作客户，而被动等待连接建立的应用进程叫作服务器



被动打开

ACK, **ACKn**和**ack=n**的区别

ACK: TCP首部中的字段，仅当ACK=1时确认号字段才有效当ACK=0时，确认号无效TCP规定，连接建立后所有传送的报文段都必须把ACK置1

ACKn: 常用于链路层三个ARQ协议中，ACKn表示对n号帧的确认

ack=n: 到序号n-1为止的所有数据都已经正确收到，下次期望对面发来起始序号为n的数据

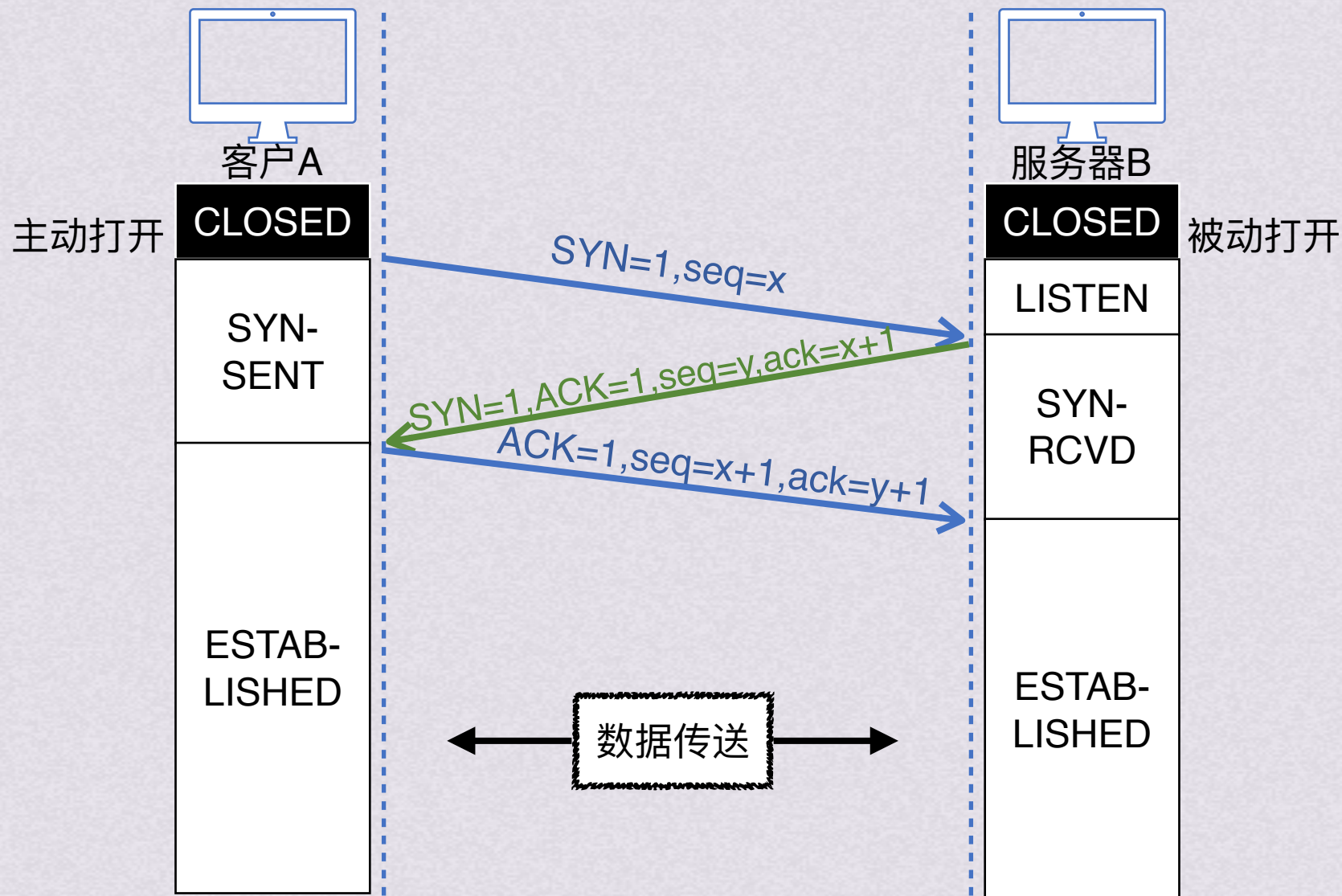
连接释放
TCP概述
TCP的连接
TCP报文段的首部格式
TCP可靠传输的实现
TCP的流量控制
TCP的拥塞控制



TCP的运输连接管理

连接建立

TCP连接的建立采用客户服务器方式主动发起连接建立的应用进程叫作客户，而被动等待连接建立的应用进程叫作服务器

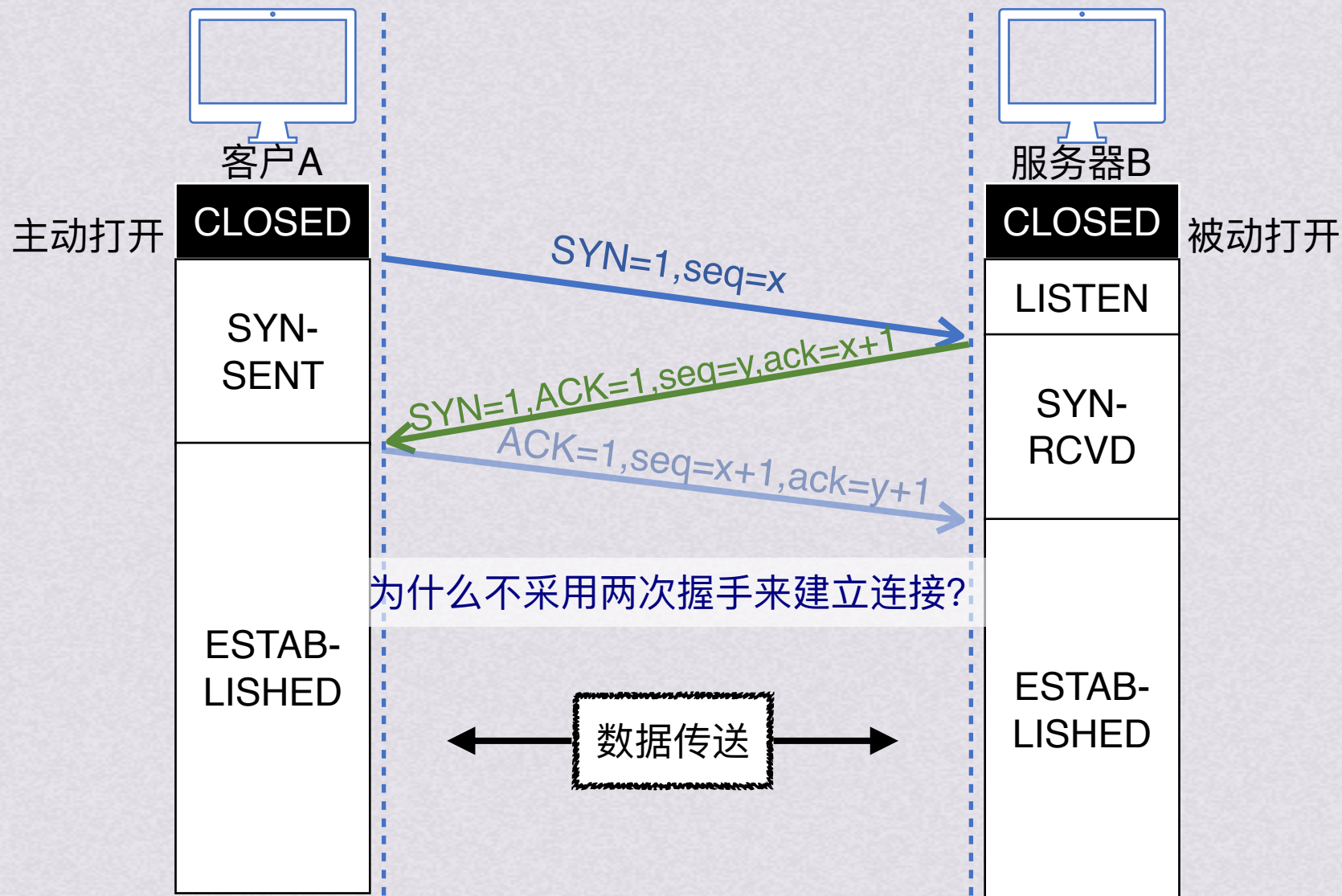




TCP的运输连接管理

连接建立

TCP连接的建立采用客户服务器方式主动发起连接建立的应用进程叫作客户，而被动等待连接建立的应用进程叫作服务器

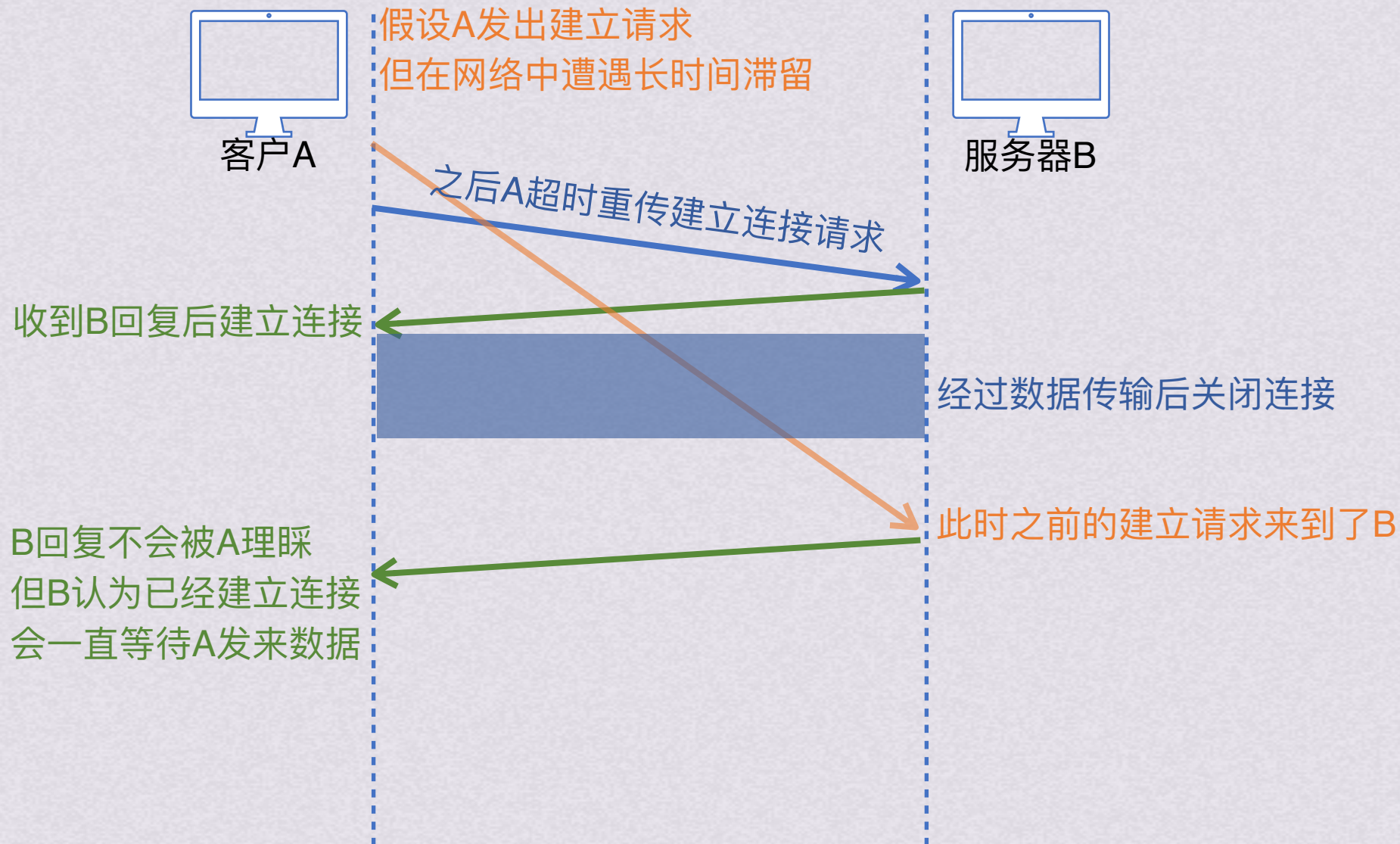




TCP的运输连接管理

连接建立

TCP连接的建立采用客户服务器方式主动发起连接建立的应用进程叫作客户，而被动等待连接建立的应用进程叫作服务器

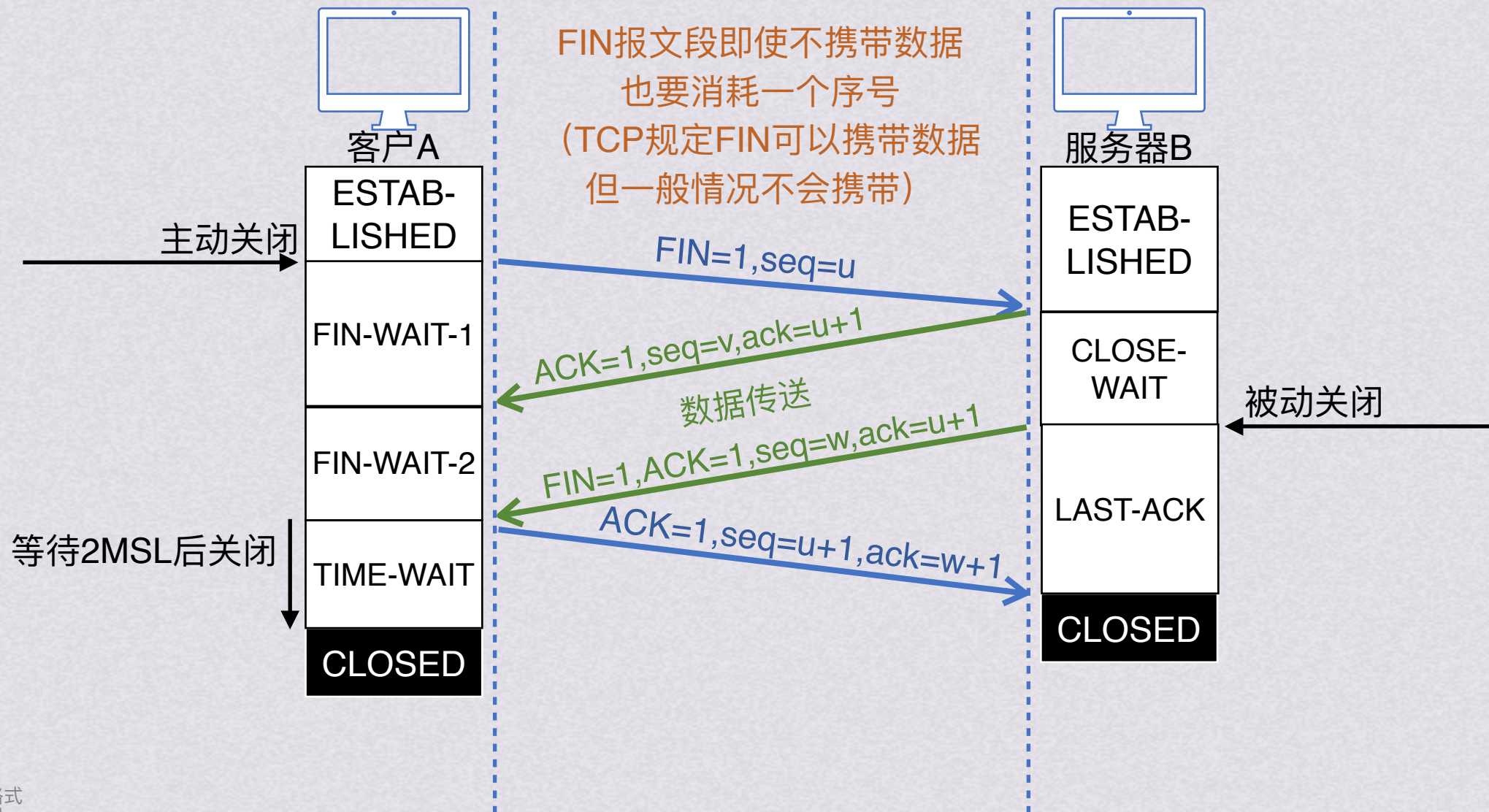




TCP的运输连接管理

连接释放

TCP连接的建立采用客户服务器方式主动发起连接建立的应用进程叫作客户，而被动等待连接建立的应用进程叫作服务器





TCP的运输连接管理

[2011]主机甲向主机乙发送一个(SYN=1,seq=11220)的TCP段, 期望与主机乙建立TCP连接, 若主机乙接受该连接请求, 则主机乙向主机甲发送的正确TCP段可能是

A.(SYN=0,ACK=0,seq=11221,ack=11221)

B.(SYN=1,ACK=1,seq=11220,ack=11220)

C.(SYN=1,ACK=1,seq=11221,ack=11221)

D.(SYN=0,ACK=0,seq=11220,ack=11220)



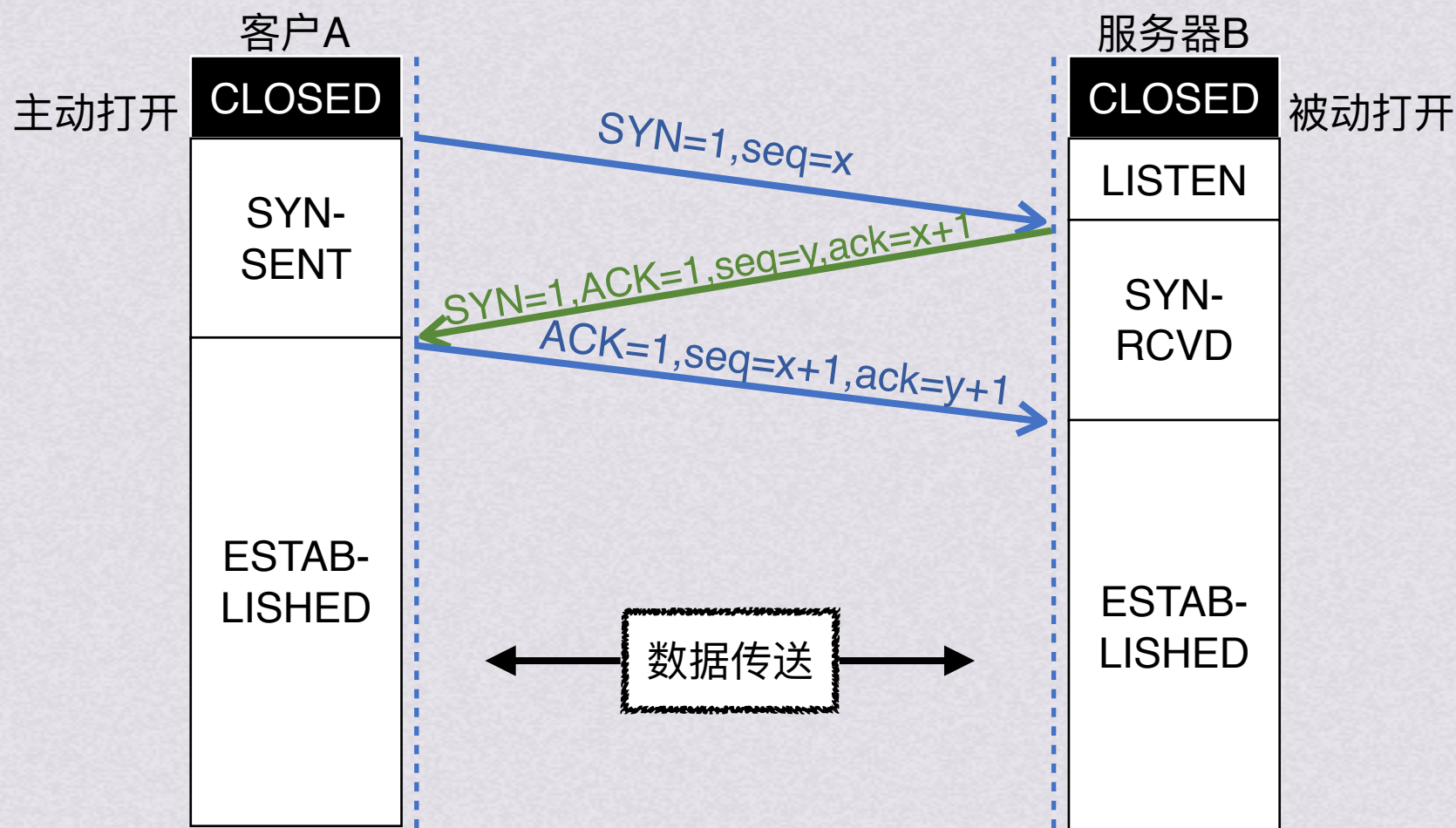
TCP的运输连接管理

[2011]主机甲向主机乙发送一个(SYN=1,seq=11220)的TCP段，期望与主机乙建立TCP连接，若主机乙接受该连接请求，则主机乙向主机甲发送的正确TCP段可能是**C.(SYN=1,ACK=1,seq=11221,ack=11221)**

A.(SYN=0,ACK=0,seq=11221,ack=11221)

B.(SYN=1,ACK=1,seq=11220,ack=11220)

D.(SYN=0,ACK=0,seq=11220,ack=11220)





TCP的运输连接管理

[2009]主机甲乙之间已建立一个TCP连接，主机甲向主机乙发送了两个连续的TCP段，分别包含300B和500B的有效载荷，第一个段的序列号是200，主机乙正确接收到这两个数据段后，发送给主机甲的确认序列号是()

A.500

B.700

C.800

D.1000



TCP的运输连接管理

[2009]主机甲乙之间已建立一个TCP连接，主机甲向主机乙发送了两个连续的TCP段，分别包含300B和500B的有效载荷，第一个段的序列号是200，主机乙正确接收到这两个数据段后，发送给主机甲的确认序列号是(**D.1000**)

A.500

B.700

C.800

ack=n：到序号n-1为止的所有数据都已经正确收到，下次期望对面发来起始序号为n的数据

第一个段序号是200，发送了300B又发送了500B，
分别使用了序号200~499

500~999

因此下一次的确认序号是1000



TCP的运输连接管理

[2022]假设客户C和服务端S已建立一个TCP连接，通信往返时间 $RTT=50ms$ ，最长报文段寿命 $MSL=800ms$ ，数据传输结束后，C主动请求断开连接，若从C主动向S发出FIN段时刻算起，则C和S进入CLOSED状态所需的时间至少分别是

A.850ms,50ms

B.1650ms,50ms

C.850ms,75ms

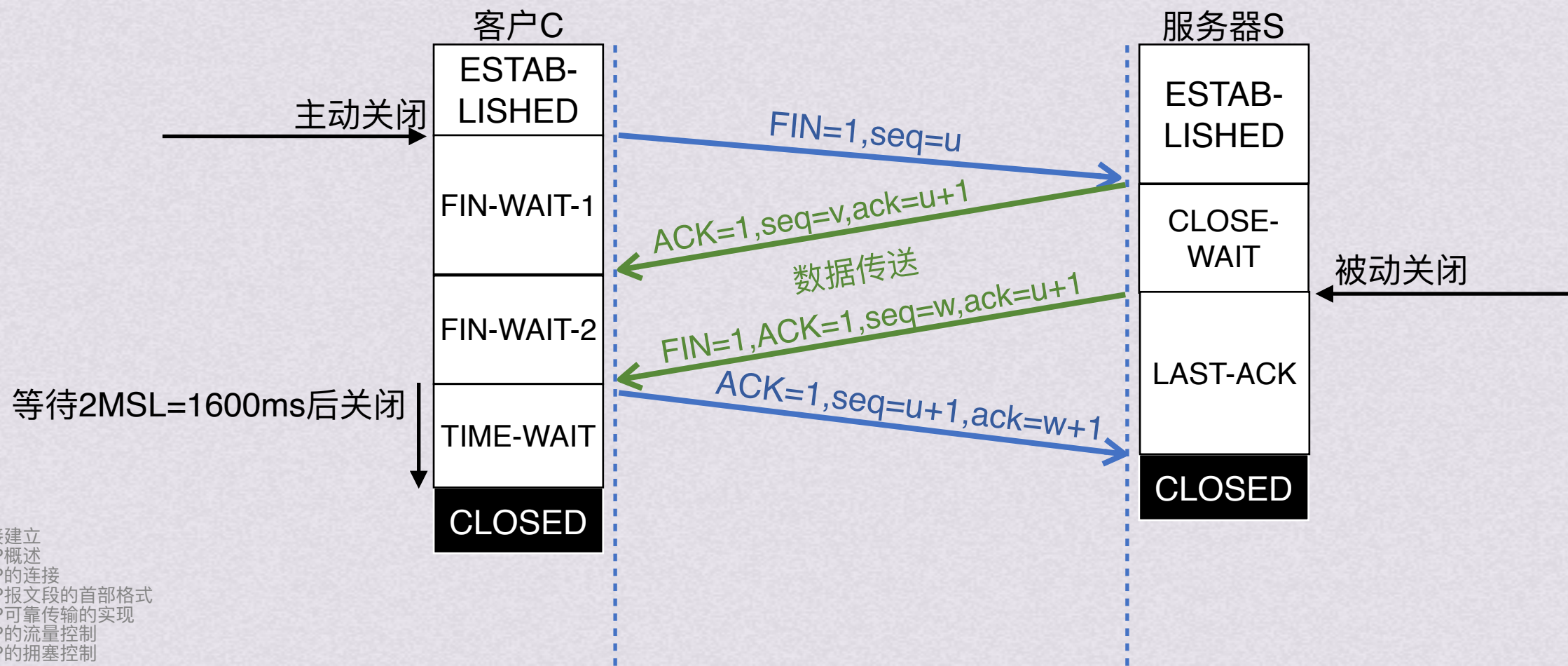
D.1650ms,75ms



TCP的运输连接管理

[2022]假设客户C和服务器S已建立一个TCP连接，通信往返时间RTT=50ms，最长报文段寿命MSL=800ms,数据传输结束后，C主动请求断开连接，若从C主动向S发出FIN段时刻算起，则C和S进入CLOSED状态所需的时间至少分别是 **D.1650ms,75ms**

A.850ms,50ms B.1650ms,50ms C.850ms,75ms

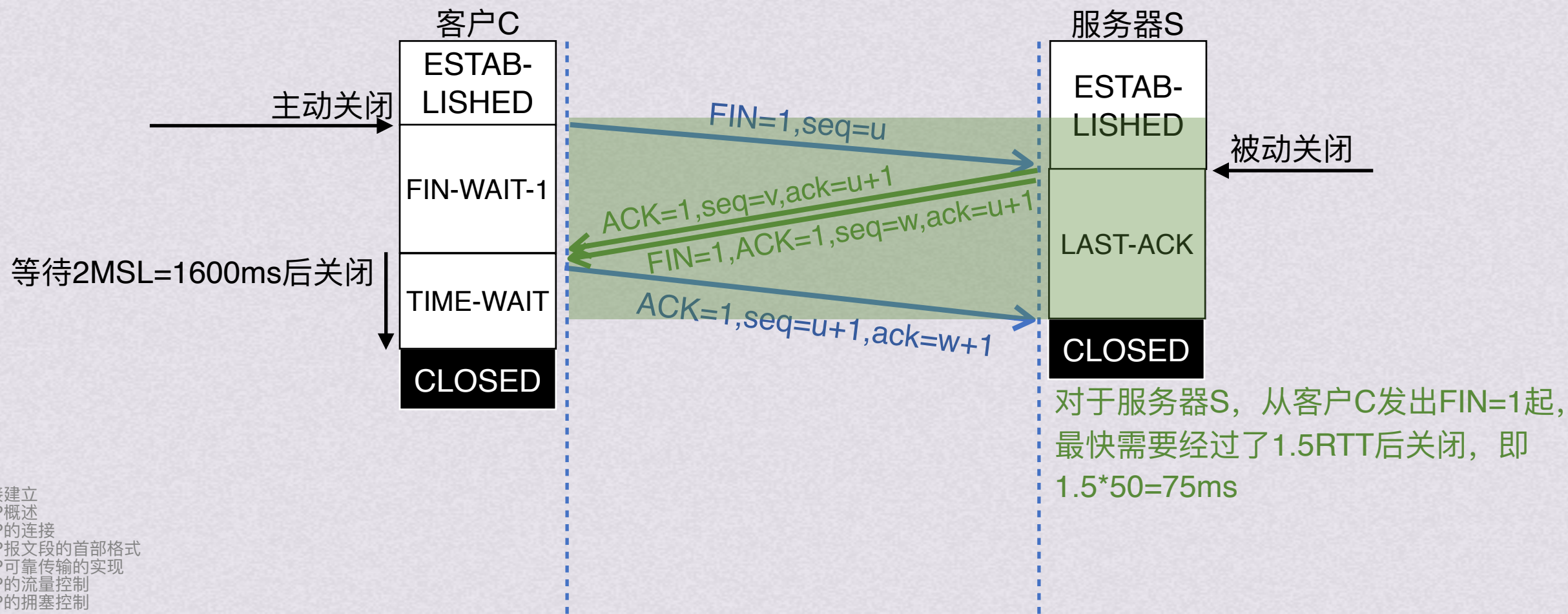




TCP的运输连接管理

[2022]假设客户C和服务器S已建立一个TCP连接，通信往返时间 $RTT=50ms$ ，最长报文段寿命 $MSL=800ms$ ，数据传输结束后，C主动请求断开连接，若从C主动向S发出FIN段时刻算起，则C和S进入CLOSED状态所需的时间至少分别是 **D.1650ms,75ms**

A.850ms,50ms B.1650ms,50ms C.850ms,75ms





TCP的运输连接管理

[2022]假设客户C和服务端S已建立一个TCP连接，通信往返时间 $RTT=50ms$ ，最长报文段寿命 $MSL=800ms$ ，数据传输结束后，C主动请求断开连接，若从C主动向S发出FIN段时刻算起，则C和S进入CLOSED状态所需的时间至少分别是 **D.1650ms,75ms**

A.850ms,50ms B.1650ms,50ms C.850ms,75ms

